

Universidad Internacional de La Rioja

Escuela Superior de Ingeniería y Tecnología

Máster Universitario en Ingeniería de Software y Sistemas Informáticos

Gondor: Certificación Privada de Reputación Financiera mediante ZK-SNARK en Arquitecturas Híbridas Web2/Web3

Trabajo fin de estudio presentado por:	Carlos Serrano Gutiérrez
Tipo de trabajo:	Tipo 1
Director/a:	Miguel Ángel Barcelona Liedana
Fecha:	28/06/2026

Resumen

Las plataformas de *Social Trading* como eToro, Darwinex, ZuluTrade o Wikifolio han facilitado el acceso de inversores minoristas a los mercados financieros, pero su arquitectura centralizada presenta tres problemas estructurales: la amplificación del riesgo por dinámicas de imitación entre usuarios, el bloqueo del historial de rendimiento del inversor en bases de datos propietarias y la distorsión de las métricas públicas por sesgo de supervivencia. Estos problemas, convierten un ecosistema que promete democratización financiera en uno donde la confianza depende de administradores de bases de datos editables.

Este Trabajo de Fin de Máster propone el diseño e implementación de **Gondor**: un prototipo funcional de arquitectura híbrida Web2/Web3 que permite a un inversor construir, certificar y demostrar su reputación financiera de forma soberana, verificable y privada. El sistema combina un motor de gestión de carteras de alta velocidad (**Minas Tirith**) con un procesador criptográfico blockchain (**Pelargir**), que se comunica con Smart Contracts desplegados sobre la testnet Amoy de Polygon. Su innovación central es el paradigma *Verify-me*: los datos se gestionan en infraestructura Web2 para garantizar rendimiento, mientras que su veracidad se certifica en una blockchain pública mediante recibos firmados (EdDSA sobre Poseidon), agrupación en Árboles de Merkle con anclaje on-chain y pruebas de conocimiento cero (ZK-SNARKs, Groth16 sobre BN254). La identidad del inversor se materializa en un *Soulbound Token* (SBT) intransferible, y un mecanismo de *Heartbeat* impide ocultar pérdidas por inactividad.

El desarrollo se organizó en cuatro fases iterativas y se validó con tests unitarios, tests de contratos inteligentes en Hardhat, tests de aceptación end-to-end con Testcontainers y un simulador de estrés. Los resultados demuestran que es técnicamente viable certificar reputación financiera mediante ZK-SNARKs sin exponer datos privados, lo que posiciona a *Gondor* como una contribución original en la intersección entre la ingeniería de software, la criptografía aplicada y las finanzas descentralizadas.

Abstract

Social Trading platforms such as eToro, Darwinex, ZuluTrade and Wikifolio have broadened retail access to financial markets, yet their centralized architecture presents three structural problems: risk amplification through user herding dynamics, the lock-in of investor performance history within proprietary databases, and the distortion of public metrics through survivorship bias. These issues, turn an ecosystem that promises financial democratization into one where trust depends on administrators of mutable databases.

This Master's Thesis proposes the design and implementation of **Gondor**: a functional prototype with a hybrid Web2/Web3 architecture that enables an investor to build, certify, and prove their financial reputation in a sovereign, verifiable, and privacy-preserving manner. The system combines a high-speed portfolio management engine (**Minas Tirith**) with a blockchain cryptographic processor (**Pelargir**), that communicate with Smart Contracts deployed on the Polygon Amoy testnet. Its core innovation is the *Verify-me* paradigm: data is managed within Web2 infrastructure for performance, while its truthfulness is certified on a public blockchain through signed receipts (EdDSA over Poseidon), Merkle Tree batching with on-chain anchoring, and Zero-Knowledge Proofs (ZK-SNARKs, Groth16 over BN254). The investor's identity is materialized as a non-transferable Soulbound Token (SBT), and a Heartbeat mechanism prevents the concealment of losses through inactivity.

Development was organized into four iterative phases and validated through unit tests, Hardhat smart contract tests, end-to-end acceptance tests using Testcontainers, and a stress simulator. Results demonstrate that certifying financial reputation via ZK-SNARKs without exposing private data is technically feasible, positioning the Gondor as an original contribution at the intersection of software engineering, applied cryptography, and decentralized finance.

Keywords: Social Trading, Blockchain, Zero-Knowledge Proofs (ZK-SNARKs), Soulbound Token (SBT), Sovereign Financial Reputation

Índice de contenidos

1. Introducción 1
1.1. Justificación 1
1.1.1. Riesgo excesivo por dinámicas sociales 1
1.1.2. El historial como propiedad de la plataforma 2
1.1.3. El sesgo de supervivencia 3
1.2. Planteamiento del problema 3
1.3. Estructura del trabajo 4
2. Contexto y estado del arte 6
2.1. Contexto 6
2.1.1. El Social trading y sus roles 6
2.1.2. Los silos de datos y el sesgo de supervivencia institucionalizado 6
2.1.3. De la base de datos editable al registro inmutable 7
2.2. Arquitecturas de software y patrones de concurrencia para sistemas financieros 8
2.2.1. Arquitectura hexagonal y el aislamiento del dominio 8
2.2.2. Monolito Modular vs. Microservicios: Estrategia de despliegue evolutivo 10
2.2.3. Patrones de concurrencia: el modelo CSP y su aplicabilidad transaccional 11
2.3. Blockchain e identidad descentralizada 12
2.3.1. Infraestructura de ejecución: EVM y el ecosistema Polygon 13
2.3.2. El Estandar ERC-721 y sus limitaciones para la Identidad Digital 14
2.3.3. Soulbound Tokens (SBTs): Reputación Intransferible en la Sociedad Descentralizada 15
2.3.4. Identificadores Descentralizados (DIDs) y Credenciales Verificables (VCs) 16
2.4. Pruebas de conocimiento cero (ZK-Proofs) 16

2.4.1. Fundamentos teóricos 16
2.4.2. ZK-SNARKs frente a ZK-STARKs: arquitectura y justificación de Groth16 18
2.4.3. Estructuras de datos autenticadas y primitivas hash orientadas a ZK 19
2.4.4. Ecosistema de compilación: justificación de Circom y SnarkJS 19
2.5. Comparativa de Soluciones en Identidad Descentralizada y Reputación Web3 21
2.5.1. Arquitecturas de Identidad Social y Resistencia Sybil 21
2.5.2. Protocolos de agregación de credenciales 22
2.5.3. Infraestructura de oráculos y puente Web2-Web3 23
2.5.4. Calificación crediticia descentralizada 23
2.5.5. Marcos institucionales: EBSI 24
2.5.6. Tabla comparativa 25
2.6. Conclusión 27
3. Objetivos concretos y metodología de trabajo 29
3.1. Objetivo general 29
3.2. Objetivos específicos 29
3.3. Metodología del trabajo 30
3.3.1. Modelo de desarrollo: iterativo e incremental 30
3.3.2. Nomenclatura del proyecto 32
3.3.3. Tecnologías principales y justificación 32
3.3.4. Herramientas de desarrollo y testing 33
3.3.5. Gestión del proyecto 34
3.3.6. Organización del código fuente 34
3.3.7. Correspondencia entre objetivos específicos y fases 34
4. Desarrollo específico de la contribución 35

4.1. Primer movimiento: La construcción del core inanciero Web2 36
4.1.1. Identificación de requisitos y diseño 36
4.1.2. Implementación de los servicios de dominio 38
4.1.3. Inicialización y estrategia de caché: Redis como fuente de verdad 39
4.1.4. Desarrollo del motor concurrente: el patrón Fan-in / Fan-out 40
4.1.5. La gestión de múltiples proveedores con rotación por cuota 41
4.1.6. Tolerancia a fallos con el fallback 42
4.1.7. Evaluación de la fase 43
4.2. Segundo movimiento: los cimientos de la Identidad Soberana 45
4.2.1. Identificación de requisitos y diseño 45
4.2.2. Diseño del Smart Contract 46
4.2.3. Implementación del módulo Pelargir en Go 48
4.2.4. Evaluación de la fase 51
4.3. Tercer Movimiento: la certificación y verificabilidad criptográfica 52
4.3.1. Identificación de requisitos y diseño 52
4.3.2. Recibos firmados deterministas 54
4.3.3. Merkle Batching y anclaje on-chain 55
4.3.4. Pruebas de Conocimiento Cero (ZK-SNARKs) 57
4.3.5. El mecanismo de Heartbeat activo 61
4.3.6. Ciclo de vida de una transacción 62
4.3.7. Tests de aceptación con Testcontainers 63
4.3.8. Evaluación de la fase 65
4.4. Cuarto Movimiento: la interfaz del usuario 65
4.4.1. Identificación de requisitos y diseño 65

4.4.2. La aplicación móvil 66
4.4.3. Evaluación de la fase 67
4.5. Evaluación experimental y análisis de viabilidad 68
4.5.1. Entorno de Experimentación 68
4.5.2. Costes de Gas On-Chain 68
4.5.3. Latencia de Generación de Pruebas ZK 69
4.5.1. Tamaño de Artefactos Criptográficos 70
4.5.2. Evaluación Comparativa con el Estado del Arte 70
5. Conclusiones y trabajo futuro 71
5.1. Conclusiones 71
5.1.1. Grado de consecución de los objetivos específicos 72
5.1.2. Contribución principal 73
5.2. Trabajo futuro 73
Referencias bibliográficas 75

Índice de figuras

Figura 2-1: Representación de la arquitectura hexagonal. Fuente: Medium 9

Figura 4-1: Arquitectura del sistema Gondor. 36

Figura 4-2: Arquitectura del módulo Minas Tirith. 39

Figura 4-3: Diagrama de flujo para la gestión de precios de las acciones. 40

Figura 4-4: Diagrama de actividad de la obtención de los precios de las acciones. 41

Figura 4-5: Diagrama de flujo del algoritmo de selección de proveedores 43

Figura 4-6: Arquitectura de Gondor después de la fase 2 50

Figura 4-7: Diagrama de secuencia de la creación de la Identidad Soberana. 51

Figura 4-8: Diagrama de flujo de la arquitectura Verify-me. 54

Figura 4-9: Diagrama de secuencia del anclaje de las transacciones de un usuario 56

Figura 4-10: Diagrama de flujo de las restricciones que ejecuta el circuito ZK-SNARK. 58

Figura 4-11: Diagrama de secuencia de la generación y verificación de la reputación. 60

Figura 4-12: Funcionamiento del hearbeat activo 62

Figura 4-13: Arquitectura de los test de aceptación con TestContainers 64

Figura 4-14: Interfaz de usuario de la aplicación Gondor 67

Índice de tablas

Tabla 1: Tabla comparativa de soluciones existentes 26

Tabla 2: Fases del desarrollo iterativo del Ecosistema Gondor y sus dependencias 31

Tabla 3: Tabla resumen de la justificación tecnológica 33

Tabla 4: Organización del código fuente en repositorios GitHub 34

Tabla 5: Correspondencia entre objetivos específicos y fases de desarrollo 35

Tabla 6: Tiempos de cada operación en el flujo de obtención de precios. 44

Tabla 7: Estructura del circuito ZK-SNARK 58

Tabla 8: Estados obserbables de cada transacción. 63

Tabla 9: Consumo de Gas por Operación On-Chain (Nodo Local) 69

Tabla 10: Latencia en milisegundos de Generación de Prueba ZK 69

Tabla 12: Tamaño de Artefactos del Sistema ZK-SNARK (Groth16/BN254) 70

Tabla 11: Comparativa Cuantitativa: Gondor vs. Soluciones del Estado del Arte 71

1. INTRODUCCIÓN

1.1.JUSTIFICACIÓN

Las plataformas de *Social Trading* como eToro o Darwinex han transformado el acceso minorista a los mercados financieros mediante la integración de redes sociales e infraestructuras de ejecución algorítmica (Apesteguia, Oechssler, & Weidenholzer , 2020). Su mecanismo nuclear, el *Copy Trading*, replica automáticamente las operaciones de un gestor líder en las cuentas de sus seguidores, proporcional al capital asignado. Esta arquitectura Web2 centralizada, pese a prometer democratización del conocimiento inversor, presenta tres disfunciones estructurales que motivan la investigación y el diseño del .

1.1.1. Riesgo excesivo por dinámicas sociales

Las interfaces de estas plataformas están deliberadamente diseñadas para maximizar el engagement mediante *leaderboards*, notificaciones de operaciones exitosas y métricas de rentabilidad a corto plazo. Esta gamificación perturba la racionalidad financiera del usuario (Apesteguia, Oechssler, & Weidenholzer , 2020), en un entorno de laboratorio controlado, demostraron empíricamente que la mera exposición al éxito financiero de otros agentes incrementa de forma estadísticamente significativa la propensión al riesgo del observador, y que dicho incremento se magnifica drásticamente cuando se habilita la opción de copiar carteras automáticamente. El *Copy Trading*, lejos de fomentar la diversificación prudente, induce a la imitación de conductas atípicas temporalmente premiadas por la varianza del mercado.

A nivel macroestructural, esta propensión a la imitación engendra el fenómeno de comportamiento gregario (herding behavior): el capital migra masivamente hacia los operadores con mayor rendimiento reciente, independientemente de que dicho rendimiento sea estructuralmente frágil. Ante un cambio de régimen de mercado, las liquidaciones forzosas de los líderes desencadenan cascadas de liquidaciones simultáneas en miles de cuentas copiadoras. El sistema falla por diseño: la arquitectura de incentivos fomenta la persecución del rendimiento cortoplacista para adquirir visibilidad. (Apesteguia, Oechssler, & Weidenholzer , 2020).

La presión del escrutinio público agrava adicionalmente la conducta del propio gestor. Pelster y Hofmann demostraron econométricamente, mediante un diseño de diferencias en diferencias sobre datos reales de plataformas, que adquirir el estatus de líder copiado incrementa de forma drástica la susceptibilidad al efecto disposición: la tendencia irracional a cerrar prematuramente posiciones ganadoras y mantener abiertas las perdedoras para evitar reconocer públicamente el error (Pelster & Hofmann, 2018). Este sesgo se correlaciona directamente con el volumen de capital encomendado. La consecuencia arquitectónica es que las plataformas Web2 recompensan el camuflaje de pérdidas, dado que las métricas de *win rate* se calculan sobre operaciones cerradas, y castigan la gestión de riesgo prudente.

1.1.2. El historial como propiedad de la plataforma

Cuando un gestor construye durante años su historial de operaciones en una plataforma optimizando su Ratio de Sharpe, gestionando *drawdowns* y cultivando una comunidad de seguidores, genera un activo digital de valor demostrable. Sin embargo, las plataformas Web2 almacenan ese historial en bases de datos relacionales cerradas bajo políticas corporativas propietarias, convirtiéndolo en un activo cautivo (Engels, 2016).

La gravedad de esta pérdida queda acreditada por Wohlgemuth, Berger y Wenzel, quienes demostraron mediante *fuzzy-set Qualitative Comparative Analysis* (fsQCA) aplicado sobre datos de eToro que la confianza inversora no es unidimensional: requiere simultáneamente señales cognitivas (ROI histórico, antigüedad de cuenta, ratio de operaciones ganadoras) y señales afectivas (consistencia comunicativa, transparencia ante adversidad). Si un gestor migra a otra plataforma, aparece con antigüedad cero y sin historial auditable, situándose automáticamente en una de las seis configuraciones de fallo de confianza identificadas por el modelo. El *switching cost* reputacional actúa así como una barrera de salida insuperable (Wohlgemuth, Berger, & Wenzel, 2016).

El GDPR (Artículo 20) y el marco Open Banking (PSD2) no resuelven este problema. Cuando el usuario ejerce el derecho de portabilidad, las plataformas entregan volcados estáticos en formato CSV. Estos archivos carecen de certificación criptográfica de inmutabilidad: cualquier tercero puede alterarlos localmente antes de presentarlos a una plataforma competidora, por lo que ningún algoritmo de evaluación de riesgo les otorga valor probatorio (Politou, Alepis, & Patsakis, 2021). **La portabilidad legal existe; la portabilidad reputacional verificable, no**.

1.1.3. El sesgo de supervivencia

La tercera falla estructural reside en la manipulación estadística de la muestra disponible para el inversor. Las infraestructuras Web2 operan sobre bases de datos CRUD (*Create, Read, Update, Delete*), lo que permite eliminar o desindexar los perfiles de cuentas liquidadas de las interfaces públicas y los *rankings*. El inversor minorista observa únicamente a los supervivientes, ignorando el universo completo de operadores que fracasaron bajo estrategias paramétricamente equivalentes.

Este fenómeno fue formalizado en la literatura financiera (Elton, Gruber, & Blake, 2015), donde se demostró que los cálculos de rendimiento de fondos de inversión basados en muestras supervivientes sobreestimaban sistemáticamente las tasas de rentabilidad promedio y deformaban las métricas de correlación inter-activos. Gran parte del *alpha* gerencial aparente era un espejismo producido por la depuración selectiva de datos. Las plataformas de *Social Trading* replican y amplifican esta disfunción a escala minorista: exhiben rendimientos de dobles y triples dígitos sin rastro del "cementerio de cuentas liquidadas", induciendo a los copistas a subestimar el riesgo de cola (*tail risk*) y a atribuir a habilidad estructural lo que es varianza estadística favorable (Handy & Meksi, 2025).

1.2. PLANTEAMIENTO DEL PROBLEMA

La industria del *Social Trading* carece, en la actualidad, de un mecanismo técnico que garantice simultáneamente tres propiedades fundamentales: la soberanía del operador sobre su historial de rendimiento, la integridad matemáticamente verificable de dicho historial, y la privacidad operacional del gestor durante la evaluación de sus métricas. La pregunta de investigación que articula el presente TFM es, por tanto, la siguiente: **¿Es posible diseñar un sistema híbrido de gestión de carteras de inversión que sea inmutable, verificable de forma autónoma e independiente de la plataforma Web2 subyacente, sin sacrificar el rendimiento operacional y preservando al mismo tiempo la privacidad del gestor frente al escrutinio social?**

Para dar respuesta a este interrogante, se propone el diseño e implementación del **Ecosistema Gondor**: una arquitectura tecnológica híbrida, concebida para trasladar la reputación de un modelo de "*Confianza Institucional*" (Web2) a uno de "*Verificabilidad Matemática*" (Web3). La solución articula dos módulos con responsabilidades segregadas y complementarias.

El primer módulo, **Minas Tirith (Core Web2)**, es el núcleo de gestión transaccional de alta velocidad: lógica de negocio financiera, precios en tiempo real mediante Redis y persistencia en MySQL. El segundo módulo, **Pelargir (Coprocesador Web3)**, asume la función de certificación criptográfica: recibos firmados, agrupamiento en Árboles de Merkle con anclaje en Polygon, circuitos ZK-SNARK para la certificación verificable del ROI, y *minting* de **Soulbound Tokens (SBTs)** intransferibles. Un mecanismo de *Heartbeat* impide la congelación indefinida de la reputación, y la doble métrica on-chain expone públicamente cualquier sesgo de selección transaccional.

La innovación central de la arquitectura reside en el desacople entre la gestión eficiente del dato (realizada en Web2 para garantizar rendimiento) y la certificación matemática de la verdad (realizada en Web3 para garantizar inmutabilidad e independencia de plataforma). Este paradigma *Verify-me* elimina la necesidad de confianza ciega en el administrador del sistema: cualquier tercero puede verificar de forma autónoma la integridad del historial de un operador confrontando las raíces de Merkle ancladas en Polygon con los hashes de las transacciones locales.

El alcance del presente TFM se delimita explícitamente al diseño, implementación y validación de un prototipo funcional (Prueba de Concepto, PoC) de Gondor, desplegado sobre la red de pruebas (*testnet*) Amoy de Polygon. Quedan expresamente fuera del alcance de esta investigación: (1) el despliegue del componente Web3 en la red principal (*mainnet*) de Polygon, así como el despliegue de la aplicación Web2 en un entorno de producción no local; (2) la realización de auditorías de seguridad, abarcando los *Smart Contracts*, los circuitos ZK-SNARK y el servidor Web2; y (3) la integración con brokers de inversión reales o con plataformas de *Social Trading* existentes, incluyendo el acceso a APIs de enrutamiento de órdenes en mercados financieros. El PoC opera con datos introducidos por los usuarios y precios obtenidos de fuentes públicas, validando la arquitectura propuesta en un entorno controlado y técnicamente representativo.

1.3. ESTRUCTURA DEL TRABAJO

La memoria se organiza en cinco capítulos con una progresión lógica desde el análisis del problema hasta la evaluación de la solución.

El **Capítulo 1 (Introducción)** justifica el trabajo analizando tres problemas del *Social Trading* actual, plantea la pregunta de investigación y describe la estructura de la memoria.

El **Capítulo 2 (Contexto y estado del arte)** establece el marco conceptual y tecnológico. Comienza con el contexto del dominio del *Social Trading* y sus limitaciones, y profundiza en cuatro áreas. El capítulo concluye identificando el vacío tecnológico que el Ecosistema Gondor propone cubrir.

El **Capítulo 3 (Objetivos concretos y metodología de trabajo)** formula el objetivo general, lo descompone en seis objetivos específicos, describe la metodología iterativa organizada en cuatro fases, justifica la selección tecnológica y detalla la estrategia de testing y la gestión del proyecto.

El **Capítulo 4 (Desarrollo específico de la contribución)** es el núcleo de la memoria y documenta el diseño, implementación y evaluación de cada una de las cuatro fases.

El **Capítulo 5 (Conclusiones y trabajo futuro)** vincula cada objetivo con los resultados obtenidos, identifica la contribución principal del trabajo y propone líneas futuras.

2. CONTEXTO Y ESTADO DEL ARTE

2.1.CONTEXTO

Antes de abordar el análisis técnico en profundidad de las soluciones existentes y de la arquitectura propuesta, resulta imprescindible establecer los conceptos de dominio sobre los que se construye el presente trabajo. Familiarizando al lector con conceptos específicos de los mercados financieros y los paradigmas de la descentralización tecnológica.

2.1.1. El Social trading y sus roles

El *Social Trading* es un modelo de participación en los mercados financieros que integra las mecánicas de interacción propias de las redes sociales con la infraestructura de operativa bursátil. En términos prácticos, es una plataforma donde los usuarios pueden observar, comentar, evaluar y replicar las decisiones de inversión de otros operadores en tiempo real. El *Copy Trading* es la capa de ejecución de este modelo: un sistema de *middleware* que vincula las cuentas de corretaje de dos participantes de modo que cualquier apertura, modificación o cierre de posición ejecutada por uno de ellos se replique automáticamente, y con latencia de milisegundos, en la cuenta del otro.

En este ecosistema se distinguen dos roles primarios. El *líder*, es el operador cuyas decisiones de inversión son copiadas. Su incentivo económico proviene de las comisiones cobradas en función del volumen de activos que terceros le confían, o bien de cuotas de suscripción. El *seguidor* es el inversor que delega su agencia de inversión en uno o varios líderes, beneficiándose potencialmente de estrategias complejas sin necesidad de operar directamente. La métrica fundamental que gobierna la relación entre ambos roles es el ROI (*Return on Investment*, o Retorno sobre la Inversión), que expresa el rendimiento porcentual de la cartera respecto al capital inicial invertido y actúa como el indicador de credibilidad primario del líder ante sus potenciales seguidores.

2.1.2. Los silos de datos y el sesgo de supervivencia institucionalizado

En el contexto del *Social Trading*, un silo de datos es la base de datos relacional cerrada y propietaria que cada plataforma centralizada mantiene sobre sus servidores. Los sistemas relacionales son, por naturaleza, bases de datos editables, lo que significa que cualquier registro, puede ser creado, leído, modificado o eliminado por los administradores del sistema.

Esta capacidad de borrado no es un detalle menor: es el mecanismo técnico que hace posible el sesgo de supervivencia institucionalizado. Cuando un operador pierde toda su cartera, sufre una liquidación forzosa o simplemente acumula pérdidas inaceptables, la plataforma puede ejecutar un `DELETE` o desindexar su perfil, haciéndolo invisible al resto de usuarios. El inversor que consulta el leaderboard nunca ve el "cementerio" de cuentas liquidadas; solo ve a los supervivientes actuales, lo que produce una percepción artificialmente optimista del riesgo. Tomar decisiones basadas únicamente en la distribución de los supervivientes, ignorando a quienes fracasaron con estrategias paramétricamente idénticas.

Adicionalmente, el historial de rendimiento de un gestor queda cautivo en el silo de datos de la plataforma donde opera. Si el gestor decide migrar a un competidor, su historial no es transferible de forma creíble: los archivos CSV que el GDPR permite exportar son registros estáticos sin firma criptográfica, por lo que cualquier plataforma receptora debe asumir que podrían haber sido manipulados. El resultado práctico es que la reputación acumulada durante años de trabajo se destruye completamente al cambiar de plataforma, generando un efecto de vendor lock-in de naturaleza reputacional.

2.1.3. De la base de datos editable al registro inmutable

Una base de datos relacional tradicional es mutable por diseño: su estado en cualquier instante es el resultado de todas las operaciones CRUD aplicadas hasta ese momento, y el pasado puede ser reescrito por quien tenga privilegios de administración. Un registro de tipo *append-only*, por el contrario, solo permite añadir nuevas entradas; nunca eliminar ni modificar las existentes. Cada nueva entrada referencia criptográficamente a la anterior mediante su hash, formando así una cadena en la que la alteración retroactiva de cualquier registro invalidaría inmediatamente todos los registros posteriores, haciéndola computacionalmente indetectable solo si se dispone de una capacidad de cómputo prácticamente infinita.

Una blockchain es, en esencia, este tipo de registro inmutable distribuido entre múltiples participantes independientes (nodos) que validan colectivamente la integridad de cada nueva entrada mediante mecanismos de consenso criptográfico. Las consecuencias prácticas de este paradigma, a muy alto nivel, son tres: (1) inmutabilidad, ya que ningún actor individual, incluidos los operadores de la plataforma, puede alterar el historial registrado; (2) desintermediación, ya que la confianza no reside en una institución central sino en el

protocolo matemático y en la descentralización de la red; y (3) confianza criptográfica, ya que cualquier participante puede verificar de forma autónoma e independiente la veracidad de cualquier registro sin necesidad de solicitar permiso a ninguna autoridad central.

Aplicado al problema del Social Trading, este cambio de paradigma significa que el historial de operaciones de un gestor podría anclarse en un registro inmutable, accesible públicamente, donde ningún administrador de plataforma pueda borrar las pérdidas ni fabricar una apariencia de infalibilidad. La reputación dejaría de ser un activo cautivo en el silo de datos de un proveedor para convertirse en un activo soberano del propio gestor, verificable por cualquier tercero en cualquier momento.

2.2. ARQUITECTURAS DE SOFTWARE Y PATRONES DE CONCURRENCIA PARA SISTEMAS FINANCIEROS

Las plataformas financieras operan bajo un paradigma de exigencia que supera con creces al del software de propósito general: deben procesar volúmenes masivos de transacciones con latencias ultrabajas, garantizar la consistencia absoluta e inmutable de los datos y escalar de forma elástica ante la volatilidad de los mercados.

2.2.1. Arquitectura hexagonal y el aislamiento del dominio

2.2.1.1. Fundamentos teóricos

El patrón de Arquitectura Hexagonal, formalmente denominado *Ports and Adapters*, fue conceptualizado por Alistair Cockburn en 2005 al observar el problema crónico de la infiltración de la lógica de negocio en las capas de interfaz y de acceso a datos. Este acoplamiento genera tres disfunciones críticas en entornos financieros: (1) la imposibilidad de ejecutar pruebas automatizadas aisladas sin levantar bases de datos o sistemas externos; (2) una rigidez operativa que impide reutilizar las mismas reglas de negocio en contextos distintos, por ejemplo, una transferencia en tiempo real y una conciliación nocturna; y (3) la incapacidad de conducir el programa de manera programática desde sistemas externos (Cockburn, 2024). La solución propuesta consiste en diseñar la aplicación de modo que funcione de forma idéntica independientemente de si es impulsada por un usuario humano, un script automatizado, una suite de pruebas o un flujo de mensajes (Cordeiro Ferreira, Dhinavahi, Trapmann, & van den Heuvel, 2025).

La arquitectura divide el sistema en tres componentes invariables:

- **Núcleo:** contiene la lógica de negocio pura, las entidades y los casos de uso, modelados según los principios del *Domain-Driven Design* (DDD). En un sistema de compensación de pagos, aquí residen las abstracciones de cuentas, saldos, débitos y créditos, completamente ignorantes de los detalles de infraestructura (Cockburn, 2024).
- **Puertos:** definen los contratos formales, expresados como interfaces, para cualquier interacción entre el núcleo y sus dependencias externas. Pertenecen exclusivamente al dominio de la aplicación, no a la infraestructura, y se clasifican en puertos de entrada (*driving ports*) y puertos de salida (*driven ports*).
- **Adaptadores:** son las implementaciones concretas que orbitan el hexágono. Un adaptador de entrada traduce señales externas (peticiones HTTP, mensajes gRPC) en llamadas tipadas a los puertos, mientras que un adaptador de salida transforma solicitudes abstractas del núcleo en las señales requeridas por tecnologías específicas (SQL, MongoDB, etc.).

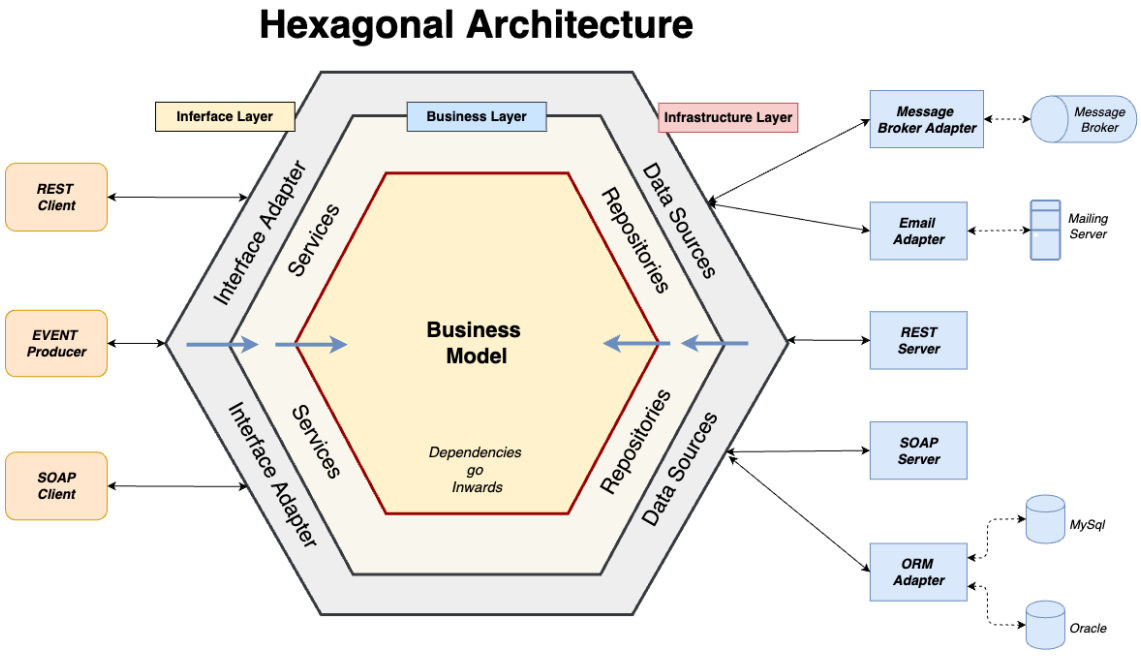


Figura 2-1: Representación de la arquitectura hexagonal. Fuente: Medium

El resultado es un desacople tecnológico absoluto: migrar de Oracle a Cassandra, o de SOAP a GraphQL, requiere únicamente reescribir el adaptador periférico correspondiente, dejando la lógica, que puede representar décadas de refinamiento, inalterada (Sabbag Filho, 2025).

2.2.1.2. Contextualización comparativa

La arquitectura clásica en capas (N-Tier) divide el sistema en Presentación, Lógica de Negocio y Acceso a Datos con dependencias estrictamente descendentes (Sabbag Filho, 2025). Su defecto estructural es la transitividad de dependencias: la capa de negocio acaba dependiendo, indirectamente, de la tecnología de persistencia, generando un diseño data-driven incompatible con la pureza del dominio y contaminando las entidades con anotaciones ORM.

La Onion Architecture y la Clean Architecture comparten la misma premisa ontológica que la propuesta de Cockburn, aislar el dominio en el núcleo y dirigir todas las dependencias hacia adentro, pero son altamente prescriptivas al imponer anillos concéntricos rígidos (Nunkesser, 2022). La literatura señala que la implementación de Clean Architecture plantea problemas derivados de la ausencia de implementaciones de referencia unificadas, lo que ha dado lugar a variantes que sacrifican la independencia e introducen un mapeo constante e ineficiente de estructuras de datos entre cada anillo (Nunkesser, 2022). Para sistemas financieros de backend, la Arquitectura Hexagonal en su forma más pura resulta más pragmática: su dicotomía binaria (aplicación vs. mundo exterior) permite crear mocks de sistemas transaccionales de terceros, de manera natural durante la definición de los puertos de salida, blindando la lógica algorítmica central frente a indisponibilidades externas (Cockburn, 2024).

2.2.2. Monolito Modular vs. Microservicios: Estrategia de despliegue evolutivo

Estudios sistemáticos recientes documentan que adoptar microservicios de forma prematura introduce una complejidad operativa devastadora en proyectos en fase de inicio. La Arquitectura del Monolito Modular (MMA, *Modular Monolith Architecture*) ha resurgido como el estado del arte preferencial para el diseño evolutivo de plataformas complejas, fundamentada en la partición de *Bounded Contexts* derivada del DDD (Al-Qora & Al-Said Ahm, 2025).

Sus ventajas sobre los microservicios en etapas tempranas son tres:

1. **Eliminación del overhead distribuido.** La comunicación entre dominios fragmentados viaja a través de la red, introduciendo latencia acumulada, fallos transitorios y la obligación de implementar patrones defensivos (*Circuit Breakers*, Sagas). En el MMA, la comunicación entre módulos ocurre mediante llamadas a funciones en el mismo

espacio de memoria, logrando un rendimiento órdenes de magnitud superior y sin dependencia de la topología de red.

2. **Reducción drástica de la complejidad de infraestructura.** Los microservicios exigen orquestadores maduros (Kubernetes) y plataformas de observabilidad distribuida. El MMA se compila y despliega como un único artefacto, simplificando los pipelines CI/CD y concentrando el esfuerzo en la lógica de valor.
3. **Preservación de la transaccionalidad ACID.** La atomicidad sobre múltiples operaciones simultáneas es el requerimiento de mayor criticidad en finanzas. Al ejecutar todos los módulos en el mismo proceso, el MMA retiene el control transaccional ACID local durante más tiempo, posponiendo la complejidad de la consistencia eventual inherente a los sistemas distribuidos.

La preparación para una futura migración hacia microservicios depende matemáticamente de la calidad de las fronteras internas establecidas durante la fase monolítica; la literatura sobre migraciones industriales advierte que el esfuerzo de refactorizar dependencias entrelazadas puede obligar a convertir miles de DTOs en formatos serializables (Faustino, 2021). En este contexto, el tipado estructural de Go, donde la satisfacción de interfaces es implícita y el consumidor define localmente el contrato que necesita, sin que el proveedor deba ser consciente de él, permite aplicar el Principio de Inversión de Dependencias en su forma más pura (Brown, Woolf, & Yoder Foreword, 2025). Cuando un módulo alcanza el umbral de escala para convertirse en microservicio, la extracción requiere únicamente construir un adaptador de red que implemente la interfaz ya definida; el núcleo de negocio del consumidor permanece matemáticamente intacto (Al-Qora & Al-Said Ahm, 2025).

2.2.3. Patrones de concurrencia: el modelo CSP y su aplicabilidad transaccional

2.2.3.1. Fundamentos del modelo CSP en Go

El modelo de concurrencia de Go se fundamenta en el trabajo algebraico de C.A.R. Hoare, publicado en 1978 bajo el título *Communicating Sequential Processes* (Lange, 2018). Go materializa esta álgebra mediante *goroutines*, hilos ligeros gestionados en el espacio de usuario con un coste inicial de pocos kilobytes, y *channels*, conductos fuertemente tipados que actúan como primitivas unificadas de sincronización y paso de mensajes (Malhotra, 2025).

El axioma rector del modelo es: *"No te comuniques compartiendo memoria; comparte memoria comunicándote"*. En los modelos tradicionales de hilos POSIX, la comunicación requiere variables compartidas protegidas con *mutexes* y semáforos, un paradigma notoriamente frágil que provoca condiciones de carrera y *deadlocks* que corrompen el estado financiero (Malhotra, 2025). CSP erradica este peligro asegurando que los datos viajan como referencias exclusivas a través de canales tipados, fomentando el aislamiento de fallos y la certidumbre matemática en la transición de estados.

2.2.3.2. Comparativa con el modelo de actores y async/await

compartida, pero sus actores poseen identidad explícita (PID) y comunicación asíncrona a través de *mailboxes*, diseñados para distribución multinodo (Hakia, 2025), Para un *backend* financiero sin requisito de distribución, las *goroutines* anónimas y efímeras de CSP resultan más apropiadas: el rendezvous sincrónico prevé el desbordamiento de buzones propio de actores saturados.

El patrón *async/await* (Node.js, C#, Rust) fue concebido para operaciones de I/O sin bloquear el hilo principal (Karrby, 2022), pero su limitación en contextos intensivos en CPU —cifrado, algoritmos heurísticos— reside en que la asincronía de I/O se estanca frente a cargas computacionales puras. El planificador M:N de Go distribuye *goroutines* de forma equilibrada sobre todos los núcleos sin requerir anotaciones `async`, eliminando el *"problema del color de las funciones"* y preservando la legibilidad en lógicas financieras complejas (Hakia, 2025).

2.3. BLOCKCHAIN E IDENTIDAD DESCENTRALIZADA

El paradigma de la **Identidad Autosoberana** (*Self-Sovereign Identity*, SSI), redistribuye el control criptográfico de los datos personales desde los proveedores de identidad (IdPs) centralizados hacia los propios usuarios. En los modelos tradicionales, los silos monolíticos de información generan puntos únicos de fallo, vulnerabilidades ante brechas masivas y privación del control soberano, contraviniendo los principios de minimización de datos. Como respuesta, la investigación académica ha convergido en arquitecturas SSI ancladas en tecnología blockchain que descentralizan el poder criptográfico hacia los márgenes de la red.

2.3.1. Infraestructura de ejecución: EVM y el ecosistema Polygon

La **Ethereum Virtual Machine** (EVM) constituye el entorno de ejecución cuasi-Turing completo sobre el cual operan los contratos inteligentes de identidad. Diseñada como una máquina basada en pila, procesa secuencias de *opcodes* compilados desde lenguajes como Solidity o Vyper, garantizando resultados deterministas a través del consenso global bajo el mecanismo *Proof-of-Stake* (PoS) (Weyl, Ohlhaver, & Buterin, 2022). La capa de consenso (Beacon Chain) gestiona de forma independiente la organización de bloques y las reglas de elección de bifurcación. El control de costes computacionales opera mediante el concepto de *gas*, reformado por **EIP-1559** hacia un modelo de tarifa base (*Base Fee*) algorítmica quemada y tarifa de prioridad (*Priority Fee*) pagada a validadores, que mitiga ataques de denegación de servicio. Sin embargo, el rendimiento nativo de la Capa 1 de Ethereum es insuficiente para sistemas de identidad de alta frecuencia, con tiempos de finalización de entre 10 y 25 segundos y sáturas del bloque bajo alta concurrencia (Ko, Jeong, Woo, & Won-Ki Hong, 2023).

Ante estas restricciones, el ecosistema ha migrado su lógica de verificación hacia **Polygon**, solución de Capa 2 que preserva la seguridad criptográfica de Ethereum (Kumar, Venkatram, Saran N, Daniel, & Joe, 2025). La justificación técnica se sostiene en tres pilares: (1) **rendimiento**, con hasta 7.000 TPS teóricos y latencias operativas sensiblemente menores que la Capa 1; (2) **viabilidad económica**, con reducciones de costes de gas documentadas de hasta el 99,90% respecto a Ethereum Mainnet, y (3) **compatibilidad EVM**, dado que Polygon zkEVM ofrece equivalencia a nivel de *bytecode* con Solidity, Truffle, Hardhat y las bibliotecas OpenZeppelin, permitiendo desplegar contratos auditados en L1 sin modificaciones estructurales (Javed & Mangues-Bafalluy, 2024).

Para el desarrollo y auditoría de los protocolos de identidad, la comunidad ha adoptado la testnet Sepolia como red de pruebas de referencia en Ethereum (PoS), y la red Amoy, anclada a Sepolia como cadena raíz, como entorno de Capa 2 para Polygon. Estos entornos, que proveen validadores, grifos de tokens (faucets) y herramientas de exploración como Polygonscan, permiten validar contratos inteligentes bajo condiciones de alta fidelidad sin comprometer valor económico real (Roy & Akand, 2026).

2.3.2. El Estandar ERC-721 y sus limitaciones para la Identidad Digital

El estándar ERC-721 define una interfaz en la que cada token es matemáticamente único e identificado mediante la asociación de la dirección del contrato emisor y un `tokenId`, con un historial de propiedad globalmente verificable sin registradores centrales (Jaiman, Pernice, & Visara, 2022). Su adopción interdisciplinaria ha abarcado la certificación académica, la autenticación en sistemas LMS y los registros de cadenas de suministro. No obstante, su extrapolación al dominio de la identidad digital evidencia tres limitaciones axiomáticas (Saputro, Setiawan, Pratama, & Tarigan, 2026).

Transferibilidad intrínseca. El estándar requiere normativamente conferir al poseedor el derecho inalienable de ceder el token (Stefanovic, Pržulj, Stefanovic, Risti, & Capko, 2023). Credenciales inalienables, títulos universitarios, licencias profesionales, pasaportes, son jurídica y lógicamente intransferibles; codificarlas como activos comerciables subvierte su función de atestación y corrompe la confianza sistémica en el emisor (López-Pimentel & Gonzalez-Sanchez, 2025). Estándares derivados como ERC-1155 o los marcos de tokenización jerárquica padre-hijo abordan la eficiencia del gas pero perpetúan el paradigma de transferibilidad (Bhujel & Rahulamathavan, 2022).

Ausencia de privacidad. ERC-721 no incorpora estructuras de datos para la protección de información de identificación personal (PII). Los metadatos de los tokens, habitualmente almacenados en IPFS con su CID público, son accesibles por cualquier actor a través de un explorador de bloques (Jaiman, Leonard, & Urovi, 2022). Esta transparencia radical viola el RGPD e HIPAA, haciendo al estándar inadecuado para historiales médicos, calificaciones crediticias o verificaciones de identidad sin capas criptográficas adicionales externas (Mubarak, Riadi, & Sutikno, 2026).

Especulación y fraude sistémico. El mercado ERC-721 ha documentado manipulación de precios, falsificaciones y esquemas *rug pull* (Ko, Jeong, Woo, & Won-Ki Hong, 2023). Evaluaciones empíricas mediante DEMATEL y ANP concluyen que la especulación a corto plazo y la alta volatilidad son los factores de riesgo rectores del ecosistema, comprometiendo la soberanía e integridad del portador de credenciales (Weyl, Ohlhaver, & Buterin, 2022).

2.3.3. Soulbound Tokens (SBTs): Reputación Intransferible en la Sociedad Descentralizada

La publicación seminal *"Decentralized Society: Finding Web3's Soul"* (Weyl, Ohlhaver, & Buterin, 2022), diagnosticó la ausencia de primitivas Web3 para representar relaciones de confianza social y propuso los **Soulbound Tokens** (SBTs) como solución. En este marco, una *Soul*, una billetera o cuenta inteligente, alberga de forma persistente los SBTs intransferibles que atestiguan las credenciales, afiliaciones y el historial de un individuo u organización. La intransferibilidad garantiza que atributos como el historial crediticio, los antecedentes académicos o la reputación financiera queden criptográficamente vinculados a su titular, habilitando: mercados de préstamos sin colateral basados en puntuaciones crediticias *on-chain*; gobernanza democrática en DAOs resistente a ataques Sybil, al anclar el poder de voto en reputación verificada y no en liquidez; y marcos de propiedad intelectual con acceso regulado a espacios del metaverso (Lee, Li, Lin, & Wang, 2024).

La seguridad en la vinculación del token se sostiene mediante claves asimétricas, controles de acceso basados en roles (*onlyOwner*) en el contrato inteligente, y sistemas automatizados de autenticación que verifican el enlace entre la identidad *on-chain* y el SBT (Singh, y otros, 2024). Una vez integrado en el protocolo blockchain, el SBT hereda plenamente la inmutabilidad y distribución del activo digital, aun cuando su origen sea una entidad emisora centralizada.

La estandarización técnica ha cristalizado en dos EIPs de referencia:

- **ERC-5192 (*Minimal Soulbound NFT*).** Implementa un estado de bloqueo permanente sobre un NFT: cualquier invocación de transferencia sobre un token bloqueado genera una excepción en el contrato, manteniendo compatibilidad hacia atrás con interfaces ERC-721 (Virovets, Obushnyi, Ramskyi, & Hulak, 2024). Es el estándar dominante para la emisión de documentos verificables institucionales, como diplomas académicos, donde los derechos de acuñación están restringidos a una autoridad única.
- **ERC-5484 (*Consensual Soulbound Token*).** Introduce un protocolo de autorización bilateral (*digital handshake* on-chain): emisor y receptor deben acordar explícitamente las condiciones inmutables antes de que el SBT quede vinculado a la billetera destino, previniendo el *spam* de identidad (Delgado-von-Eitzen, Anido-Rifón, & Fernández-Iglesias, 2024). Este estándar es primordial para ecosistemas con escrutinio fiduciario

mutuo, como el registro de calificaciones crediticias, el rastreo de deudas corporativas o las afiliaciones en estructuras de gobernanza.

A pesar de su profundo avance técnico, el despliegue puramente *on-chain* de los SBTs mantiene el problema de la hiper-transparencia: la grabación permanente de credenciales en un libro mayor público es incompatible con los derechos constitucionales de privacidad y facilita la vigilancia masiva.

2.3.4. Identificadores Descentralizados (DIDs) y Credenciales Verificables (VCs)

El modelo SSI opera bajo un ecosistema tripartito: un Emisor (que certifica atributos), un Titular (usuario con cartera digital) y un Verificador (que demanda la prueba). Los Identificadores Descentralizados (DIDs), estandarizados por el W3C, son URIs criptográficamente seguros, resolubles y globalmente únicos, independientes de cualquier registro central (Satybaldy, Subedi, & Nowostawski, 2022).

Las Credenciales Verificables (VCs) son contenedores de datos portables protegidos por firmas asimétricas que operan *off-chain*, eliminando la latencia de red y los costes de gas asociados a registrar cada credencial en cadena (Eriksson & Hammerman, 2025). El estado del arte eleva la protección de datos mediante técnicas de Divulgación Selectiva basadas en Criptografía de Conocimiento Cero (ZKP), como SD-JWT y firmas BBS+, adoptadas en marcos institucionales como el EUDI Wallet (eIDAS 2.0). Aunque el Ecosistema Gondor no implementa estos estándares directamente, su arquitectura de SBTs y ZK-SNARKs es compatible con esta familia de primitivas.

2.4. PRUEBAS DE CONOCIMIENTO CERO (ZK-PROOFS)

Las Pruebas de Conocimiento Cero (Zero-Knowledge Proofs, ZKP) constituyen el pilar criptográfico sobre el que Gondor fundamenta su capacidad de construir reputación financiera privada y verificable.

2.4.1. Fundamentos teóricos

La teoría de las ZKP tiene su origen en el trabajo seminal de Goldwasser, Micali y Rackoff (1989), cuyo tratado *"The Knowledge Complexity of Interactive Proof Systems"* introdujo el marco formal para cuantificar el conocimiento que un demostrador transfiere

inadvertidamente a un verificador durante una demostración matemática (Goldwasser, Micali, & Rackoff, 1989).

La noción central es que un protocolo ZKP permite demostrar la validez de un enunciado, la posesión de una clave privada, la pertenencia de un valor a un conjunto o la satisfacción de una restricción financiera, sin revelar información alguna sobre el testimonio (*witness*) subyacente (Goldwasser, Micali, & Rackoff, 1989). Para ser formalmente clasificado como tal, un protocolo debe satisfacer tres propiedades axiomáticas:

- **Completitud:** Si el enunciado es verdadero, un demostrador honesto convencerá al verificador honesto con probabilidad abrumadora, garantizando la viabilidad operativa del sistema.
- **Solidez:** Si el enunciado es falso, ningún demostrador malicioso puede convencer al verificador, salvo con probabilidad computacionalmente despreciable, asegurando que la forja de pruebas es inviable sin conocer el testimonio real.
- **Conocimiento Cero:** El verificador no aprende nada más allá de la confirmación del enunciado. Esta propiedad se formaliza mediante la existencia de un algoritmo simulador cuya salida es computacionalmente indistinguible de una interacción auténtica, sin necesidad de acceder al testimonio.

El concepto de *indistinguibilidad computacional*, establece que dos familias de distribuciones son equivalentes si ningún circuito de tamaño polinómico puede distinguirlas con probabilidad no despreciable. Cuando las distribuciones son idénticamente distribuidas, se alcanza el Conocimiento Cero Perfecto (*Perfect Zero-Knowledge*), estándar que satisface el protocolo Groth16 (Cyr, 2025).

Originalmente concebidas como sistemas interactivos de múltiples rondas, las ZKP evolucionaron hacia las **Pruebas No Interactivas de Conocimiento Cero (NIZK)**. Mediante la heurística de Fiat-Shamir, el demostrador genera internamente la aleatoriedad requerida y empaqueta toda la prueba en un criptograma estático, verificable de forma universal y asíncrona por cualquier nodo de la red sin comunicación bidireccional (Cyr, 2025). Esta propiedad las convierte en el estándar de facto para aplicaciones sobre registros distribuidos y cadenas de bloques.

2.4.2. ZK-SNARKs frente a ZK-STARKs: arquitectura y justificación de Groth16

El ecosistema de pruebas no interactivas está dominado por dos familias: los ZK-SNARKs (*Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge*) y los ZK-STARKs (*Zero-Knowledge Scalable Transparent Arguments of Knowledge*), con filosofías matemáticas divergentes (Nainwal, Kamble, & Awathare, 2025).

Los **ZK-SNARKs** operan mediante criptografía de curvas elípticas con emparejamientos bilineales y esquemas de compromiso polinomial KZG (*Kate-Zaverucha-Goldberg*) (Cyr, 2025). Su limitación estructural crítica es la necesidad de un *Trusted Setup*: una ceremonia de generación de parámetros cuya entropía secreta, el denominado "residuo tóxico", debe destruirse completamente; si algún participante la retuviese, podría generar pruebas fraudulentas que el sistema aceptaría como válidas (Nainwal, Kamble, & Awathare, 2025). Adicionalmente, su dependencia del problema del logaritmo discreto en curvas elípticas los hace vulnerables a ataques cuánticos mediante el algoritmo de Shor.

Los **ZK-STARKs** eliminan el *Trusted Setup* mediante una arquitectura transparente apoyada en el protocolo FRI (*Fast Reed-Solomon Interactive Oracle Proofs of Proximity*) y compromisos sobre árboles de Merkle (Nainwal, Kamble, & Awathare, 2025). Su resistencia post-cuántica deriva del uso exclusivo de funciones hash simétricas. Sin embargo, presentan desventajas determinantes para entornos blockchain: el tamaño de prueba es masivo (~68 KB) frente a los ~131-384 bytes de los SNARKs; el tiempo de generación es aproximadamente 68 veces superior; y su verificación *on-chain* requeriría ingerir decenas de kilobytes de *calldata*, saturando los límites del bloque de Ethereum y elevando los costes a niveles económicamente insostenibles (Nainwal, Kamble, & Awathare, 2025).

En este marco, el protocolo **Groth16**, presentado por Jens Groth en 2016, emerge como el estándar industrial para aplicaciones sobre la EVM. Su diseño reduce la prueba a únicamente tres elementos de curva, ocupando entre 131 y 160 bytes al instanciar sobre la curva bn128 (Groth, 2016). La verificación requiere resolver tan solo tres emparejamientos bilineales, resultando en un coste de verificación asintóticamente constante $O(1)$, independiente de la complejidad del circuito subyacente (Abdolmaleki, Baghery, Lipmaa, & Zajac, 2017). En la EVM, la verificación de una prueba Groth16 mediante precompilaciones especializadas de curvas elípticas consume aproximadamente 200.000 unidades de gas, lo que permite procesar

hasta ~150 pruebas por bloque, equivalente a unas 12 pruebas por segundo (Delphi Consulting, 2025).

2.4.3. Estructuras de datos autenticadas y primitivas hash orientadas a ZK

Las Pruebas de Merkle constituyen los certificados de pertenencia que los circuitos ZKP emplean para acreditar que un elemento reside en una estructura de datos verificada sin revelar su contenido completo. Un árbol de Merkle binario, concebido originalmente por Ralph Merkle, agrega iterativamente hashes de datos hasta un nodo raíz único, permitiendo generar y verificar pruebas de inclusión con coste logarítmico `O(log n)` (Dubinin, Panasenko, Volynets, Zhebka, & Nishchemenko, 2025). En el contexto de un circuito ZKP, el demostrador aporta como entradas privadas la ruta de Merkle (nodos hermanos) y como entrada pública la raíz consensuada por la red; el circuito reconstruye determinísticamente el hash ascendente y verifica su igualdad con la raíz pública, probando la inclusión sin revelar el elemento ni su posición exacta (Ethereum Org, 2025).

La viabilidad de este patrón depende críticamente de la función hash empleada. Los algoritmos estándar como SHA-256, optimizados para operaciones bit a bit en hardware convencional, requieren más de 25.000 restricciones R1CS por evaluación, generando cuellos de botella insostenibles al modelarlos algebraicamente en circuitos ZK. Para resolver este impedimento, la comunidad desarrolló funciones hash nativas al dominio aritmético de los cuerpos finitos. La función **Poseidon**, representa el estado del arte (Grassi, Khovratovich, Rechberger, Roy, & sSchofnegger, 2021). Su arquitectura *HADES Design Strategy* combina rondas completas y parciales de S-boxes algebraicas (x^5) con capas lineales de matrices MDS difusivas, logrando apenas 200-300 restricciones R1CS para árboles de aridad binaria, una reducción de hasta 8 veces respecto al Pedersen Hash y órdenes de magnitud superior a SHA-256. Poseidon ha superado extensos programas de recompensas de vulnerabilidades confiriéndole la madurez de producción necesaria.

2.4.4. Ecosistema de compilación: justificación de Circom y SnarkJS

La compilación de circuitos criptográficos ZK requiere lenguajes específicos de dominio (DSL). Un análisis comparativo instrumental que modeló el juego *Mastermind* en múltiples *frameworks* reveló los siguientes tamaños de implementación: Noir (77 LoC), Circom (90 LoC),

gnark (101 LoC) y Halo2 (156 LoC), con compromisos distintos entre abstracción, rendimiento y seguridad (Neal, 2016).

- **Noir** (Aztec Network) compila hacia una representación intermedia agnóstica (ACIR) compatible con múltiples backends, alcanzando la máxima succinticidad de código; sin embargo, su alto nivel de abstracción ha ocasionado históricamente vulnerabilidades semánticas al enmascarar *underflows* de campo finito, resultando en circuitos susceptibles de compromiso.
- **gnark** (Consensys, Go) está orientado a sistemas empresariales de alto rendimiento con soporte para PLONK y Groth16 mediante aceleración GPU, pero ofrece menor portabilidad hacia entornos web.
- **Halo2** (Zcash/Ethereum Foundation, Rust) permite arithmetización PLONKish con *custom gates* y sistemas sin *Trusted Setup*, pero impone extenso código *boilerplate* (156 LoC), resultando inadecuado para equipos con plazos de desarrollo acotados.

El ecosistema **Circom + SnarkJS**, desarrollado por investigadores de la Universidad Complutense de Madrid y arquitectos del ecosistema Ethereum constituye la elección estandarizada para aplicaciones que convergen con la EVM (Muñoz-Tapia, Belles, Isabel, & Rubio, 2022). Su compilador, reescrito en Rust, transforma especificaciones de circuitos en sistemas R1CS predecibles y auditables, exponiendo al desarrollador a los límites del campo aritmético sin sacrificar productividad; el proceso promedia tiempos de carga de 0,38 segundos en entornos típicos, habilitando flujos de desarrollo ágil (Giri, 2026). Crucialmente, genera artefactos **WebAssembly** portables que permiten la generación de pruebas Groth16 directamente en el navegador del cliente mediante SnarkJS, eliminando la transmisión de datos sensibles a servidores externos y materializando la premisa del conocimiento cero real (Muñoz-Tapia, Belles, Isabel, & Rubio, 2022).

2.5. COMPARATIVA DE SOLUCIONES EN IDENTIDAD DESCENTRALIZADA Y REPUTACIÓN WEB3

2.5.1. Arquitecturas de Identidad Social y Resistencia Sybil

Gitcoin Passport[1] constituye uno de los agregadores de verificación de identidad y resistencia Sybil más adoptados en Web3. Arquitectónicamente, no almacena datos en cadenas de Capa 1, sino que asienta su núcleo sobre *Ceramic Network*, un protocolo de gestión descentralizada de flujos de datos inmutables. Sus *Stamps* agregan credenciales de fuentes Web2 OAuth y Web3 vinculadas a un DID agnóstico a la cadena, calculando un *Personhood Score* ponderado que terceras aplicaciones pueden consultar desde ComposeDB. Si bien ofrece alta interoperabilidad y portabilidad conforme a estándares W3C, la validación de credenciales descansa en la revelación pública de vinculación de cuentas y en booleanos binarios. El sistema no implementa circuitos ZK-SNARKs capaces de procesar series temporales de métricas financieras ni de generar pruebas de rentabilidad; resulta por tanto una herramienta estática e inadecuada para demostrar de forma privada un historial de *trading* dinámico.

Lens Protocol[2] registra el grafo social completo del usuario, publicaciones, comentarios (*mirrors*), seguimientos, como transacciones inmutables en Polygon, asociadas a un *Profile NFT* ERC-721 que otorga la soberanía del grafo al usuario. El contenido multimedia se referencia mediante *ContentURIs* hacia redes de almacenamiento persistente (IPFS, Arweave) para evitar la ineficiencia de almacenar cargas pesadas en la EVM. Infraestructuras auxiliares como OpenRank cuantifican la influencia ponderando interacciones visibles. La transparencia radical de Lens, su mayor virtud, constituye una vulnerabilidad insalvable para la privacidad financiera: cualquier historial de *Social Trading* construido sobre este protocolo expondría públicamente cada posición, activo y volumen operado, posibilitando ataques de *front-running* y réplica parasitaria de estrategias.

Worldcoin (World ID)[3] garantiza *Proof of Personhood* biométrico mediante el estándar criptográfico Semaphore, el cual, permite demostrar la pertenencia a un conjunto y emitir señales sin revelar la identidad subyacente, usando circuitos Groth16 y nulificadores

[1] https://www.gate.com/es/learn/articles/gitcoin-passport-the-gateway-to-a-decentralized-network/4127
[2] https://docs.lens.xyz/
[3] https://whitepaper.world.org/

deterministas que previenen la reutilización de identidades en múltiples contextos. A pesar de su excelencia técnica en garantizar anonimato sobre señales discretas, sus circuitos están optimizados para lógicas estáticas de membresía. El protocolo no dispone de las primitivas algebraicas necesarias para ingestar flujos asíncronos multidimensionales ni ejecutar polinomios aritméticos sobre registros temporales de rentabilidad, requisito imprescindible para el modelo de reputación de Gondor.

2.5.2. Protocolos de agregación de credenciales

Sismo[4] emite *ZK Badges* (SBTs) que permiten portar la reputación de una billetera fría a una caliente sin vincularlas públicamente, usando el esquema criptográfico Hydra-S1, implementado sobre circom y snarkjs. Su *Registry Tree* indexa grupos de cuentas con valores de estado; la prueba ZK atestigua simultáneamente la inclusión de la cuenta origen en el grupo, el valor acreditado y un nulificador determinista que previene la doble reclamación. Los contratos atestadores rechazan automáticamente colisiones de nulificadores. La limitación estructural de Sismo es la estaticidad: sus *badges* operan como capturas (snapshots) discretas de condiciones existenciales pasadas. El protocolo carece de un mecanismo de *Heartbeat* forzado que penalice la inactividad y automatice la degradación temporal de la reputación, lo cual perpetúa el sesgo de supervivencia identificado en el dominio del *Social Trading*.

Galxe[5] opera como la red de credenciales gamificadas más extensa de Web3. Su protocolo de identidad separa roles en Emisores, Titulares y Verificadores, con integraciones off-chain (Discord, GitHub, X/Twitter) y on-chain en siete ecosistemas de blockchain. Versiones recientes incorporan divulgación selectiva (*selective disclosure*) mediante ZK-SNARKs con nulificadores de identidad propios. No obstante, la plataforma está fuertemente orientada al marketing Web3 y a programas de fidelización; sus OATs atestiguan eventos consumados de forma discreta y no proveen un entorno de circuitos personalizables para auditar matemáticamente flujos aritméticos financieros ni actualizar iterativamente un registro de rentabilidad anclado a un SBT.

[4] https://github.com/sismo-core
[5] https://docs.galxe.com/

2.5.3. Infraestructura de oráculos y puente Web2-Web3

Clique[6] construye oráculos de atestación Web2 a Web3 mediante Entornos de Ejecución Confiables (TEE, Computación Multipartita (MPC) y ZKPs. Los TEEs (Intel SGX, ARM TrustZone) procesan sesiones HTTPS en enclaves aislados, validan la autenticidad mediante certificados X.509 de PKI y emiten atestaciones firmadas sin exponer datos en bruto. No obstante, los TEEs implican supuestos estructurales de confianza en el fabricante del hardware y han sido históricamente susceptibles a ataques de canal lateral (*side-channel attacks*), diferenciándose de la seguridad irrefutable basada en verificación polinomial propia de los esquemas ZK puros.

Reclaim Protocol[7] **y el estándar zkTLS** resuelven el "problema de la clave simétrica" del protocolo TLS. En una sesión HTTPS, la clave simétrica compartida, que garantiza la privacidad del tráfico frente a ataques *man-in-the-middle*, destruye la verificabilidad ante terceros: el poseedor de la clave podría haber alterado el contenido y re-cifrado. Arquitecturas como Reclaim Protocol, TLSNotary y zkPass (soportadas en capas como Sophon y Oasis Network) introducen un modelo Proxy que atestigua el flujo cifrado sin descifrarlo, sobre el cual el cliente genera una prueba ZK de divulgación selectiva. Esta prueba demuestra, por ejemplo, que el campo de rentabilidad del bróker equivale a un valor dado, sin exponer credenciales de sesión ni datos personales. zkTLS constituye la infraestructura más óptima para la ingesta inicial de métricas Web2, pero es, por naturaleza, un protocolo de procedencia de datos (*Data Provenance Protocol*), no un motor de cálculo reputacional ni una plataforma con ciclos de vida dinámicos de estado o incentivos de teoría de juegos.

2.5.4. Calificación crediticia descentralizada

Spectral Finance[8] introduce el concepto de Crédito No Fungible (NFC), que agrega criptográficamente múltiples billeteras en un único activo exportable y componible. Su *Multi-Asset Credit Risk Oracle* (MACRO Score) analiza eventos DeFi (Aave, Compound, MakerDAO): liquidaciones, cronogramas de amortización y volúmenes de deuda, actuando como análogo on-chain de las calificaciones FICO. Los kits de modelado propuestos exploran técnicas de ZKML para la generación de puntuaciones. Su posicionamiento es el más cercano

[6] https://docs.clique.tech/
[7] https://docs.reclaimprotocol.org/
[8] https://docs.spectrallabs.xyz/

arquitectónicamente al cálculo de reputación de Gondor. Sin embargo, presenta dos limitaciones sistémicas: procesa exclusivamente transacciones on-chain públicas, marginando cualquier historial previo en mercados cerrados Web2, y carece de privacidad profunda, pues la auditoría del MACRO Score requiere el rastreo lineal de direcciones DeFi públicas, exponiendo la propiedad intelectual (*alpha*) del trader a ataques de *front-running* MEV.

ARCx[9] gradúa el comportamiento DeFi en un índice, basado en tres parámetros dinámicos sobre una ventana temporal de 120 días: la *Daily Score Reward* (utilización del LTV, con óptimo estadístico en torno al 60%), el *Survival Score Reward* (resiliencia ante liquidaciones en ciclos macroeconómicos adversos) y la *Liquidation Penalty* (−250 puntos por liquidación efectiva). La ventana finita de 120 días impide camuflar la toxicidad presente con estadísticas prehistóricas positivas, siendo la aproximación más rigurosa a la mitigación del sesgo de supervivencia observada en el estado del arte. No obstante, la arquitectura sigue vinculada a métricas on-chain transparentes, sin operar en topología ZK.

Masa Finance[10] funciona como macro-agregador de perfiles de riesgo. Sus SBTs (Masa Soulbound Credit Score, Masa Soul Names) son inherentemente intransferibles conforme al estándar. Versiones recientes exploran la integración de ZK-SNARKs para atestiguar externamente la tenencia del SBT. Sin embargo, el modelo primario sigue optimizado para Single Sign-On en protocolos DeFi de préstamos, sin desplegar circuitos algebraicos especializados en auditar la volatilidad del rendimiento de cuentas de *trading* de alta frecuencia o largo plazo.

2.5.5. Marcos institucionales: EBSI

La Infraestructura Europea de Servicios Blockchain (EBSI) implementa el modelo triangular de confianza (Trusted Issuer, Holder, Verifier) bajo los estándares W3C de DIDs y VCs. Aunque constituye el referente institucional de mayor madurez en identidad descentralizada a escala europea, su diseño basado en atestaciones estáticas emitidas por autoridades centralizadas y restringido al EBSI Ledger permisionado lo hace inadecuado para ecosistemas especulativos de reputación descentralizada abierta como el que propone el Ecosistema Gondor.

[9] https://docs.arch.network/
[10] https://docs.massa.net/

2.5.6. Tabla comparativa

Plataforma / Protocolo	Tipo de Reputación	Privacidad	Verificabilidad	Portabilidad
Gitcoin Passport	Identidad genérica, resistencia Sybil	Baja (booleanos públicos)	Off-chain en Ceramic ComposeDB	Máxima (DID agnóstico, VCs W3C)
Lens Protocol	Capital social, métricas de influencia	Nula por arquitectura	On-chain estricta (Polygon ERC-721)	Alta componibilidad interna
Worldcoin (World ID)	Biometría única, Proof of Personhood	Alta (Semaphore + nulificadores)	On-chain/Off-chain híbrido (Groth16)	Alta (SDK multiplataforma)
Sismo	Portabilidad cruzada de credenciales	Altísima (Hydra-S1 ZK)	On-chain/Off-chain (snarkjs)	Alta (SBTs EVM L1/L2)
Galxe	Gamificación, identidad multiparamétrica	Media-Alta (ZK Selective Disclosure)	On-chain híbrida + oráculos OATs	Media (dependencia de red Galxe)
Reclaim Protocol (zkTLS)	Procedencia de datos Web2	Superior (Selective Reveal ZK/TLS)	Off-chain atestado → on-chain	Extrema (interoperable)

Plataforma / Protocolo	Tipo de Reputación	Privacidad	Verificabilidad	Portabilidad
Spectral Finance	Riesgo crediticio DeFi (MACRO Score)	Nula (auditoría pública on-chain)	Oráculos crediticios on-chain	Excelente (NFC componible)
ARCx	Riesgo DeFi (LTV, ventana 120 días)	Escasa/Nula (on-chain transparente)	Métricas on-chain puras	Limitada (dependiente de Vaults propios)
Masa Finance	Perfil crediticio híbrido Web2+Web3	Variable→Media (ZK-SNARKs exploratorios)	On-chain (SBTs Ethereum/Polygon/BNB)	Flexible (ERC estándar)
EBSI	Credenciales institucionales y académicas	Aceptable (modelo VC W3C, GDPR)	On-chain permisionada (EBSI Ledger)	Acotada al marco regulatorio europeo

Tabla 1: Tabla comparativa de soluciones existentes

2.6.CONCLUSIÓN

El recorrido sistemático por las cuatro dimensiones del estado del arte permite trazar un mapa preciso de las carencias que justifican la arquitectura de Gondor.

La primera brecha es de naturaleza estructural y tiene su origen en el paradigma Web2. Las plataformas de Social Trading operan sobre bases de datos CRUD mutables que facilitan el borrado retrospectivo de pérdidas, institucionalizando el sesgo de supervivencia en el contexto de los fondos de inversión (Elton, Gruber, & Blake, 2015). El historial de rendimiento queda, además, criptográficamente cautivo en el silo del bróker, sin mecanismo alguno de portabilidad verificable: los volcados CSV permitidos por el artículo 20 del GDPR carecen de cadena de atestación, por lo que ningún tercero puede distinguirlos de registros manipulados. El fundamento técnico para resolver ambas patologías, la inmutabilidad de un registro append-only distribuido y la transferencia soberana de credenciales criptográficamente firmadas, existe en el ecosistema Web3; lo que no existe es su aplicación integrada al dominio del Social Trading.

La segunda brecha es la tensión irresuelta entre privacidad y verificabilidad en los protocolos de reputación descentralizada. Los modelos de calificación crediticia *on-chain* más sofisticados, Spectral Finance y ARCx, que constituyen el antecedente arquitectónico más próximo a Gondor, operan sobre transacciones públicas de la EVM, exponiendo el alpha del inversor a ataques de front-running y réplica parasitaria de estrategias. En el extremo opuesto, los protocolos de identidad con privacidad elevada (Sismo con Hydra-S1, Worldcoin con Semaphore) demuestran la viabilidad de los ZK-SNARKs para disociar una cuenta de su historial; pero sus circuitos están optimizados para lógicas de membresía binaria o capturas estáticas de condiciones pasadas, careciendo de las primitivas algebraicas necesarias para ingestar series temporales multidimensionales y calcular sobre ellas métricas de rentabilidad ajustada al riesgo. La teoría criptográfica revisada, en particular, el protocolo Groth16 con su verificación de coste asintóticamente constante $O(1)$ y un consumo de aproximadamente 200.000 unidades de gas por prueba en la EVM17, junto con la generación de pruebas directamente en el navegador del cliente mediante Circom y SnarkJS, ofrece los instrumentos formales para romper esta dicotomía: es posible demostrar que unas métricas financieras satisfacen un umbral sin revelar los registros subyacentes.

La tercera brecha, la más sutil y la más directamente vinculada al problema original, es la ausencia de mecanismos criptográficos de degradación temporal de la reputación. Ninguna de las soluciones

examinadas, ni los ZK Badges de Sismo, ni los OATs de Galxe, ni los Soulbound Credit Scores de Masa Finance, implementa una primitiva que penalice algorítmicamente la inactividad del operador y fuerce la actualización periódica del activo reputacional. Esta omisión perpetúa, por diseño, la misma patología que el paradigma descentralizado pretende erradicar: un gestor puede congelar indefinidamente un SBT emitido en su momento de máximo rendimiento, haciendo que la representación on-chain de su historial diverga de su comportamiento real. La propuesta seminal de Weyl, Ohlhaver y Buterin sobre los Soulbound Tokens como primitiva para la reputación intransferible sienta las bases conceptuales correctas, pero no especifica mecanismos de ciclo de vida dinámico (Weyl, Ohlhaver, & Buterin, 2022).

El vacío funcional así delimitado, reputación financiera privada, verificable, portátil y con degradación temporal criptográficamente forzada, construida a partir de datos procedentes de Web2, define con precisión la contribución del presente TFM. Gondor propone colmarlo mediante la articulación de tres primitivas complementarias que el estado del arte ha desarrollado por separado pero nunca ha sintetizado en una arquitectura unificada: los SBTs como soporte intransferible del activo reputacional soberano; los ZK-SNARKs sobre circuitos Circom con esquema Groth16 como motor de actualización privada y verificable de las métricas del SBT; y el Merkle Batching como estructura de datos que agrega eficientemente cientos de operaciones en una única raíz on-chain, amortizando el coste de gas y eliminando la necesidad de publicar cada transacción individualmente. Sobre estas primitivas se articula, además, un mecanismo de Heartbeat obligatorio que degrada el valor paramétrico del SBT ante la inactividad, erradicando el sesgo de supervivencia de forma algorítmica. La combinación de estas cuatro decisiones de diseño no se ha encontrado implementada de forma integrada en ninguna de las plataformas y protocolos examinados a lo largo del presente estado del arte.

3. OBJETIVOS CONCRETOS Y METODOLOGÍA DE TRABAJO

Este capítulo formula los objetivos que articulan la propuesta del Ecosistema Gondor como respuesta a las carencias encontradas en capítulos anteriores y describe la metodología de trabajo empleada para su consecución.

3.1.OBJETIVO GENERAL

Diseñar e implementar un ecosistema de software híbrido Web2/Web3 que permita a un inversor construir, certificar y demostrar su reputación financiera de forma soberana, verificable y privada, eliminando la dependencia de cualquier plataforma centralizada.

Este objetivo se materializa en la construcción de un prototipo funcional (Proof of Concept) denominado **Gondor**, cuya arquitectura articula un motor de gestión de carteras de alta velocidad (Web2) con un coprocesador criptográfico blockchain (Web3). La innovación central reside en el paradigma *Verify-me*: la gestión eficiente del dato se realiza en infraestructura Web2 para garantizar rendimiento operacional, mientras que la certificación matemática de la verdad se ancla en una blockchain pública (Polygon) para garantizar inmutabilidad, verificabilidad autónoma e independencia de plataforma. De este modo, cualquier tercero puede verificar la integridad del historial de un operador sin necesidad de confiar en el administrador del sistema ni de solicitar permiso a ninguna autoridad central.

3.2.OBJETIVOS ESPECÍFICOS

El objetivo general se descompone en seis objetivos específicos que abarcan, de forma progresiva, las distintas capas del ecosistema: desde la infraestructura financiera Web2 hasta la capa de presentación descentralizada.

- **OE1. Desarrollar un motor de gestión de carteras de inversión robusto y escalable,** capaz de procesar operaciones de compra y venta de acciones, calcular el rendimiento (ROI) en tiempo real y servir cotizaciones de mercado con latencia de microsegundos.
- **OE2. Diseñar e implementar un mecanismo de identidad descentralizada basado en *Soulbound Tokens* (SBT)** desplegado en la red Polygon, vinculado de forma automática y transparente al registro Web2 del usuario. Este mecanismo debe generar la identidad blockchain sin imponer

fricción al usuario, materializando la premisa de que la reputación de un inversor debe pertenecer al individuo y no a un servidor corporativo.

- **OE3. Implementar un sistema de certificación criptográfica de transacciones** mediante recibos firmados deterministas y agrupación en Árboles de Merkle (*Merkle Batching*) con anclaje periódico on-chain, que garantice la integridad retroactiva de los datos. Este mecanismo debe hacer que cualquier manipulación posterior de la base de datos sea detectable por cualquier tercero mediante la confrontación de hashes con las raíces ancladas públicamente en la blockchain.
- **OE4. Diseñar y verificar un circuito ZK-SNARK (Groth16 sobre BN254)** que permita demostrar el rendimiento acumulado de un inversor a la blockchain sin revelar el detalle de sus operaciones financieras (importes, símbolos, cantidades), resolviendo así la tensión entre verificabilidad pública y privacidad financiera.
- **OE5. Implementar un mecanismo de *Heartbeat* (latido)** que prevenga la ocultación de pérdidas por inactividad, forzando la actualización periódica y verificable de la reputación. Este mecanismo, tanto a nivel de contrato inteligente (caducidad pasiva) como a nivel de servidor (inyección activa de actualizaciones), debe hacer matemáticamente imposible que un inversor congele su reputación.
- **OE6. Desarrollar una capa de presentación** compuesta por una aplicación móvil multiplataforma, que permita al usuario operar con su cartera e inspeccionar su reputación de forma desintermediada, consultando la blockchain directamente sin depender del backend.

3.3. METODOLOGÍA DEL TRABAJO

3.3.1. Modelo de desarrollo: iterativo e incremental

El desarrollo del Ecosistema Gondor se organizó siguiendo un modelo **iterativo e incremental**, estructurado en cuatro fases progresivas. Cada fase añade una capa de complejidad y funcionalidad sobre la anterior, respetando una cadena de dependencias técnicas estricta que justifica el orden de ejecución:

Fase	Nombre	Descripción	Dependencia
Fase 1	Minas Tirith	Core financiero Web2: gestión de carteras, precios en tiempo real, autenticación	— (fundacional)
Fase 2	Pelargir I	Identidad soberana: SBT en Polygon, registro atómico Web2+Web3	Fase 1 (requiere un sistema de usuarios estable)
Fase 3	Pelargir II	Certificación criptográfica: recibos firmados, Merkle Batching, ZK-SNARKs	Fase 2 (requiere identidad blockchain para anclar la reputación)
Fase 4	Frontend	Aplicación móvil y portal de auditoría web	Fases 1–3 (requiere que los tres pilares backend estén operativos)

Tabla 2: Fases del desarrollo iterativo del Ecosistema Gondor y sus dependencias

La justificación de este enfoque iterativo no es arbitraria, sino que responde a la naturaleza acumulativa de las capas tecnológicas del sistema:

- **Minas Tirith debe ser estable antes de introducir blockchain.** La identidad soberana se vincula al usuario Web2; si el módulo de autenticación o la gestión de transacciones no son fiables, la capa blockchain carece de fundamento sobre el que operar.
- **La identidad soberana (Fase 2) debe existir antes de poder actualizarla con pruebas criptográficas (Fase 3).** El circuito ZK-SNARK produce una prueba que actualiza el SBT del inversor; sin un SBT previamente acuñado, no hay destino para la prueba.
- **El frontend (Fase 4) requiere que los tres pilares backend estén operativos.** La aplicación móvil consume la API REST de Minas Tirith para la operativa financiera y consulta directamente la blockchain para la verificación de la reputación; ambos canales deben funcionar para que la interfaz tenga sentido.

Dentro de cada fase, se aplicó el ciclo **Requisitos, Desarrollo y Evaluación**. Los requisitos se formularon como historias de usuario; el desarrollo se documentó con diagramas de arquitectura, flujos de secuencia y descripciones técnicas; y la evaluación combinó pruebas automatizadas con análisis de rendimiento.

3.3.2. Nomenclatura del proyecto

A lo largo de este desarrollo, se adoptó una nomenclatura inspirada en la obra de J.R.R. Tolkien, reflejando de forma metáforica el rol de cada componente dentro de la arquitectura:

- **Gondor:** El ecosistema completo. Así como en la obra literaria designa a la nación entera y principal reino de los Hombres, aquí representa el ecosistema global que defiende la verdad financiera del inversor frente a la opacidad y manipulación.
- **Minas Tirith:** El *core* financiero Web2. Su nombre remite a la ciudad capital y fortaleza más importante del reino y principal escudo defensivo.
- **Pelargir:** El coprocesador Web3. Toma el nombre del gran puerto marítimo de Gondor, simbolizando la puerta de conexión hacia el exterior (la red descentralizada pública).

3.3.3. Tecnologías principales y justificación

La selección del *stack* tecnológico respondió a criterios de adecuación técnica para cada capa del sistema.

Capa	Tecnología	Justificación y problema que resuelve
Backend	Go (Golang)	**Latencia en procesamiento masivo.** Idóneo para paralelizar operaciones de red mediante sus primitivas nativas de concurrencia (*goroutines*), evitando cuellos de botella por I/O.
Base de datos	MySQL 8.0	**Integridad del dato.** Provee las garantías transaccionales ACID necesarias para asegurar que ninguna operación financiera quede en un estado inconsistente.

Caché	Redis 7	**Agotamiento de cuotas API y latencia.** Su velocidad en memoria permite usarla como fuente de verdad en tiempo de ejecución, eliminando llamadas HTTP externas redundantes ante ráfagas de tráfico.
Smart Contracts	Solidity 0.8.24	**Inmutabilidad estandarizada.** Como lenguaje nativo EVM y con herencia de OpenZeppelin, permite desplegar de forma auditada y segura la lógica de identidad intransferible (SBT).
Blockchain	Polygon Amoy (testnet)	**Costes de anclaje prohibitivos.** Sus bajísimas comisiones (*gas fees*) y alta velocidad permiten escalar el sistema para anclar raíces Merkle periódicamente sin los costes inasumibles de una *Mainnet*.
Circuitos ZK	Circom 2.1.6 + SnarkJS	**Fricción privacidad y verificabilidad.** Permite demostrar matemáticamente que un historial de operaciones es real (con pruebas Groth16) sin revelar públicamente el detalle de las transacciones.
App móvil	React Native + Expo	**Desarrollo multiplataforma (iOS y Android)** desde una única base de código. Ecosistema de librerías maduro, incluyendo viem para la conexión RPC directa con la blockchain.

Tabla 3: Tabla resumen de la justificación tecnológica

3.3.4. Herramientas de desarrollo y testing

El entorno de desarrollo se apoyó en **Docker** para la contenerización local, **Hardhat** y **Anvil** para el desarrollo y despliegue en Solidity, y abigen para la generación de *bindings* Go.

La estrategia de aseguramiento de calidad combinó tests unitarios con mocks (gomock) y *golden files*, tests de contrato inteligente (Hardhat + Chai), tests de aceptación end-to-end (Testcontainers + Anvil) y

un simulador de estrés. Los detalles y resultados de cobertura de cada nivel se desarrollan en la evaluación de la fase correspondiente.

Asimismo, se integró la librería go-metrics para la instrumentación del código, permitiendo obtener datos empíricos sobre cuellos de botella y rendimiento que justifican las decisiones de diseño.

3.3.5. Gestión del proyecto

Al tratarse de un desarrollo **individual**, la gestión se organizó mediante un enfoque ágil basado en Kanban utilizando Notion. Esta herramienta proporcionó una vista continua del estado de las tareas (estructuradas en un flujo simple: *To Do*, *In Progress*, *Done*) sin la sobrecarga administrativa de metodologías como Scrum, permitiendo un flujo de trabajo continuo y adaptativo a la complejidad variable de cada fase.

3.3.6. Organización del código fuente

El código del Ecosistema Gondor se distribuyó en repositorios independientes alojados en GitHub, adoptando este enfoque frente al monorepo para facilitar la gestión, dado el carácter heterogéneo de las tecnologías (Go, Solidity/Node.js y React/React Native):

Repositorio	**Contenido**
Gondor	Monolito modular backend (Minas Tirith + Pelargir) y circuitos Circom
gondor-blockchain	Contratos inteligentes Solidity y scripts de despliegue
gondor-app	Aplicación móvil multiplataforma (React Native)

Tabla 4: Organización del código fuente en repositorios GitHub

3.3.7. Correspondencia entre objetivos específicos y fases

Para facilitar la trazabilidad entre los objetivos formulados y su materialización en el desarrollo, la siguiente tabla resume la correspondencia entre cada objetivo específico y la fase del proyecto en la que se aborda:

Objetivo específico	**Fase**	**Componente principal**
OE1. Motor de gestión de carteras	Fase 1	Minas Tirith (Go)
OE2. Identidad descentralizada (SBT)	Fase 2	Pelargir I + Smart Contract (Solidity)
OE3. Certificación criptográfica (recibos + Merkle)	Fase 3	Pelargir II (Go) + Smart Contract
OE4. Circuito ZK-SNARK	Fase 3	Circuito Circom + SnarkJS + Smart Contract
OE5. Mecanismo de *Heartbeat*	Fases 2–3	Smart Contract (pasivo) + Pelargir II (activo)
OE6. Capa de presentación	Fase 4	gondor-app (React Native) + gondor-verifier (React)

Tabla 5: Correspondencia entre objetivos específicos y fases de desarrollo

Esta estructura iterativa garantiza que cada objetivo específico es abordado en el momento en que sus dependencias técnicas han sido satisfechas por las fases anteriores, y que su evaluación se realiza de forma progresiva con los mecanismos de testing descritos en este capítulo.

4. DESARROLLO ESPECÍFICO DE LA CONTRIBUCIÓN

Antes de detallar el desarrollo iterativo por fases, el siguiente diagrama presenta una vista arquitectónica global del Ecosistema Gondor. Este esquema ilustra cómo los distintos componentes tecnológicos, el *core* financiero Web2 (Minas Tirith), el coprocesador criptográfico Web3 (Pelargir), las infraestructuras de persistencia y la red blockchain, interactúan para formar la solución completa:

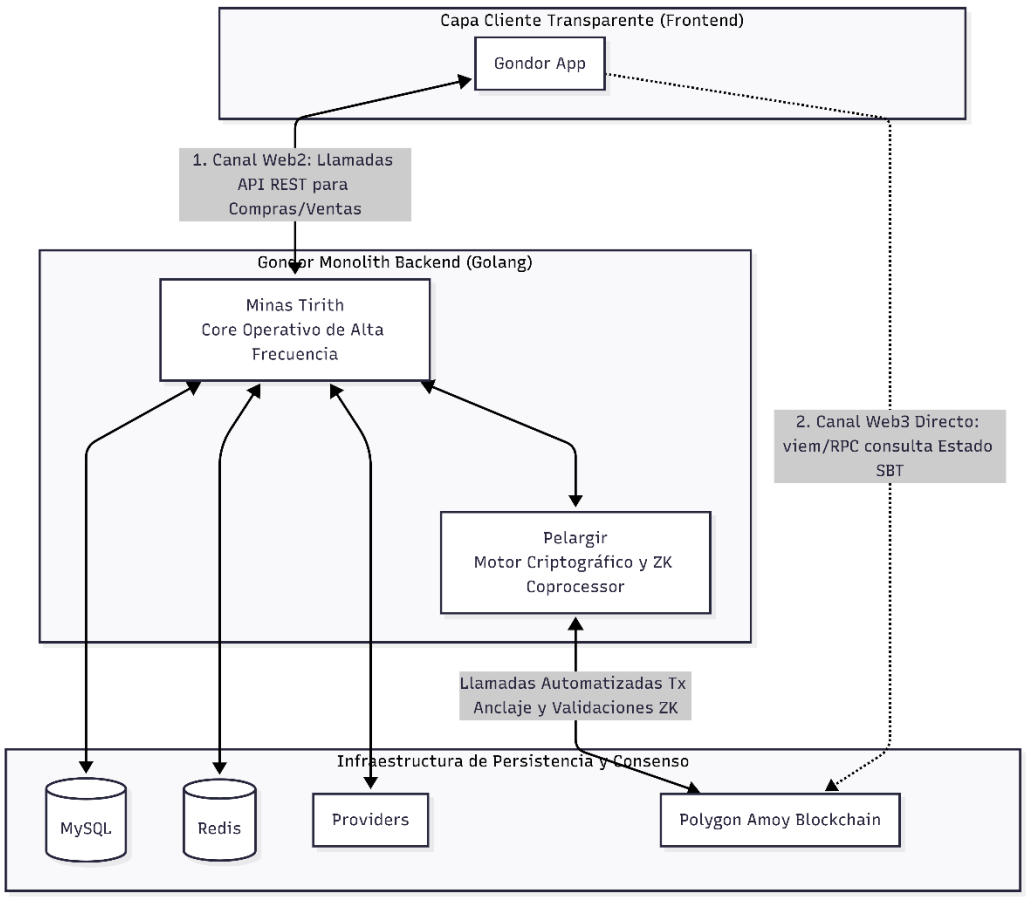


Figura 4-1: Arquitectura del sistema Gondor.

4.1. PRIMER MOVIMIENTO: LA CONSTRUCCIÓN DEL CORE INANCIERO WEB2

4.1.1. Identificación de requisitos y diseño

La historia del desarrollo de Gondor comienza con una necesidad clara y fundamental: antes de intentar descentralizar la confianza o aportar pruebas criptográficas, era imprescindible contar con un motor de gestión y simulación de carteras de inversión que fuera excepcionalmente robusto, ágil y escalable. Este fue el origen de **Minas Tirith**, el módulo central Web2, de todo Gondor.

El objetivo primario de esta fase fue construir la base fundacional del proyecto. Minas Tirith debía ser capaz de gestionar operaciones de compra y venta de acciones, calcular el rendimiento (ROI) y valorar un portafolio en tiempo real. En esta etapa, el sistema operaba bajo la premisa tradicional del software financiero centralizado: los datos de entrada y la base de datos son de confianza absoluta. La complejidad criptográfica quedaría reservada para fases posteriores.

Las historias de usuario que guiaron el diseño fueron las siguientes:

- **Como usuario**, quiero registrar una compra de acciones por número de unidades, para poder hacer un seguimiento preciso de mis posiciones.
- **Como usuario**, quiero registrar una compra de acciones por importe total invertido, para que el sistema calcule automáticamente mis unidades.
- **Como usuario**, quiero ver un resumen en tiempo real de mi cartera actual, para monitorizar el rendimiento de mi inversión.
- **Como usuario**, quiero recibir actualizaciones continuas de los precios de mis acciones, para que mi panel de control se mantenga actualizado con la mínima latencia.

4.1.1.1. Necesidad de desacople tecnológico: la arquitectura hexagonal

La decisión arquitectónica más determinante de esta fase fue la adopción de la **Arquitectura Hexagonal**. El sistema debía integrar, de forma simultánea, múltiples tecnologías de infraestructura: una base de datos relacional (MySQL), una caché en memoria (Redis) y clientes de APIs externas (Alpaca, Finnhub). Acoplar la lógica de negocio directamente a cualquiera de estas tecnologías habría creado una deuda técnica insostenible, especialmente en un proyecto cuya hoja de ruta preveía añadir una capa blockchain de alta complejidad en fases posteriores.

La Arquitectura Hexagonal resolvió este problema definiendo una frontera explícita entre el **dominio** (las reglas del negocio financiero, inmutables e independientes) y los **adaptadores** (las implementaciones concretas de infraestructura, intercambiables). Esta frontera se materializa mediante interfaces Go (*ports*). El resultado es que la lógica de cálculo de ROI o el procesamiento de transacciones pueden ser probadas de forma completamente aislada, sin necesidad de levantar ningún contenedor externo.

4.1.1.2. Gestión de alta concurrencia y latencia para la actualización de precios

El segundo reto identificado en el diseño fue el de la obtención de cotizaciones en tiempo real. Un análisis preliminar reveló dos problemas estructurales:

1. **Latencia inaceptable:** Si cada consulta al precio de una acción implicara una llamada HTTP a un proveedor externo, la experiencia de usuario sería inaceptablemente lenta-
2. **Agotamiento de cuotas de API:** Los proveedores de datos financieros imponen límites estrictos de uso. Un escenario realista en el que 80 usuarios consultan simultáneamente su cartera, y

todos tienen Apple (AAPL) en su portafolio, generaría 80 llamadas externas al mismo proveedor en décimas de segundo, quemando instantáneamente las cuotas disponibles.

Para resolver ambos problemas de forma elegante, el diseño estableció dos mecanismos complementarios que se detallan en la siguiente sección.

4.1.2. Implementación de los servicios de dominio

La capa de dominio se estructuró en cuatro servicios principales, cada uno encapsulando una responsabilidad bien definida:

- `StockService`**:** Orquesta toda la lógica relacionada con precios de mercado. Es el servicio más singular del sistema: gestiona la carga de precios en segundo plano y sirve el precio actual de cualquier símbolo bajo demanda, abstrayendo al dominio de los detalles de Redis o las APIs externas.
- `HoldingService`**:** Gestiona las posiciones (*holdings*) del usuario. Un *holding* representa la información agregada de los activos del usuario para un símbolo dado.
- `PortfolioService`**:** Agrega la información de todos los *holdings* del usuario para presentar una vista consolidada de la cartera, incluyendo valores calculados como el valor actual total, el capital invertido y las métricas de rendimiento globales.
- `TransactionService`**:** Procesa operaciones de compra y venta, actualizando los *holdings* mediante el `HoldingService` y registrando cada transacción en base de datos para garantizar la trazabilidad completa. Admite dos modalidades de compra: por número de unidades o por importe total invertido (en cuyo caso el sistema calcula automáticamente las unidades al precio de mercado vigente).
- `AuthService`**:** Gestiona el ciclo de vida de la identidad Web2 del usuario: registro y login. Se apoya en el `TokenService` para emitir y verificar tokens JWT. El middleware de autenticación HTTP verifica el JWT en cada petición protegida antes de dejar pasar la llamada al controlador correspondiente.

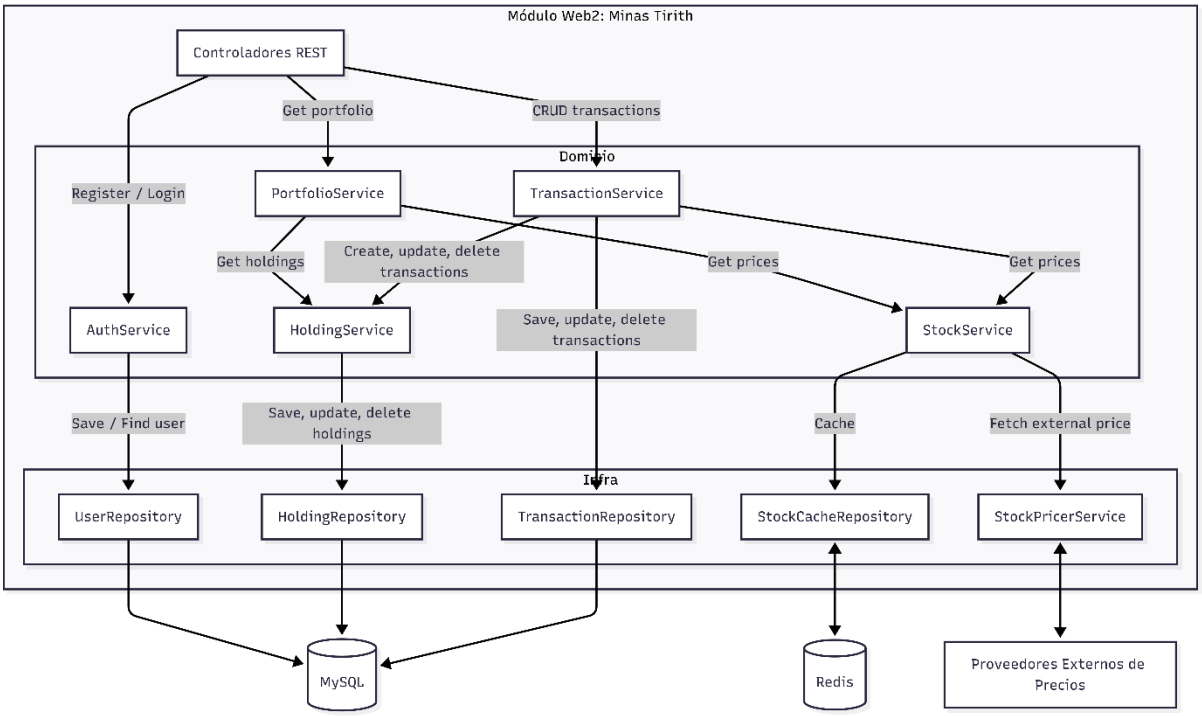


Figura 4-2: Arquitectura del módulo Minas Tirith.

4.1.3. Inicialización y estrategia de caché: Redis como fuente de verdad

Los precios siempre se obtendrán de la Redis y nunca directamente del proveedor mediante una llamada HTTP, sirviendo los precios con latencia de microsegundos. Por lo tanto se abarcan tres escenarios diferenciados:

1. **Al arrancar el servicio.** Con Redis vacía, se inicializa la caché consultando a MySQL todos los símbolos activos y almacenándolos sin precio. El job de actualización poblará estos precios en su primer ciclo, asegurando que el motor concurrente disponga de todos los símbolos a monitorizar.
2. **Política estándar: siempre desde caché.** Completado el primer ciclo del job, **los precios conocidos se sirven exclusivamente desde Redis**. Las consultas o transacciones habituales no realizan llamadas HTTP a proveedores externos, quedando estas relegadas al job de actualización en segundo plano.
3. **Símbolo nuevo (primera aparición).** Cuando un usuario compra por primera vez un activo no registrado en Redis, el StockService realiza una llamada puntual al StockPricerService para obtener el precio externo y lo persiste en caché. A partir de entonces, el job lo incluirá en sus actualizaciones periódicas.

El siguiente diagrama resume la gestión de los precios de las acciones:

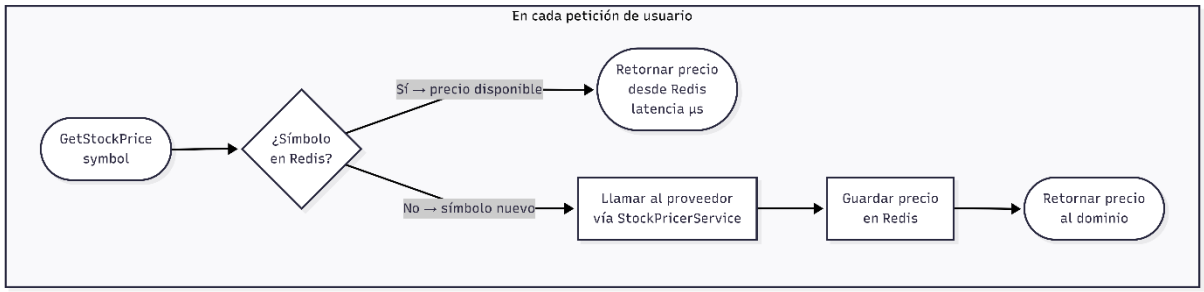


Figura 4-3: Diagrama de flujo para la gestión de precios de las acciones.

Gracias a esta estrategia, cuando un usuario realiza una llamada al sistema para consultar su cartera o registrar una transacción, en la mayor parte de los casos, **no se produce ninguna llamada HTTP externa**: el precio ya esté precalculado en Redis y se sirve con latencia de microsegundos.

4.1.4. Desarrollo del motor concurrente: el patrón Fan-in / Fan-out

El núcleo diferencial de Minas Tirith es su motor concurrente de actualización de precios. La función para actualizar los precios se dispara periódicamente y se implementa utilizando el patrón **Fan-in / Fan-out** para actualizar en paralelo todos los precios del sistema.

La justificación de este diseño reside en los datos de perfilado empírico: la operación de obtener el precio de un símbolo desde un proveedor externo consume aproximadamente un **190% más de tiempo** que cualquier otra operación del sistema (acceso a base de datos, escritura en Redis, etc.). Esto hace que sea el cuello de botella natural y el candidato ideal para la paralelización.

El patrón se implementa de la siguiente forma:

1. **Fan-Out:** Se lanzan tantas *goroutines* paralelas como núcleos de CPU disponibles tiene la máquina. Cada *goroutine* consume símbolos de un canal compartido, realiza la llamada externa para obtener el precio y deposita el resultado en su propio canal de salida.
2. **Fan-In:** Una función recorre el array de canales individuales de cada *goroutine* y los fusiona en un único canal central, desde el cual un trabajador final lee los pares símbolo-precio y los persiste en Redis de forma serializada, evitando condiciones de carrera en escritura.

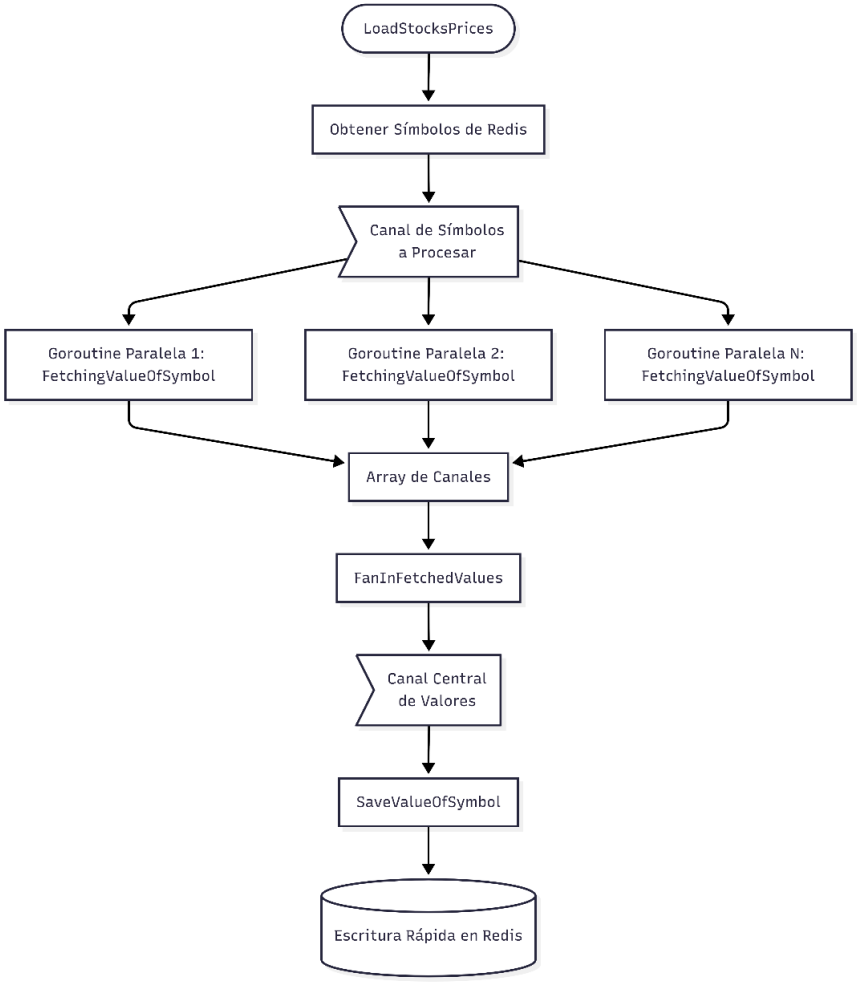


Figura 4-4: Diagrama de actividad de la obtención de los precios de las acciones.

La implementación coordina la finalización de todas las *goroutines* antes de concluir el ciclo de actualización, usando canales con semántica *done* para propagar cancelaciones de contexto de forma limpia a través de toda la cadena de procesamiento.

4.1.5. La gestión de múltiples proveedores con rotación por cuota

El componente que resuelve el problema de la dependencia en un único proveedor de datos es el `StockPricerService`. Este servicio mantiene internamente un inventario de múltiples clientes de APIs (como Alpaca y Finnhub), seleccionando en cada petición el cliente más apropiado según su disponibilidad de cuota en el segundo actual.

El algoritmo de selección funciona así:

1. Los clientes se ordenan en el momento de la inicialización de mayor a menor límite de peticiones por segundo.

2. Se mantiene un índice y un contador de peticiones por segundo para cada cliente.
3. Cuando se solicita un precio, se selecciona el cliente actual. Si ha alcanzado su límite, se rota al siguiente.
4. Si todos los clientes han agotado su cuota, el sistema espera hasta el inicio del siguiente segundo (con un pequeño jitter aleatorio para evitar tormentas de peticiones) y reintenta.

Esto garantiza que el sistema nunca bloquee por agotamiento de cuota y, a la vez, distribuye la carga de forma óptima entre todos los proveedores disponibles. En la configuración de referencia, Finnhub (30 req/s) actúa como proveedor primario y Alpaca (3 req/s) como secundario.

El siguiente diagrama de actividad muestra el flujo completo de la función `GetStockPrice` del `StockPricerService`, desde la selección de proveedor hasta la respuesta al dominio:

4.1.6. Tolerancia a fallos con el fallback

Además del agotamiento de cuota, el `StockPricerService` debe gestionar los fallos transitorios de los proveedores externos. Para ello, se desarrolló desde esta primera fase el fallback reutilizable con el siguiente mecanismo:

1. Si la función tiene éxito, devuelve el resultado inmediatamente anotando el número de reintentos consumidos.
2. Si la función falla con un error transitorio (p. ej., *timeout* de red), se reintenta hasta agotar el límite.
3. Si la función falla con un error envuelto como `Abortable`, es decir, un error fatal que no se resolverá reintentando (p. ej., símbolo no reconocido por el proveedor), el bucle se aborta inmediatamente sin reintentos adicionales.
4. Si la función falla con un error de servidor (5xx), o de autenticación o de cuota, el proveedor se invalida durante un tiempo y se rota al siguiente proveedor.

El `StockPricerService` utiliza este mecanismo al obtener el precio de cada símbolo: si el proveedor seleccionado responde con un error de red, el fallback reintenta hasta el límite configurado antes de rotar al siguiente cliente del inventario. Note el lector que este sistema permitiría tener todos los proveedores que fuesen necesarios, aunque de momento solo tiene dos (Finnhub y Alpaca).

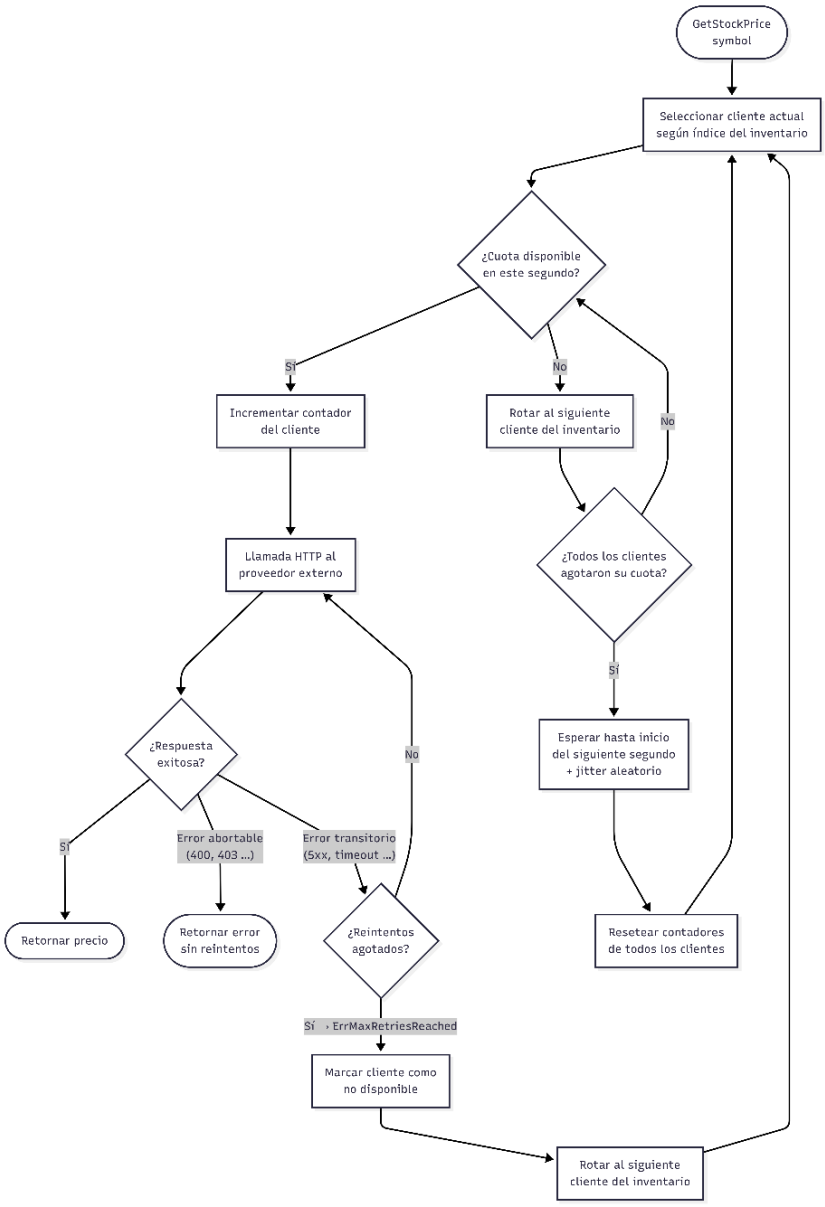


Figura 4-5: Diagrama de flujo del algoritmo de selección de proveedores

4.1.7. Evaluación de la fase

4.1.7.1. Pruebas de la lógica financiera: verificación del cálculo de ROI

La estrategia de testing adoptada en Minas Tirith fue deliberadamente exhaustiva. La filosofía subyacente fue la de maximizar la cobertura de los flujos reales de la aplicación sin depender de servicios externos: la mayoría del código se ejecuta tal cual en los tests, recurriendo a *mocks* únicamente en los puntos de integración externos (clientes de API de terceros, bases de datos). Esto hace que los tests se

comporten casi como tests de integración, pero con ejecución en memoria y sin necesidad de contenedores.

La cobertura resultante supera el **85% en la capa de servicio** y el **70% en las capas de API y persistencia**. La infraestructura de servicios externos alcanzó una cobertura del **100%** en las partes testeables (excluyendo por definición los clientes HTTP reales). Las respuestas de la API fueron validadas mediante *golden files*: ficheros de referencia que almacenan la salida esperada de cada *endpoint*, permitiendo detectar regresiones de forma determinista.

4.1.7.2. Análisis de rendimiento: caché de Redis y concurrencia como solución al agotamiento de cuotas

El resultado más significativo de la Fase 1 fue la demostración empírica de que la combinación Redis + Fan-in/Fan-out resuelve eficazmente los dos problemas identificados en el diseño.

Las métricas instrumentadas directamente en el `StockService` permitieron medir los tiempos medios de cada operación del ciclo de carga de precios:

Operación	Descripción	Tiempo relativo
`FindSavedSymbol`	Lectura de símbolos desde Redis	~1x (referencia)
`GetStockPrice`	Llamada externa al proveedor de precios	**~2,9x** (cuello de botella)
`SaveAll`	Escritura de precios en Redis	~1x

Tabla 6: Tiempos de cada operación en el flujo de obtención de precios.

Esta disparidad de casi un **190%** entre la operación de red y el resto justifica plenamente el diseño Fan-in/Fan-out: al paralelizar `GetStockPrice` con N *goroutines* (una por núcleo de CPU), el tiempo total del ciclo de actualización se divide aproximadamente por N en la parte dominante, manteniendo las escrituras serializadas en Redis para evitar condiciones de carrera.

Desde la perspectiva del consumo de cuotas de API, el impacto es igualmente sustancial. Sin la caché de Redis, cada petición de usuario desencadenaría una llamada externa. Con la caché activa, el número de llamadas externas se reduce al número de símbolos únicos activos en el sistema multiplicado por la

frecuencia de actualización del job (configurable, por defecto 10 segundos), independientemente del número de usuarios concurrentes. Esta optimización hace que el sistema sea viable con los planes básicos de los proveedores de datos financieros.

4.2. SEGUNDO MOVIMIENTO: LOS CIMIENTOS DE LA IDENTIDAD SOBERANA

4.2.1. Identificación de requisitos y diseño

Con Minas Tirith estable y demostrado como motor financiero robusto, Gondor podía enfrentarse al verdadero núcleo de su propuesta de valor: resolver el problema sistémico del **Secuestro de Reputación** que impera en las plataformas de *Social Trading* Web2 como eToro o Darwinex.

En estas plataformas, cuando un gestor construye un historial de éxito, sus datos de rendimiento, su reputación, pertenecen técnicamente a la base de datos centralizada de la empresa. Si el gestor decide abandonar la plataforma, pierde todo su historial verificable. Su reputación queda secuestrada.

La Fase 2 introdujo la primera capa blockchain del proyecto: el módulo **Pelargir**, un coprocesador Web3 implementado dentro del mismo monolito modular en Go. En esta etapa inicial, Pelargir tenía una misión concreta y fundamental: crear la **Identidad Soberana** del usuario. La premisa era que la reputación de un inversor no debe pertenecer a un servidor corporativo, sino al propio individuo, materializada de forma pública e inmutable en una red blockchain.

Las historias de usuario que guiaron el diseño fueron las siguientes:

- **Como usuario**, quiero que al registrarme en la plataforma se me asigne automáticamente una identidad blockchain, sin necesidad de manejar billeteras ni claves privadas.
- **Como usuario**, quiero que mi identidad descentralizada sea intransferible, para que nadie pueda comprar o falsificar mi reputación.
- **Como usuario**, quiero que mi registro sea atómico: o existo en ambos mundos (Web2 y Web3) simultáneamente, o no existo en ninguno.

4.2.1.1. La necesidad de vincular Web2 y Web3 sin fricción

El diseño de Pelargir estableció un modelo de custodia delegada: el sistema genera una billetera Ethereum para el usuario, asume el coste del gas y ejecuta la acuñación del token de identidad de forma completamente transparente. Desde la perspectiva del usuario, el proceso es idéntico al de un registro

convencional Web2 (correo y contraseña). Los beneficios de la inmutabilidad blockchain se obtienen sin imponer ningún coste cognitivo adicional

4.2.1.2. El reto de la consistencia entre dos mundos

El segundo reto identificado fue de naturaleza transaccional. Vincular un sistema determinista y sincrónico (MySQL) con un sistema asincrónico y distribuido (la blockchain de Polygon) introduce un riesgo de estados inconsistentes: ¿qué ocurre si la base de datos guarda al usuario pero la red de Polygon está caída? O, al contrario, ¿qué ocurre si el token se acuña correctamente pero falla la escritura en MySQL? En ambos casos, el sistema quedaría en un estado corrupto: un usuario sin identidad soberana, o una identidad huérfana sin usuario asociado.

El diseño estableció que la operación de registro debía ser atómica en ambos mundos, reutilizando el mecanismo de fallback con reintentos y aborto selectivo que se detalló en la sección anterior.

4.2.2. Diseño del Smart Contract

El contrato inteligente central fue programado en Solidity 0.8.24 utilizando el framework Hardhat y hereda de dos contratos base de **OpenZeppelin v5**: `ERC721` (el estándar de tokens no fungibles) y `Ownable` (que restringe las operaciones administrativas al propietario del contrato, es decir, el backend Pelargir). El nombre del token es `GondorSovereignIdentity` y su símbolo `GSID`.

4.2.2.1. La propiedad Soulbound

La característica definitoria del contrato es su condición de **Soulbound Token (SBT)**. A diferencia de un NFT tradicional que puede comprarse y venderse en mercados abiertos, un SBT bloquea cualquier intento de transferencia a nivel de contrato inteligente.

Esto garantiza que el token se adhiere permanentemente a la billetera del usuario. Su estatus de inversor no puede ser adquirido con dinero: tiene que ser demostrado. El SBT actúa como un currículum inmutable y público que certificará, en fases posteriores, el rendimiento financiero del usuario.

4.2.2.2. Estructuras de datos on-chain

El contrato no se limita a registrar la propiedad del token, sino que actúa como contenedor para los datos de identidad y reputación. Para cada TokenID, el contrato almacena:

- `tokenDIDs`: El *Decentralized Identifier* (DID) del usuario, generado con el formato `did:gondor:{walletAddress}`.
- `reputationLevels` **y** `reputationLevelsRealized`**:** Los niveles de reputación (ROI no realizado y realizado), inicializados a 1 en el momento de la acuñación y actualizables mediante pruebas criptográficas en fases posteriores. (Nota: el nivel 0 está reservado para cuentas invalidadas).
- `lastHeartbeat`**:** La marca de tiempo de la última actualización de reputación, esencial para el mecanismo de latido que se describe a continuación.
- `identityStates`**:** Un struct que almacena el estado financiero acumulado (`totalRevenue`, `totalCost`, `totalCostRealized`), preparado para recibir actualizaciones verificadas criptográficamente.

4.2.2.3. El mecanismo de Hearbeat

Una de las lógicas más sofisticadas codificadas dentro del contrato es el **Heartbeat**. En las plataformas Web2, un inversor que comienza a tener grandes pérdidas puede simplemente dejar de operar o cerrar su perfil público para ocultar su fracaso, creando un *sesgo de supervivencia*. Para evitar esto criptográficamente, el contrato exige un latido constante:

- Cada vez que la reputación se actualiza, el contrato registra el `block.timestamp` actual en `lastHeartbeat`.
- La función `isReputationExpired` compara el timestamp actual con `lastHeartbeat` + `heartbeatInterval` (configurado por defecto a 30 días).
- Si el intervalo se supera, el usuario dejó de reportar datos porque está perdiendo dinero, cualquier persona puede invocar `revokeExpiredReputation`, que resetea los niveles de reputación a 0 y emite un evento `ReputationExpired`.
- Es matemáticamente imposible esconder las pérdidas por omisión.

4.2.2.4. Preparación para la verificación criptográfica

Aunque la verificación criptográfica completa se desarrolla en la Fase 3, el contrato fue diseñado desde el inicio con las interfaces necesarias para soportarla. La función `updateReputationWithProof` valida que las *Merkle Roots* proporcionadas estén previamente ancladas y delega la verificación de la prueba ZK-

SNARK a un contrato verificador externo (`IGondorVerifier`), cuya dirección se registra dinámicamente. Este diseño modular permite actualizar la lógica de verificación sin redesplegar el contrato de identidad.

El mecanismo de **Sovereign Handover** permite, además, que el propietario del contrato (Pelargir) transfiera la custodia del SBT desde la billetera gestionada por el sistema a una billetera personal del usuario, materializando la verdadera soberanía de la identidad.

4.2.3. Implementación del módulo Pelargir en Go

Pelargir se implementó siguiendo la misma Arquitectura Hexagonal de Minas Tirith. El servicio central de esta fase es el `RegisterService`, que orquesta el flujo de creación de identidad soberana en cuatro pasos:

1. **Creación de la billetera:** genera un par de claves ECDSA, obteniendo la dirección Ethereum derivada de la clave pública.
2. **Generación del DID:** construye el identificador descentralizado con el formato `did:gondor:{address}`.
3. **Acuñación del SBT:** llama al adaptador `PolygonClient`, cuyas interfaces con el contrato Solidity se generan automáticamente mediante `abigen` a partir de la ABI, envolviendo la operación en el mecanismo de fallback.

4.2.3.1. Reutilización del fallback para la comunicación con la Blockchain

La comunicación con una red blockchain es inherentemente poco fiable: los nodos RPC pueden estar congestionados, las transacciones pueden fallar por falta de gas, o la red puede experimentar latencias impredecibles. Para gestionar esta realidad, el `RegisterService` reutiliza el fallback, construido en la Fase 1 para aportar tolerancia a fallos al `StockPricerService`, aplicándolo ahora a la acuñación del SBT con un máximo de **5 reintentos**.

El cliente de Polygon clasifica los errores de la blockchain en función de si son reintentables o no: los mensajes de error fatales del EVM (*execution reverted*, *insufficient funds*, *gas required exceeds allowance*, *nonce too high*) se envuelven como `Abortable`, haciendo que el bucle de fallback se interrumpa inmediatamente sin reintentos adicionales. Los errores transitorios de red, en cambio, se dejan reintentar hasta agotar el límite.

4.2.3.2. La integración con Minas Tirith

Minas Tirith y Pelargir se comunican a través de la interfaz `IdentityBridge`: un puerto de la Arquitectura Hexagonal que Minas Tirith declara como dependencia y Pelargir implementa. El registro de un usuario es un proceso no bloqueante: en cuanto el usuario queda guardado en MySQL, se devuelve el JWT y se dispara la creación de identidad blockchain en segundo plano. Si la acuñación tiene éxito, el registro del usuario se actualiza con los campos blockchain (`SbtTokenID`, `WalletAddr`, `DID`, `PublicKey`, `PrivateKey`); si falla tras agotar los reintentos, el usuario queda operativo en Web2.

La arquitectura completa del módulo y el flujo de registro se ilustran en los siguientes diagramas:

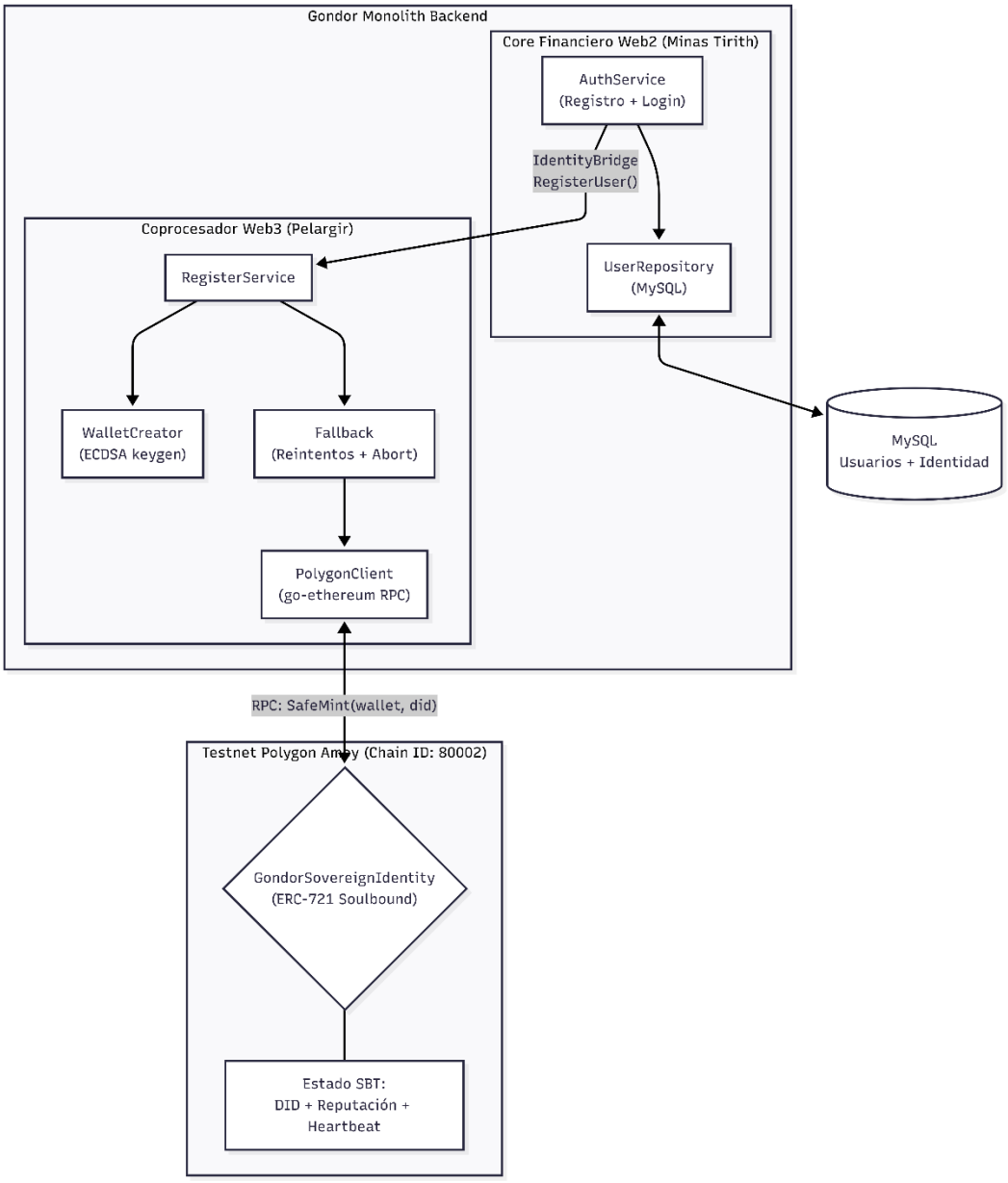


Figura 4-6: Arquitectura de Gondor después de la fase 2.

El flujo completo del proceso de registro con creación de identidad soberana se ilustra en el siguiente diagrama de secuencia:

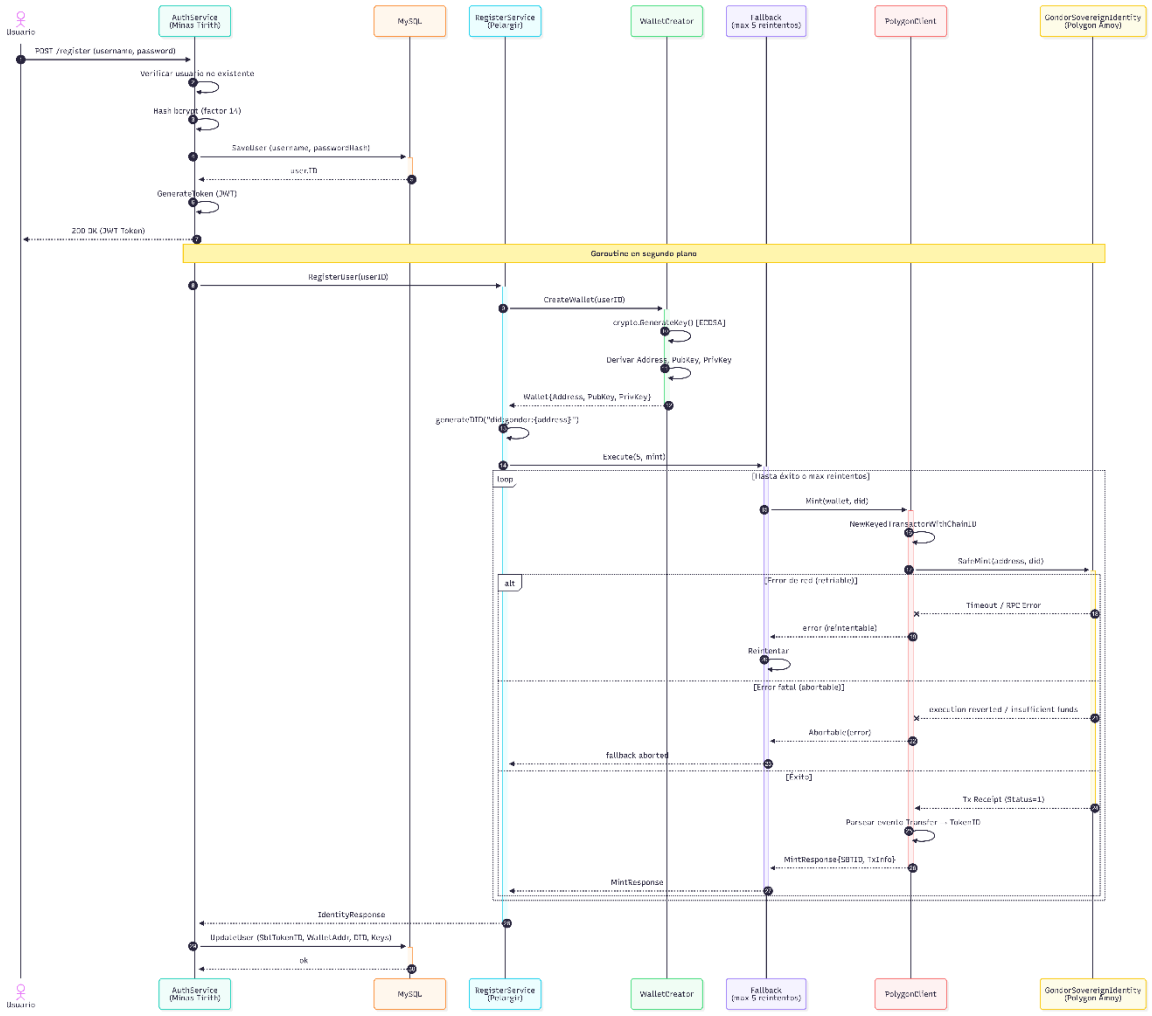


Figura 4-7: Diagrama de secuencia de la creación de la Identidad Soberana.

4.2.4. Evaluación de la fase

4.2.4.1. Pruebas del Smart Contract: validación de la propiedad Soulbound

La suite de tests del contrato `GondorSovereignIdentity` fue desarrollada en TypeScript utilizando Hardhat. Cada test despliega una instancia limpia del contrato, garantizando el aislamiento completo entre casos de prueba.

La propiedad más crítica a verificar era la transferibilidad del SBT. La estrategia de test consistió en desplegar el contrato en un entorno local de Hardhat (EVM en memoria), acuñar un token a una cuenta y después intentar ejecutar `transferFrom` desde esa misma cuenta hacia otra dirección. El test confirma que la transacción revierte con el mensaje "`Gondor: SBT is non-transferable`", demostrando que el override de `_update` funciona correctamente. Para el caso del *Sovereign Handover*, se verifica que incluso

tras el traspaso administrativo de soberanía el nuevo propietario sigue sin poder transferir el token de forma autónoma, confirmando que la protección soulbound es permanente e irrevocable a nivel de contrato.

La integración con el verificador externo se validó con un contrato `MockVerifier` configurable, con el que se comprobó tanto el camino feliz como los casos de rechazo, sentando las bases de la infraestructura de verificación que se ejercitará con pruebas ZK reales en la Fase 3.

4.2.4.2. Validación de la integración backend-blockchain

Desde la perspectiva del backend, la integración fue validada mediante los tests del `AuthService` de Minas Tirith, que utilizan *mocks* generados con `gomock` para la interfaz `IdentityBridge`. Estos tests verifican que el flujo de registro invoca correctamente `RegisterUser` en Pelargir y que el servicio de autenticación maneja adecuadamente tanto los escenarios de éxito como los de fallo de la blockchain.

4.2.4.3. Despliegue en la Testnet de Polygon Amoy

El contrato fue desplegado exitosamente en la testnet Polygon Amoy utilizando Hardhat Ignition, con verificación del código fuente habilitada en PolygonScan (Amoy). La configuración de compilación utilizó Solidity 0.8.24 con la EVM *Cancun*, compilación vía IR habilitada y el optimizador configurado a 200 runs, logrando un balance óptimo entre el coste de despliegue y el coste de ejecución de las funciones del contrato.

4.3. TERCER MOVIMIENTO: LA CERTIFICACIÓN Y VERIFICABILIDAD CRIPTOGRÁFICA

4.3.1. Identificación de requisitos y diseño

La Fase 2 dotó a cada usuario de una identidad soberana materializada en un Soulbound Token, pero esa identidad nació vacía: el SBT certificaba *quién era* el inversor, no *cómo le había ido*. Rellenar esa identidad con datos de rendimiento verificables constituye el verdadero reto del TFM y la aportación central de este trabajo.

El problema puede descomponerse en dos amenazas complementarias que un adversario interno (el operador de la plataforma) podría explotar:

1. **Manipulación retroactiva de datos.** Un administrador de Minas Tirith con acceso a MySQL podría alterar el historial de transacciones de un usuario, borrar una compra desafortunada, inflar un precio de venta, para mejorar artificialmente su rendimiento. Ningún mecanismo de la Fase 1 ni de la Fase 2 lo impide.
2. **Publicación de saldos privados.** Demostrar un rendimiento verificable on-chain requiere, en principio, exponer el detalle de todas las transacciones al contrato inteligente. Esto violaría la privacidad financiera del usuario y haría inviable el sistema por motivos regulatorios y de experiencia de usuario.

Las historias de usuario que guiaron el diseño de esta fase fueron las siguientes:

- **Como usuario**, quiero que cada transacción que ejecuto quede sellada criptográficamente en el momento de su ejecución, de forma que cualquier modificación posterior en la base de datos sea detectable.
- **Como usuario**, quiero que mi nivel de reputación on-chain refleje fielmente mi rendimiento real, sin necesidad de confiar en que el operador no ha manipulado los datos.
- **Como usuario**, quiero que mi rendimiento se demuestre a la blockchain sin revelar el detalle de mis operaciones (importes, símbolos, cantidades).
- **Como usuario**, quiero que si dejo de operar durante un periodo prolongado, mi reputación caduque de forma automática y pública, impidiendo ocultar pérdidas por omisión.

4.3.1.1. Justificación de la privacidad transaccional frente a la alternativa transparente

Una arquitectura **transparente**, en la que cada transacción se anclase públicamente sin ZK-SNARKs, fue descartada por tres razones: (1) publicar el detalle de operaciones (activo, precio, momento) permite la réplica parasitaria y el *front-running*; (2) una arquitectura transparente con oráculos de precio se asemejaría funcionalmente a las plataformas ya descritas en el Estado del Arte, sin cubrir la brecha de privacidad identificada; (3) la publicación permanente de datos financieros detallados en una blockchain inmutable plantea tensiones con el GDPR.

La decisión de preservar la privacidad mediante ZK-SNARKs implica un *trade-off* consciente: la reputación on-chain solo puede reflejar transacciones ejecutadas, no el valor de mercado de las

posiciones abiertas. Para mitigar esta limitación, el sistema implementa la doble métrica de reputación (`reputationLevels` y `reputationLevelsRealized`) y el Heartbeat activo.

Para satisfacer estos requisitos se diseñó la arquitectura *Verify-me*, que combina tres mecanismos criptográficos: , Merkle Batching con anclaje on-chain y pruebas de conocimiento cero (ZK-SNARKs). El siguiente diagrama sintetiza el flujo completo:

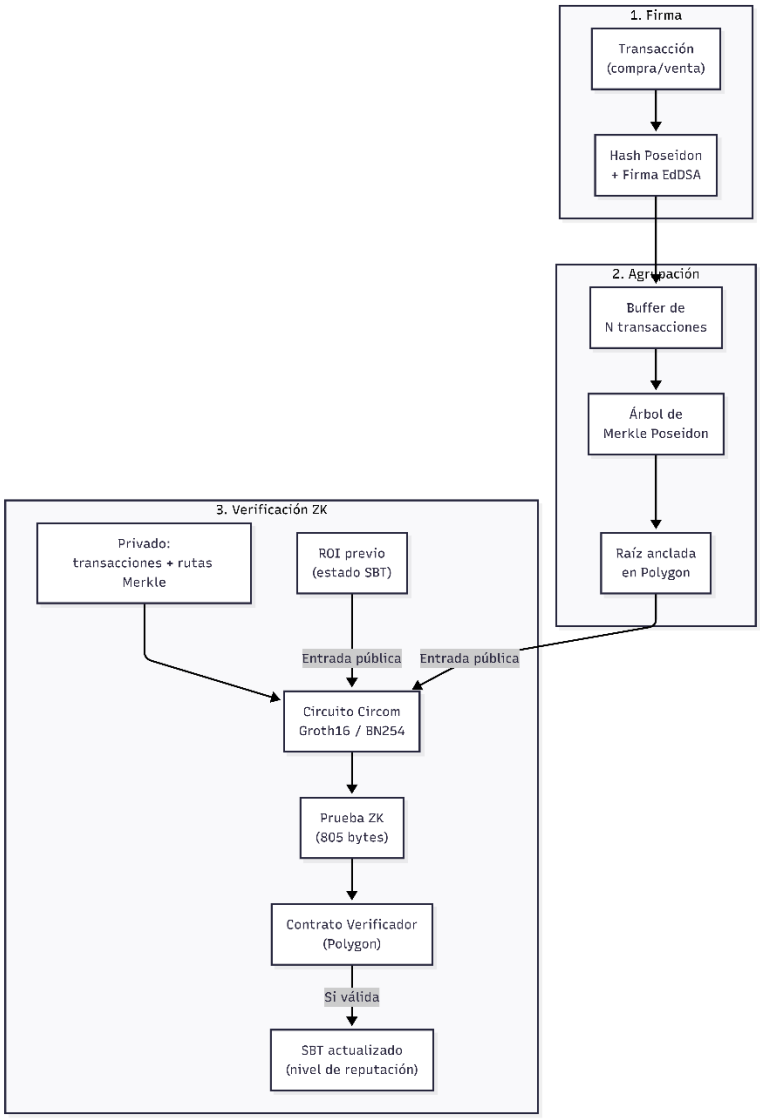


Figura 4-8: Diagrama de flujo de la arquitectura Verify-me.

4.3.2. Recibos firmados deterministas

El primer problema a resolver era garantizar que los datos de una transacción quedaran sellados en el instante de su ejecución, antes de que un administrador pudiera alterarlos. Para ello, Pelargir intercepta cada transacción ejecutada por Minas Tirith de forma síncrona, y la procesa en dos pasos:

1. **Conversión a elementos de campo.** Los valores decimales de la transacción (unidades, precio, importe) se convierten a enteros escalados mediante un factor de precisión de 10^6 (*Fixed-Point Arithmetic*). Esta conversión es necesaria porque la criptografía de campos finitos utilizada tanto en la firma como en el circuito ZK no soporta aritmética de punto flotante. El símbolo del activo se codifica como `big.Int` a partir de sus bytes ASCII, y el timestamp se registra en nanosegundos Unix.
2. **Firma EdDSA sobre Poseidon.** Los ocho elementos de campo resultantes (`tokenID`, `symbol`, `units`, `price`, `amount`, `costBasis`, `txType`, `time`) se pasan a la función hash Poseidon y el hash resultante se firma con la clave privada BabyJubJub del servidor mediante el esquema EdDSA. La firma comprimida de 64 bytes constituye el **recibo determinista**: un compromiso criptográfico irrevocable del contenido exacto de la transacción en el momento de su ejecución. Esta firma se devuelve de forma sincrona al usuario cuando crea la transacción.

El recibo firmado se almacena en el buffer del `TransactionBatcher` y se devuelve a Minas Tirith como `TransactionReceipt` con el hash de la firma en hexadecimal, que Minas Tirith persiste en MySQL. De este modo, el usuario puede consultar en cualquier momento el hash de su recibo y compararlo con el dato anclado en la blockchain.

4.3.3. Merkle Batching y anclaje on-chain

Anclar cada transacción individualmente a la blockchain sería prohibitivo en coste de gas. Pelargir resuelve esto mediante un sistema de **agrupación por lotes** (*batching*) que condensa N transacciones en una única huella criptográfica antes de enviarla a Polygon.

4.3.3.1. El proceso de agrupación

El componente `TransactionBatcher` opera como un *worker* en segundo plano con doble criterio de disparo: se activa cuando el buffer acumula el máximo de transacciones (configurado a 5 para alinearse con el circuito ZK) o cuando transcurre el intervalo máximo de espera. Al activarse, ejecuta el siguiente secuencia:

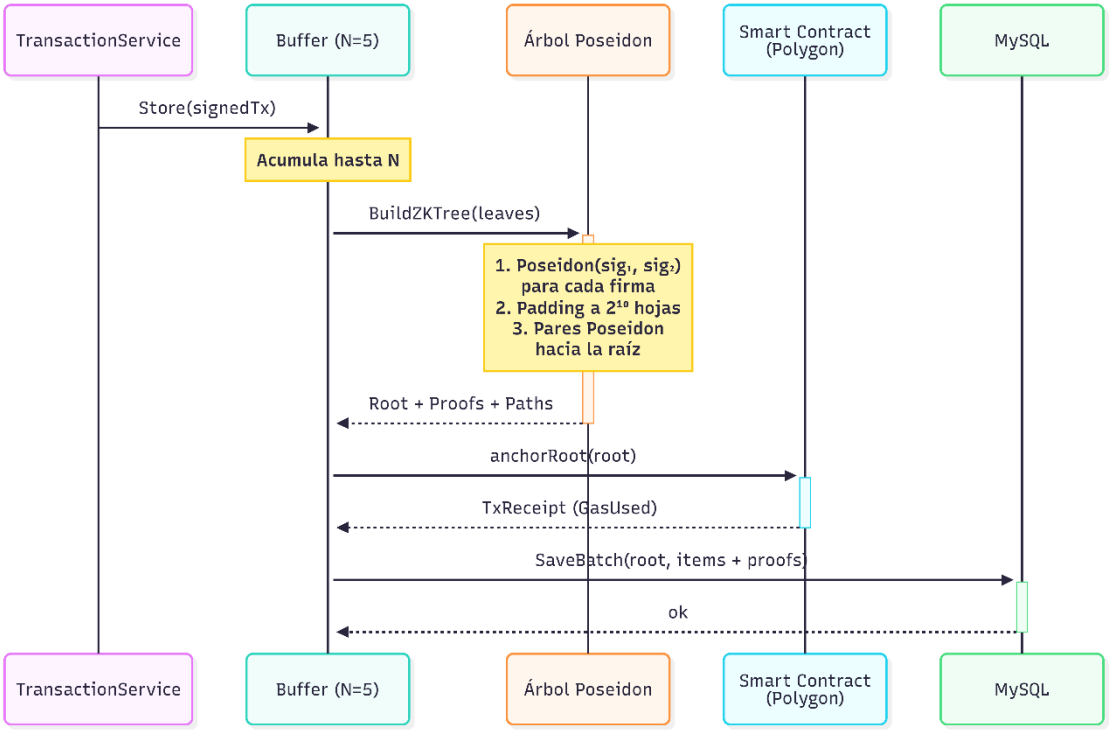


Figura 4-9: Diagrama de secuencia del anclaje de las transacciones de un usuario

1. **Construcción de hojas.** Para cada transacción firmada, la firma de 64 bytes se divide en dos mitades de 32 bytes. Cada mitad se reduce módulo el orden del campo escalar de BN254 y ambas mitades se combinan para obtener la hoja del árbol.

2. **Construcción del árbol.** Las hojas se acomodan (*padding*) hasta completar $2^{10} = 1024$ posiciones (profundidad 10), rellenando las vacías con ceros. Nivel a nivel, cada par de nodos se combina hasta alcanzar una única raíz de 32 bytes. La profundidad de 10 niveles permite hasta 1024 transacciones por lote, dejando margen para escalar sin modificar el circuito.

3. **Generación de pruebas de pertenencia.** Para cada hoja real se registra su *Merkle Proof*: los hashes hermanos en cada nivel del árbol y la dirección del camino (0 = izquierda, 1 = derecha). Estos datos se almacenan en MySQL junto con la raíz del lote y son imprescindibles para que el circuito ZK pueda verificar posteriormente que una transacción pertenece al lote anclado.

4. **Anclaje on-chain.** La raíz se envía al contrato `GondorSovereignIdentity` que la registra en el mapping `canonicalRoots` y emite el evento `RootAnchored`. El fallback, construido desde la Fase 1, se reutiliza para gestionar la comunicación con la blockchain con reintentos y aborto selectivo.

4.3.3.2. La garantía de integridad

Si un administrador alterase una sola transacción en MySQL, por ejemplo, cambiando el importe de una compra, el hash de su firma cambiaría, la hoja del árbol se recalcularía diferente y, al intentar reconstruir el árbol, la raíz resultante ya no coincidiría con la que fue anclada públicamente en Polygon meses atrás. El fraude quedaría en evidencia matemática.

4.3.4. Pruebas de Conocimiento Cero (ZK-SNARKs)

Con las transacciones selladas y ancladas, queda resolver el segundo requisito: demostrar a la blockchain que un inversor tiene un rendimiento concreto sin revelar el detalle de sus operaciones. Para ello se diseñó un circuito aritmético compilado con el framework Circom 2.1.6, verificable mediante el esquema Groth16 sobre la curva elíptica BN254.

4.3.4.1. El problema de la aritmética en campos finitos

Los circuitos ZK-SNARK no operan con números decimales ni con enteros con signo: toda la aritmética se ejecuta en el campo escalar primo de BN254. Esto impone dos restricciones fundamentales que se resolvieron de la siguiente forma:

- **Punto flotante → punto fijo.** Los valores monetarios se multiplican por el factor de escala 10^6 y se truncan a enteros antes de entrar al circuito. Si una acción vale 150,55$ y se compran 2,5 unidades, el circuito trabaja con 150550000 y 2500000. Las divisiones del ROI se sustituyen por multiplicaciones cruzadas, evitando la pérdida de precisión inherente a la división entera.
- **ROI negativo → aritmética modular.** Un ROI negativo no existe como tal en un campo primo. Se mapea a su equivalente positivo sumando el orden del campo: si ROI = −50, el circuito trabaja con $q - 50$, donde q es el primo de BN254 de 254 bits. El contrato inteligente en Solidity interpreta este valor como un entero con signo de 256 bits para recuperar el número negativo original.

4.3.4.2. Estructura del circuito

El circuito principal `BatchTxVerifier(5, 10)`, parametrizado para lotes de 5 transacciones y árboles de profundidad 10, define las siguientes señales:

Tipo	Señales	Descripción
Públicas	`tokenId`, `oldTotalRevenue`, `oldTotalCostInvested`, `oldTotalCostRealized`, `newTotalRevenue`, `newTotalCostInvested`, `newTotalCostRealized`, `root[5]`	El estado previo y posterior del inversor, más las raíces de Merkle de cada transacción. Visibles en la blockchain.
Privadas	`userID[5]`, `symbol[5]`, `units[5]`, `price[5]`, `amount[5]`, `costBasis[5]`, `txType[5]`, `time[5]`, firmas EdDSA, `pathElements[5][10]`, `pathIndices[5][10]`	El detalle completo de las transacciones y sus pruebas de pertenencia Merkle. Jamás salen del servidor.

Tabla 7: Estructura del circuito ZK-SNARK

Para cada una de las 5 transacciones del lote, el circuito ejecuta secuencialmente las siguientes restricciones:

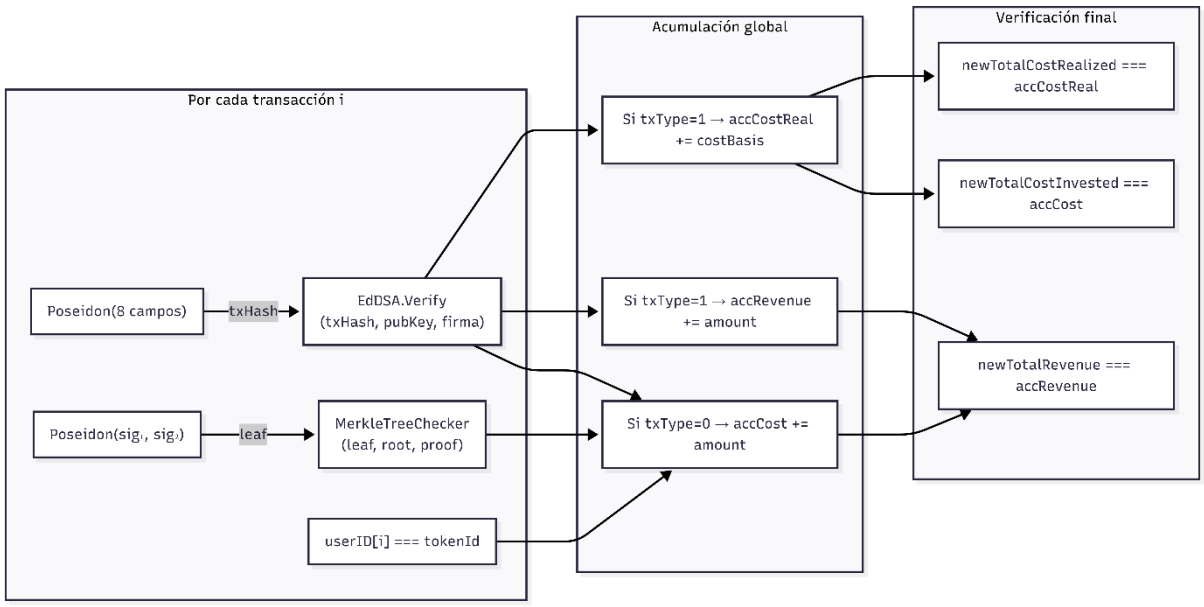


Figura 4-10: Diagrama de flujo de las restricciones que ejecuta el circuito ZK-SNARK.

1. **Verificación de firma.** Se recalcula el hash Poseidon de los 8 campos de la transacción y se verifica la firma EdDSA contra la clave pública del servidor. Esto demuestra que los datos privados no han sido inventados: fueron firmados por el servidor en el momento de la ejecución.

2. **Verificación de pertenencia Merkle.** A partir de la firma, se recalcula la hoja del árbol y se verifica que el camino de hashes hermanos conduce a la raíz pública. Esto demuestra que la transacción pertenece a un lote que fue anclado en la blockchain.
3. **Restricción de identidad.** Se exige que el `userID` de cada transacción coincida con el `tokenId` público del SBT, impidiendo que se utilicen transacciones de otro usuario.
4. **Acumulación matemática.** Los importes se acumulan según el tipo de operación: las compras (`txType=0`) incrementan el coste invertido; las ventas (`txType=1`) incrementan los ingresos y el coste realizado (*cost basis*).
5. **Igualdad final.** El circuito exige que los totales acumulados coincidan exactamente con los valores declarados como señales públicas. Si se intenta declarar un rendimiento diferente al real, el circuito no generará una prueba válida.

4.3.4.3. Orquestación: del circuito al Smart Contract

La generación y verificación de la prueba se orquesta mediante la siguiente secuencia, coordinado por el componente `ReputationUpdater` y el `Saver` dentro de Pelargir:

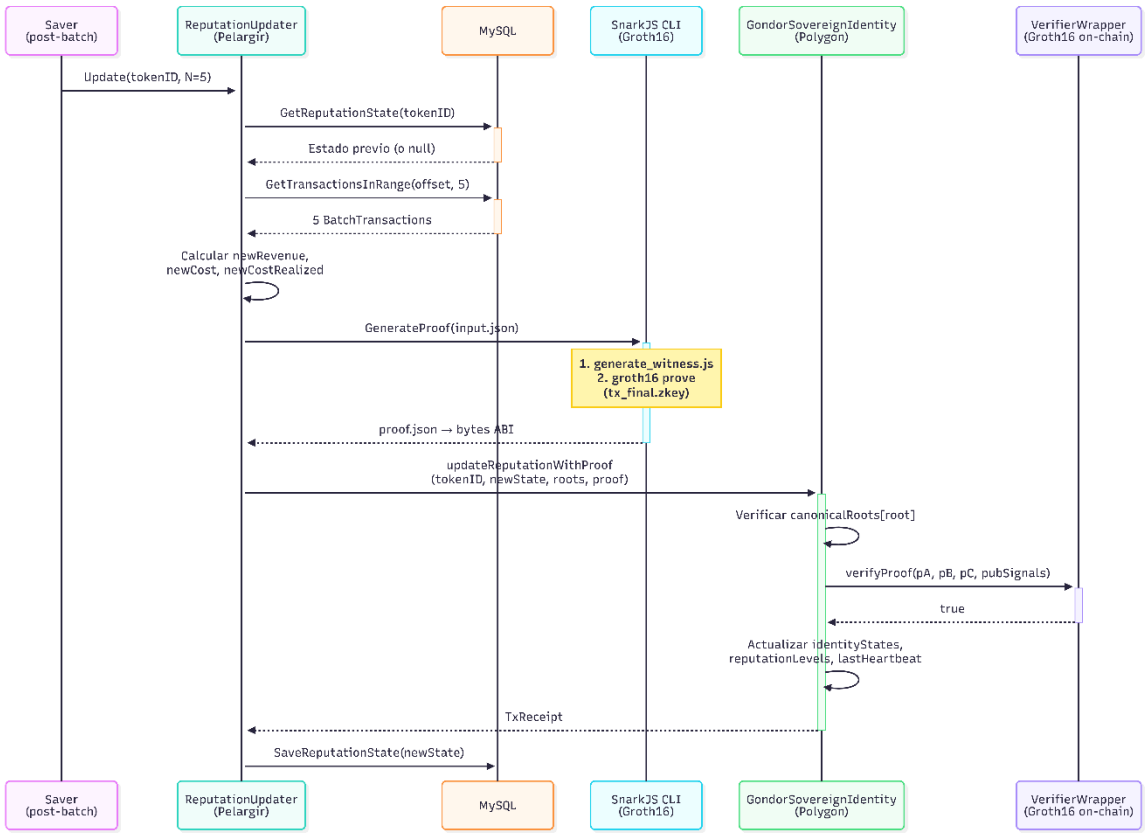


Figura 4-11: Diagrama de secuencia de la generación y verificación de la reputación.

El `SnarkJSProver`, un adaptador de infraestructura, ejecuta el binario `snarkjs` como proceso externo en dos pasos: primero genera el *witness* a partir del WASM compilado del circuito, y después produce la prueba Groth16 utilizando la *proving key* generada durante la ceremonia *trusted setup*. La prueba resultante se codifica en formato ABI para su envío directo como *calldata* al contrato.

En el lado del contrato, la función `updateReputationWithProof` realiza tres validaciones antes de actualizar el estado:

1. Verifica que cada raíz de Merkle proporcionada esté previamente registrada en `canonicalRoots`.
2. Selecciona el verificador correcto del mapping `verifiers[batchSize]`, un diseño modular que permite registrar verificadores diferentes para distintos tamaños de lote sin redesplegar el contrato principal.
3. Delega la verificación criptográfica al contrato `VerifierWrapper`, que desempaqueta la prueba ABI y la pasa al verificador Groth16 generado automáticamente por SnarkJS. La verificación on-chain se ejecuta en milisegundos y consume una fracción del gas de la generación.

Si la prueba es válida, el contrato actualiza el estado financiero acumulado, recalcula los niveles de reputación invertido y realizado, y refresca el `lastHeartbeat`. El SBT del usuario refleja ahora un rendimiento verificado criptográficamente.

4.3.5. El mecanismo de Heartbeat activo

La Fase 2 describió el concepto del Heartbeat como un mecanismo pasivo codificado en el contrato inteligente: si la reputación no se actualiza en 30 días, cualquiera puede revocarla. La Fase 3 complementó este mecanismo con un componente activo en el lado del servidor: el `HeartbeatCron`.

Este *worker* periódico recorre todos los tokens del sistema y, para cada uno, evalúa si la última actualización de reputación supera el umbral configurado. Si el usuario no ha generado suficientes transacciones orgánicas para completar un lote de 5, el cron inyecta **transacciones dummy**, operaciones de compra con importe cero y símbolo `HRTBT`, hasta completar el lote. Estas transacciones neutras no alteran los acumulados financieros (su `amount` y `costBasis` son cero), pero sí se firman, se incluyen en un árbol de Merkle y se anclan en la blockchain, forzando un ciclo completo de generación y verificación ZK. De este modo, el contrato registra un nuevo `lastHeartbeat` y la reputación del usuario sobrevive al periodo de expiración.

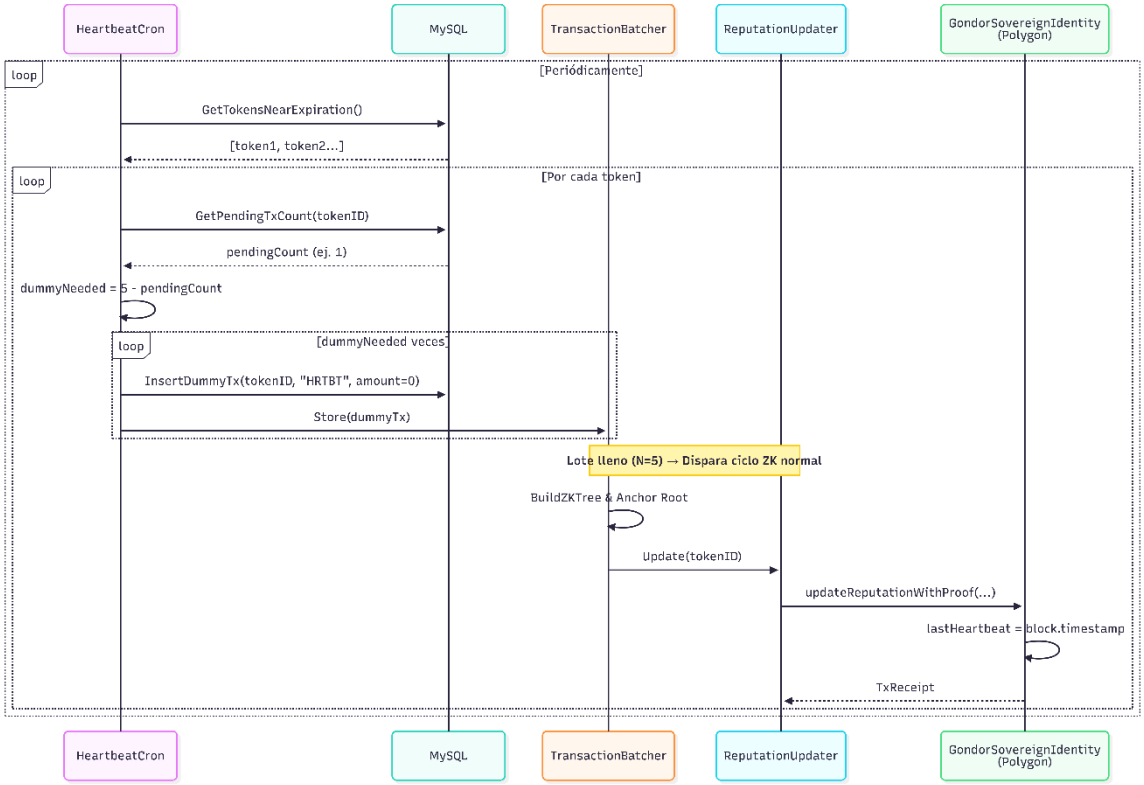


Figura 4-12: Funcionamiento del hearbeat activo

La función esencial del HeartbeatCron no es la degradación activa de la reputación, sino la **inclusión forzada de transacciones orgánicas pendientes** que el usuario podría no tener interés en certificar. Si un usuario ejecuta compras desfavorables y deja de operar, el cron completa el lote con transacciones dummy e inicia el flujo ZK, incrementando el `totalCost` on-chain sin añadir ingresos y reflejando una caída del ROI invertido.

No obstante, el mecanismo presenta una limitación inherente: la reputación on-chain solo puede reflejar **transacciones ejecutadas**, no el valor de mercado de las posiciones abiertas. Un inversor con posiciones en pérdida flotante no verá ese *drawdown* reflejado hasta que cierre las posiciones.

4.3.6. Ciclo de vida de una transacción

Como resultado de la implementación de los tres pilares, cada transacción ejecutada en el ecosistema Gondor atraviesa un ciclo de vida con tres estados observables:

Estado	Significado	Punto de transición
`SIGNED`	La transacción ha sido firmada por Pelargir y devuelta a Minas Tirith	Inmediato tras la ejecución
`IN_MERKLE_TREE`	La transacción ha sido incluida en un árbol de Merkle cuya raíz fue anclada en Polygon	Al completarse el flush del lote
`REPUTATION_UPDATED`	La transacción ha sido utilizada para generar una prueba ZK que fue verificada on-chain	Al completarse la actualización de reputación

Tabla 8: Estados obserbables de cada transacción.

El usuario puede consultar el estado de cada transacción a través de la API REST de Minas Tirith, obteniendo visibilidad completa sobre el progreso de su recibo desde la firma hasta la verificación criptográfica.

4.3.7. Tests de aceptación con Testcontainers

Dada la complejidad extrema de la Fase 3, donde intervienen simultáneamente una base de datos relacional, un servicio de precios concurrente, árboles de Merkle Poseidon, circuitos ZK-SNARK en Circom/SnarkJS y un nodo blockchain con contratos Solidity, los tests unitarios, aunque exhaustivos para cada componente individual, resultaban insuficientes para garantizar la corrección del sistema integrado. Un test unitario del circuito ZK puede verificar que las restricciones se satisfacen para un input dado, pero no puede demostrar que el hash Poseidon calculado en Go produce la misma hoja que el circuito espera en Circom, ni que la prueba generada por SnarkJS es aceptada por el verificador desplegado en la EVM.

Para cubrir este hueco se desarrolló una suite de tests de aceptación end-to-end utilizando la librería Testcontainers, que arranca entornos efímeros completos en Docker.

4.3.7.1. Arquitectura del entorno efímero

Cada ejecución de la suite levanta cuatro contenedores:

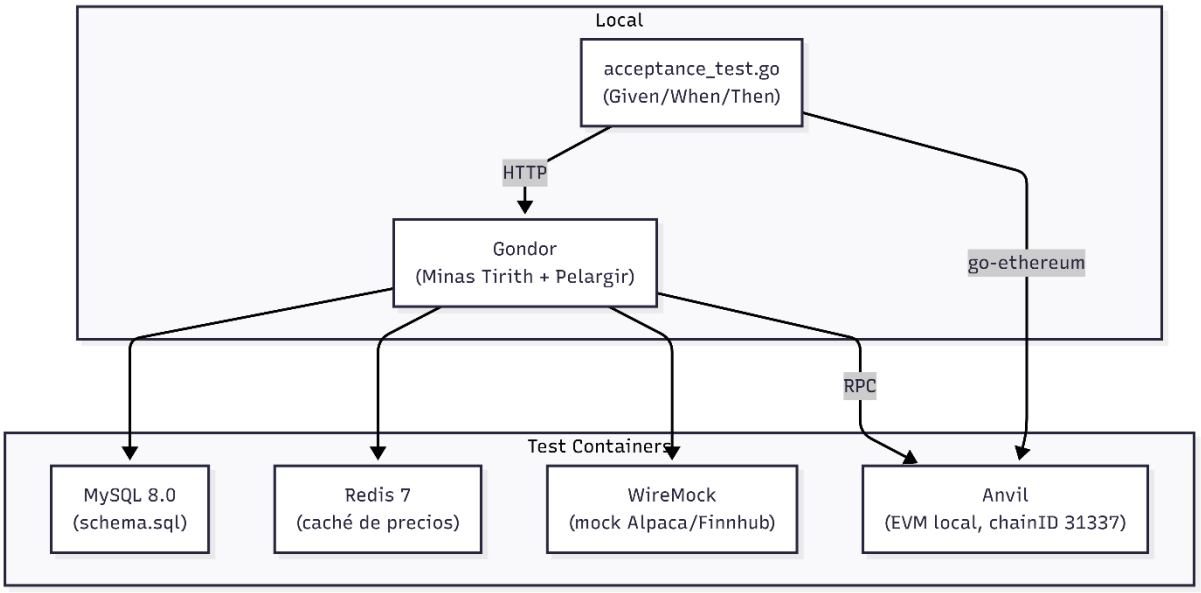


Figura 4-13: Arquitectura de los test de aceptación con TestContainers

- MySQL: Se arranca con el esquema real de producción y se limpia entre tests.
- Redis: Se utiliza como caché de precios para Minas Tirith.
- WireMock: Simula las APIs de proveedores de precios (Alpaca y Finnhub), permitiendo controlar el precio de cada activo en cada momento del test.
- Anvil (Foundry): Un nodo Ethereum local con automine, donde se despliegan el contrato `GondorSovereignIdentity`, el `Verifier` generado por SnarkJS y el `VerifierWrapper`.

Los contenedores se comparten entre todos los tests de la suite (`TestMain`). Con una estructura Given-When-Then, se crearon test de todos los happy paths de todas las historias de usuario, así como algunos casos comunes de fallo. Se trata de una suite de test muy fiable pues es prácticamente la misma configuración que estaría en producción.

4.3.7.2. El simulador de estrés

Complementando los tests deterministas, se desarrolló un ejecutable de simulación que permite estresar el sistema completo: registra N usuarios concurrentemente, genera tráfico HTTP masivo intercalado (compras y ventas aleatorias) y fuerza a los workers de Merkle a empaquetar lotes continuamente mientras la CPU calcula pruebas ZK en segundo plano, validando la estabilidad del protocolo bajo carga sostenida.

4.3.8. Evaluación de la fase

4.3.8.1. Validación criptográfica del circuito ZK

La corrección del circuito se validó en tres niveles progresivos: 1) test unitarios sobre el cliente de Polygon y el constructor de árboles de merkle verificando que se producen pruebas de pertenencia correctas; 2) Los test de aceptación end-to-end con TestContainers; 3) El simulador de estrés.

4.3.8.2. Validación del Smart Contract

La suite de tests del contrato en Hardhat se amplió con casos específicos para la Fase 3, como el anclaje de las raíces Merkle, la actualización de reputación con prueba y el registro de verificadores

4.4.CUARTO MOVIMIENTO: LA INTERFAZ DEL USUARIO

En las fases previas, el ecosistema Gondor materializa la visión central del TFM: un sistema donde la confianza en el operador es sustituida por la verificabilidad matemática. El administrador de Minas Tirith puede poseer acceso completo a la base de datos MySQL sin que ello comprometa la integridad de la reputación, porque cada actualización on-chain ha sido validada criptográficamente por un circuito ZK-SNARK que demuestra, sin revelar datos privados, que el rendimiento declarado es el resultado fidedigno de operar estrictamente con las transacciones que fueron selladas y ancladas tiempo atrás.

La capa de presentación que permite interactuar con el ecosistema híbrido construido en las fases anteriores. Tras haber asegurado en la Fase 3 un motor backend capaz de generar pruebas criptográficas inmutables, la Fase 4 se centra en hacer esta innovación accesible, proporcionando a los inversores interfaces intuitivas para operar y verificar sin perder la confianza criptográfica.

4.4.1. Identificación de requisitos y diseño

Para cerrar el círculo de confianza del ecosistema Gondor, la interfaz debía empoderar al usuario final sin introducir nuevos puntos de centralización que anularan el esfuerzo arquitectónico previo. Los requisitos principales fueron:

- Operativa Web2 sin fricción: Proveer una experiencia de usuario ágil e intuitiva, similar a la de un bróker tradicional (búsqueda de activos, visualización de precios en tiempo real y ejecución rápida de órdenes).

- Desintermediación en la consulta Web3: El usuario debe poder consultar su reputación y el estado de su identidad soberana directamente desde la blockchain, garantizando que el backend no pueda falsear la información mostrada.

4.4.2. La aplicación móvil

La puerta principal de entrada al ecosistema financiero es la aplicación móvil, construida de forma multiplataforma utilizando React Native y el framework Expo. Esta aplicación actúa como el dashboard financiero del usuario, integrando los dos mundos del ecosistema:

- Canal Operativo (Web2): El usuario puede explorar su porfolio y emitir órdenes de compra o venta. Estas acciones realizan llamadas REST estándar a la API de Minas Tirith, beneficiándose de la bajísima latencia.
- Canal de Verificación (Web3): Para mostrar el nivel de reputación del inversor (su Soulbound Token), la aplicación no consulta al backend. En su lugar, utiliza librerías nativas como `viem` para abrir una conexión RPC directa hacia la red de Polygon Amoy. Consulta el contrato inteligente `GondorSovereignIdentity` pasando el identificador del usuario, extrayendo y pintando en pantalla los datos de la fuente inmutable.

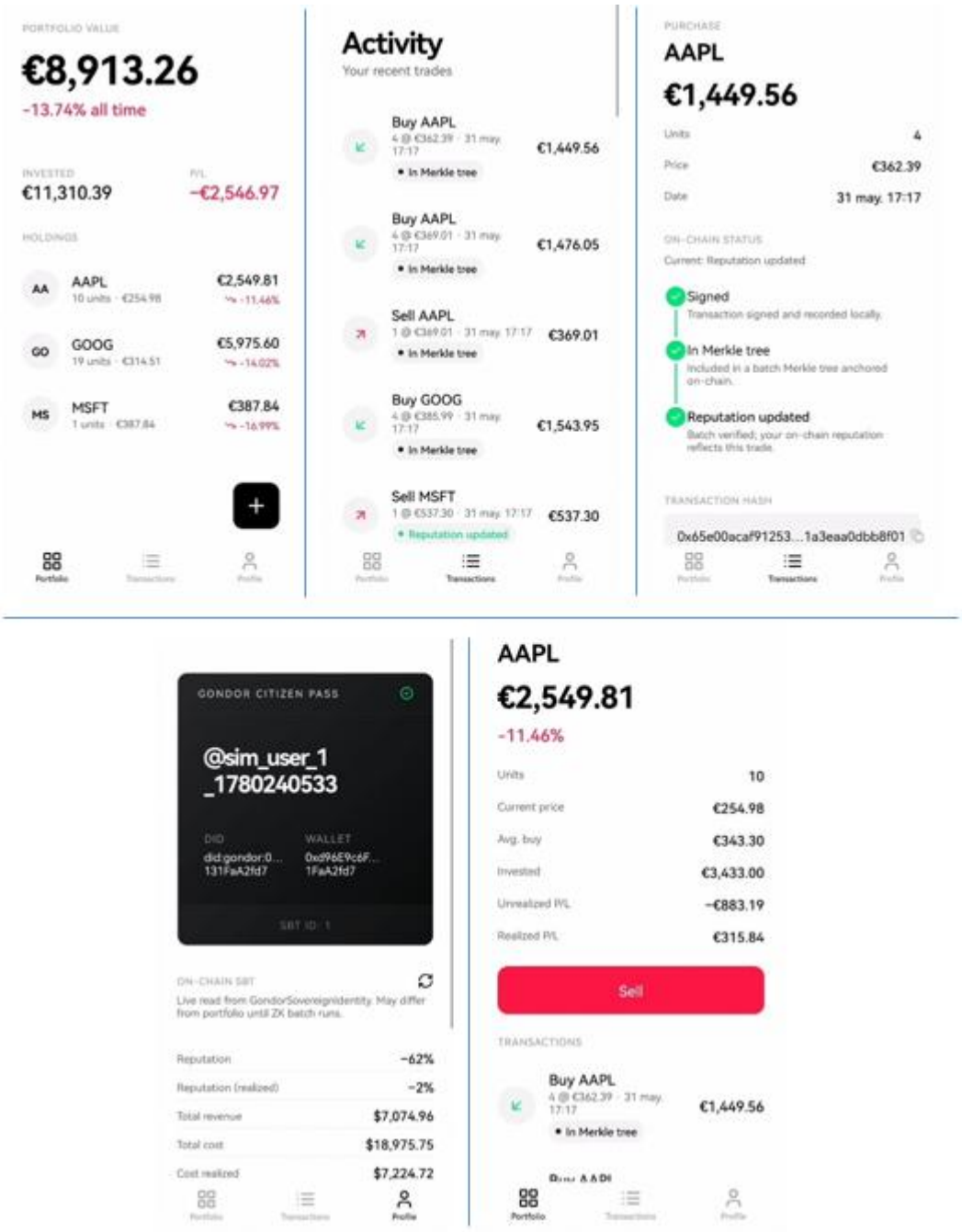


Figura 4-14: Interfaz de usuario de la aplicación Gondor

4.4.3. Evaluación de la fase

A diferencia de las fases anteriores, cuya evaluación fue puramente técnica y algorítmica, la Fase 4 validó la autonomía del usuario y la robustez del modelo híbrido. Se comprobó que la aplicación móvil era capaz de leer el estatus del Soulbound Token y el rendimiento verificado directamente de Polygon, confirmando que la interfaz empodera al usuario frente a una posible interrupción del backend.

Con la capa frontend completada y el ecosistema visualizable e interactivo, Gondor cierra su ciclo de desarrollo logrando solucionar los problemas de opacidad con una herramienta transparente, rápida y criptográficamente demostrable.

4.5.EVALUACIÓN EXPERIMENTAL Y ANÁLISIS DE VIABILIDAD

Esta sección presenta una evaluación cuantitativa del prototipo Gondor, complementando la validación funcional desarrollada en las fases anteriores con métricas de rendimiento, costes de ejecución y tiempos de generación y verificación de pruebas. Las mediciones se obtuvieron instrumentando el sistema durante las sesiones de prueba descritas en las fases 2 y 3, recopilando registros automáticos mediante el sistema de métricas asíncronas implementado.

4.5.1. Entorno de Experimentación

Las pruebas se realizaron sobre un Nodo Local (Anvil) y la propia Testnet Plygoin Amoy. El coste del gas es idéntico entre el nodo local y Amoy el coste computacional on-chain de Polygon no depende de la congestión de la red, sino únicamente de las instrucciones de bytecode ejecutadas. Esta propiedad permite usar el nodo local como banco de pruebas completamente representativo en términos de consumo de gas.

El hardware utilizado para las pruebas fue el siguiente:

- **Procesador**: 11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz
- **Memoria RAM**: 16.0 GB (15.7 GB usable)
- **Tarjeta Gráfica**: Intel(R) Iris(R) Xe Graphics
- **Sistema Operativo**: 64-bit operating system, x64-based processor

4.5.2. Costes de Gas On-Chain

La tabla recoge el consumo medio de gas y las latencias de red experimentadas para cada una de las operaciones on-chain del sistema. Las métricas fueron recolectadas mediante instrumentación automática durante la ejecución de los flujos de la aplicación sobre el nodo local Anvil- A partir de estas métricas, se estima el coste económico real de un despliegue en la red principal (Mainnet) asumiendo gas price = 30 Gwei y precio de mercado de POL = $0,45.

Operación	Gas Medio	Latencia Media (ms)	Coste estimado Mainnet*
`Mint`	213.721	17	$0,00287
`AnchorRoot`	47.680	108	$0,00073
`UpdateReputation`	363.294	158	~$0,00491
Ciclo Completo	**~630.085**	**283**	**~$0,00851**

Tabla 9: Consumo de Gas por Operación On-Chain (Nodo Local)

La variabilidad en las métricas de gas (típicamente pequeña) refleja diferencias en el estado previo de los slots de almacenamiento de la EVM para operaciones como el `Mint`, o variaciones dependientes del tamaño del `calldata` para `UpdateReputation`. En cualquier caso, el coste total de un ciclo de vida completo de un usuario, registro, 5 transacciones y primera actualización de reputación certificada. se sitúa en torno a **$0,0085 en Polygon mainnet**. A efectos comparativos, el mismo ciclo en la red principal de Ethereum (asumiendo 30 Gwei y ETH a $3.500) supondría aproximadamente **$0,22**, validando la elección de Polygon como red de despliegue para un sistema de uso masivo

4.5.3. Latencia de Generación de Pruebas ZK

La generación de pruebas de conocimiento cero es el componente computacional más exigente y el que determina la latencia máxima del ciclo de actualización de reputación

Fase	Tiempo Medio	Desv. Típ.	Mínimo	Máximo
Generación de Witness	921	± 489	304	2.670
Prueba Groth16	13.596	± 6.341	5.118	31.800
Tiempo Total	**14.586**	**± 6.531**	**5.445**	**32.840**

Tabla 10: Latencia en milisegundos de Generación de Prueba ZK

4.5.1. Tamaño de Artefactos Criptográficos

La siguiente tabla recoge el tamaño de los artefactos generados durante el proceso de *trusted setup* y compilación del circuito, relevantes para evaluar los requisitos de almacenamiento y la viabilidad de distribución del sistema.

Artefacto	Tamaño	Descripción
pot17_final.ptau	~151 MB	Ceremonia MPC *Powers of Tau* (fase 1)
tx_final.zkey	~38,5 MB	*Proving Key* específica del circuito (fase 2)
tx_verifier.wasm	~4,2 MB	Binario WebAssembly para generación de witness
verification_key.json	~5 KB	Clave de verificación (embebida en el contrato Solidity)
Proof serializada (calldata)	~805 bytes	Tamaño de cada prueba enviada a la blockchain

Tabla 11: Tamaño de Artefactos del Sistema ZK-SNARK (Groth16/BN254)

Los artefactos de mayor tamaño (`ptau` y `zkey`) son artefactos del operador, gestionados exclusivamente por el servidor de Pelargir y no expuestos al usuario final. La prueba que viaja on-chain (805 bytes) es independiente del tamaño del circuito y una característica inherente del esquema Groth16 sobre BN254: toda prueba de este tipo se serializa en tres puntos de curva elíptica con tamaño fijo.

4.5.2. Evaluación Comparativa con el Estado del Arte

La siguiente tabla consolida una comparativa cuantitativa entre Gondor y las soluciones de reputación Web3 analizadas en el Estado del Arte

Dimensión	**Gondor**	**Sismo**	**Worldcoin**
Gas verificación on-chain	~363K gas	~300K gas	~200K gas

Tamaño de prueba	~805 bytes	~256 bytes	~256 bytes
Esquema criptográfico	Groth16/BN254	Groth16/BN254	Groth16/BN254
Datos privados ocultados	Historial completo (8 campos × 5 tx)	1 nulificador de identidad	1 señal binaria
Actualización dinámica	Batch continuo + Heartbeat	Snapshot puntual	Señal única
Penalización inactividad	On-chain	-	-
Coste ciclo completo (Mainnet L2)	~$0,0085	~$0,0041	~$0,0027

Tabla 12: Comparativa Cuantitativa: Gondor vs. Soluciones del Estado del Arte

El mayor consumo de gas de Gondor respecto a Sismo o Worldcoin es consecuencia directa de la mayor **riqueza semántica** de su circuito: mientras Sismo demuestra un único bit de pertenencia a un grupo y Wordcoin emite una señal anónima, el circuito de Gondor valida la coherencia aritmética de un historial de 5 transacciones con 8 campos cada una. Esta funcionalidad sustancialmente mayor justifica el sobrecoste de gas observado.

5. CONCLUSIONES Y TRABAJO FUTURO

5.1.CONCLUSIONES

El presente Trabajo de Fin de Máster abordó las disfunciones estructurales del *Social Trading* centralizado, secuestro reputacional en silos propietarios, manipulación retroactiva de historiales mediante operaciones CRUD y sesgo de supervivencia institucionalizado, mediante el diseño e implementación del **Ecosistema Gondor**, un prototipo funcional de arquitectura híbrida Web2/Web3

desplegado sobre la testnet Amoy de Polygon. La innovación central reside en el paradigma *Verify-me*: la gestión eficiente del dato se realiza en infraestructura Web2 (Minas Tirith) para garantizar rendimiento operacional, mientras que la certificación matemática de la verdad se ancla en blockchain pública (Pelargir) para garantizar inmutabilidad, verificabilidad autónoma e independencia de plataforma.

5.1.1. Grado de consecución de los objetivos específicos

OE1 Motor de gestión de carteras (Minas Tirith). Implementado en Go con Arquitectura Hexagonal, el motor gestiona operaciones de compra/venta, calcula el ROI en tiempo real y sirve cotizaciones con latencia de microsegundos mediante Redis y el patrón concurrente Fan-in/Fan-out. Las métricas demostraron un cuello de botella del 190% en las llamadas externas, validando la paralelización. Cobertura de testing >85% en servicios, 100% en infraestructura testeable, con validación determinista mediante *golden files*. **Objetivo alcanzado.**

OE2 Identidad descentralizada (SBT). El contrato fue desplegado en Polygon Amoy con verificación en PolygonScan. La propiedad *soulbound* fue validada mediante tests Hardhat que confirman la reversión de `transferFrom`. El registro del usuario genera automáticamente billetera ECDSA, DID y SBT sin fricción, con *Sovereign Handover* para transferencia de custodia real. **Objetivo alcanzado.**

OE3 Certificación criptográfica (recibos + Merkle Batching). Cada transacción se sella con firma EdDSA sobre hash Poseidon (curva BabyJubJub) en el instante de ejecución. Las transacciones firmadas se agrupan en Árboles de Merkle cuyas raíces se anclan en Polygon. Cualquier alteración posterior de MySQL invalida la raíz, haciendo el fraude detectable por cualquier tercero. **Objetivo alcanzado.**

OE4 Circuito ZK-SNARK. El circuito en Circom 2.1.6 (Groth16/BN254) certifica el ROI acumulado sin revelar operaciones privadas. Validado con test unitarios y end-to-end con Testcontainers + Anvil. **Objetivo alcanzado.**

OE5 Mecanismo de Heartbeat. Opera en dos niveles: pasivo (on-chain, revocación automática tras 30 días sin actualización) y activo (off-chain, HeartbeatCron inyecta transacciones *dummy* para forzar ciclo ZK completo en periodos de inactividad). Imposibilita congelar la reputación en un estado favorable. **Objetivo alcanzado.**

OE6 Capa de presentación. La aplicación móvil (React Native/Expo) integra un canal Web2 (API REST, Minas Tirith) y un canal Web3 directo (viem, RPC Polygon) que consulta el SBT sin intermediación del backend, garantizando la desintermediación en la verificación de reputación. **Objetivo alcanzado.**

5.1.2. Contribución principal

La contribución central es la **demostración de viabilidad de la certificación de reputación financiera mediante ZK-SNARKs sobre infraestructura pública, sin exposición de datos privados**. El Ecosistema Gondor prueba que es posible conseguir simultáneamente cuatro propiedades que ninguna solución existente integra: (1) **soberanía** de la reputación mediante SBTs intransferibles, (2) **verificabilidad autónoma** mediante Merkle Batching con anclaje on-chain, (3) **privacidad** del detalle operacional mediante circuitos ZK-SNARK, y (4) **prevención del sesgo de supervivencia** mediante Heartbeat con degradación temporal forzada.

5.2.TRABAJO FUTURO

Integración con brokers reales. El prototipo actual opera con datos introducidos manualmente y precios de fuentes públicas de Market Data. La conexión con APIs de brokers reales (Interactive Brokers, Alpaca Trading, brokers regulados bajo MiFID II) permitiría ingestar órdenes ejecutadas reales en lugar de registradas manualmente, cerrando el circuito de confianza desde la ejecución en mercado hasta la certificación on-chain.

Desacople a microservicios. La Arquitectura Hexagonal y las interfaces Go dejan preparada la evolución hacia microservicios. Los puertos podrían reemplazarse por contratos gRPC (comunicación síncrona) y colas RabbitMQ (notificación asíncrona al batcher), permitiendo escalar independientemente el motor financiero, el coprocesador criptográfico y el generador de pruebas ZK.

Despliegue en mainnet. La migración a Polygon PoS mainnet requiere un análisis de costes de gas reales bajo distintos escenarios de congestión, así como estrategias de optimización como el empaquetamiento de múltiples raíces en una sola transacción.

Trusted Setup distribuido. El esquema Groth16 requiere una ceremonia de *trusted setup* que en el prototipo fue ejecutada por un único participante. Una ceremonia pública distribuida (*multi-party computation*), donde basta un solo participante honesto, eliminaría la asunción de confianza en el generador de claves.

Circuitos recursivos y agregación de pruebas. El diseño actual exige lotes fijos de 5 transacciones. Técnicas como IVC (Nova/SuperNova) permitirían actualizaciones incrementales continuas, y la agregación de pruebas reduciría los costes de gas al combinar múltiples pruebas Groth16 en una única verificación on-chain.

Integración con estándares de identidad. La conexión con Verifiable Credentials (W3C VC) permitiría empaquetar el rendimiento verificado como credencial interoperable con otros ecosistemas de identidad descentralizada (EBSI, eIDAS 2.0), sin que la entidad receptora necesite conocer la existencia del Ecosistema Gondor.

Expansión del modelo de reputación. Incorporación de métricas de riesgo ajustadas (Ratio de Sharpe, drawdown máximo) dentro del circuito ZK-SNARK, y soporte para múltiples clases de activos (criptomonedas, renta fija), enriqueciendo la capacidad expresiva del SBT como certificado de competencia financiera.

Referencias bibliográficas

Abdolmaleki, B., Baghery, K., Lipmaa, H., & Zajac, M. (2017). *A Subversion-Resistant SNARK.*

Al-Qora , L., & Al-Said Ahm, A. (2025). *Modular Monolith Architecture in Cloud Environments: A Systematic Literature Review.*

Apesteguia, J., Oechssler, J., & Weidenholzer , S. (2020). *Copy trading.*

Bhujel, S., & Rahulamathavan, Y. (2022). *A Survey: Security, Transparency, and Scalability Issues of NFT's and Its Marketplaces.*

Brown, K., Woolf, B., & Yoder Foreword, J. (2025). *loud Application Architecture Patterns: Designing, Building, and Modernizing for the Cloud.*

Cockburn, A. (2024). *Hexagonal Architecture.*

Cordeiro Ferreira, R., Dhinavahi, A., Trapmann, R., & van den Heuvel, W.-J. (2025). *Reusability in MLOps: Leveraging Ports and Adapters to Build a Microservices Architecture for the Maritime Domain.*

Cyr, A. (2025). *Daily Proof of Liabilities.*

Delgado-von-Eitzen, C., Anido-Rifón, L., & Fernández-Iglesias, M. J. (2024). NFTs for the Issuance and Validation of Academic Information That Complies with the GDPR.

Delphi Consulting. (2025). *zkVerify: Optimizing ZK Proof Verification At Scale.*

Dubinin, V., Panasenko, Y., Volynets, S., Zhebka, V., & Nishchemenko, D. (2025). *Analyzing the Rationality of using Different Merkle Tree Constructions in Blockchain-based Accounting Systems.*

Elton, E., Gruber, M., & Blake, C. (2015). *Survivor Bias and Mutual Fund Performance.*

Engels, B. (2016). *Data portability among online platforms.*

Eriksson, F., & Hammerman, A. (2025). Authentication with Decentralized Identifiers.

Ethereum Org. (2025). *Merkle Patricia Trie.*

Faustino, D. A. (2021). *Stepwise Migration of a Monolith to a Microservices.*

Giri, S. (2026). *Privacy-Preserving Digital Phenotyping: A Zero-Knowledge Framework for Early Autism Spectrum Disorder Detection.*

Goldwasser, S., Micali, S., & Rackoff, C. (1989). *The Knowledge Complexity of Interactive Proof-Systems.*

Grassi, L., Khovratovich, D., Rechberger, C., Roy, A., & sSchofnegger, M. (2021). *POSEIDON:A New Hash Function for Zero-Knowledge Proof Systems.*

Groth, J. (2016). *On the Size of Pairing-based Non-interactive Arguments.*

Hakia. (2025). *Concurrency Models Compared: Threading, Async/Await, Actor Model & More.*

Handy, J., & Meksi, L. (2025). *Blinded by Bias: The Effects of Hindsight and Survivorship Bias in Managed Futures.*

Jaiman, V., Leonard, P., & Urovi, V. (2022). *User incentives for blockchain-based data sharing platforms.*

Jaiman, V., Pernice, L., & Visara, U. (2022). *User incentives for blockchain-based data sharing platforms.*

Javed , F., & Mangues-Bafalluy, J. (2024). *An Empirical Smart Contracts Latency Analysis on Ethereum Blockchain for Trustworthy Inter-Provider Agreements.*

Javed, F., & Mangues-Bafalluy, J. (2025). *Performance Analysis, Lessons Learned and Practical Advice for a 6G Inter-Provider DApp on the Ethereum Blockchain.*

Karrby, A. (2022). *Evaluating Swift concurrency on the iOS platform.*

Ko, K., Jeong, T., Woo, J., & Won-Ki Hong, J. (2023). *Survey on blockchain-based non-fungible tokens: History, technologies, standards, and open challenges.*

Kumar, H., Venkatram, P., Saran N, Daniel, D., & Joe, P. (2025). *Decentralized digital health ecosystems: a unified architecture for AI-enhanced medical record management.*

Lange, J. (2018). *An empirical study of messaging passing concurrency in Go projects.*

Lee, M.-F., Li, J.-T., Lin, W.-R., & Wang, Y.-H. (2024). Evaluation of Digital Asset Investment Platforms: A Case Study of Non-Fungible Tokens (NFTs).

López-Pimentel, J.-C., & Gonzalez-Sanchez, J. M.-R. (2025). *A Digital Identity Blockchain Ecosystem: Linking Government-Certified and Uncertified Tokenized Objects.*

Malhotra, A. (2025). *Concurrency Patterns in Golang: Real-World Use Cases and Performance Analysis.*

Mubarak, R., Riadi, I., & Sutikno, T. (2026). *Integration of blockchain and cryptographic algorithms for education certification and verification: a systematic literature.*

Muñoz-Tapia, J. L., Belles, M., Isabel, M., & Rubio, A. (2022). *CIRCOM: A Robust and Scalable Language for Building Complex Zero-Knowledge Circuits.*

Nainwal, A., Kamble, A., & Awathare, N. (2025). *A Comparative Analysis of zk-SNARKs and zk-STARKs: Theory and Practice.*

Neal, I. (2016). *Exploring ZK frameworks: Mastermind game in 5 different ZK languages.*

Nunkesser, R. (2022). *Using Hexagonal Architecture for Mobile Applications.*

Pelster, M., & Hofmann, A. (2018). *About the fear of reputational loss: Social trading and the disposition effect.*

Politou, E., Alepis, E., & Patsakis, C. (2021). *The Right to Data Portability under the GDPR: Implementation and .*

Roy, H., & Akand, A. (2026). *Leveraging the Sepolia Testnet for Enhanced Blockchain Transaction Management: An Architectural and Educational Paradigm Shift.*

Sabbag Filho, N. (2025). *Separation of Responsibilities in Practice: Comparison between N-Tier Architecture and Clean Architecture.*

Saputro, D., Setiawan, N., Pratama, A., & Tarigan, A. (2026). *NFT-Based Authentication in Education: ERC-721 Token Use for Moodle LMS Security and Credentialing.*

Satybaldy, A., Subedi, A., & Nowostawski, M. (2022). A Framework for Online Document Verification Using Self-Sovereign Identity Technology.

Singh, P., Sagar, S., Singh, S., Mesfer Alshahrani, H., Getahun, M., & Othman Soufiene, B. (2024). Blockchain-enabled verification of medical records using soul-bound tokens and cloud computing.

Stefanovic, M., Pržulj, Đ., Stefanovic, D., Risti, S., & Capko, D. (2023). *he Proposal of New Ethereum Request for Comments for Supporting Fractional Ownership of Non-Fungible Tokens.*

Virovets, D., Obushnyi, S., Ramskyi, A., & Hulak, H. (2024). Soul Bound Token as Digital Twins in Peer-to-Peer.

Weyl, E., Ohlhaver, P., & Buterin, V. (2022). *Decentralized Society: Finding Web3's Soul.*

Wohlgemuth, V., Berger, E., & Wenzel, M. (2016). *More than just financial performance: Trusting investors in social trading.*

Gondor: Certificación Privada de Reputación Financiera mediante ZK-SNARK en Arquitecturas Híbridas Web2/Web3

Carlos Serrano Gutiérrez

Universidad Internacional de la Rioja, Logroño (España)

Fecha: 28 de Junio de 2026

Resumen

El Social Trading democratiza el acceso a estrategias de inversión expertas, pero adolece de tres fallas estructurales: amplificación algorítmica del riesgo, secuestro reputacional por monopolio de plataformas centralizadas, y sesgo de supervivencia inducido por sistemas CRUD. Este artículo presenta el Ecosistema Gondor, una arquitectura híbrida Web2/Web3 compuesta por un motor de gestión financiera de alto rendimiento (Módulo Web2) y un coprocesador criptográfico ZK-SNARK (Módulo Web3). Mediante firmas EdDSA sobre la curva BabyJubJub con función hash Poseidon, agrupación de transacciones en árboles de Merkle y pruebas Groth16 sobre BN254, se certifica matemáticamente el historial de rendimiento financiero en un Soulbound Token (SBT) desplegado en Polygon, sin revelar activos negociados ni precios de ejecución. La evaluación experimental demuestra un coste de ciclo de vida completo de ~$0,0085 y un tiempo medio de generación de prueba de 14,5 segundos en CPU, con mecanismos programáticos específicos para mitigar el sesgo de supervivencia.

Palabras Clave

Social Trading, ZK-SNARK, Groth16, Soulbound Token, Blockchain, Reputación Financiera, Web3

I. Introducción

El SOCIAL TRADING ha experimentado un crecimiento exponencial durante la última década, facilitando que inversores no profesionales repliquen automáticamente las estrategias de gestores con demostrado historial de rentabilidad. Plataformas como eToro, Darwinex o ZuluTrade han integrado mecanismos de ranking y seguimiento que democratizan el acceso al mercado financiero. Sin embargo, bajo esta aparente transparencia subyacen tres vectores de fallo estructural que comprometen la integridad del sistema en su conjunto.

Amplificación Algorítmica del Riesgo

La exposición continua al rendimiento de otros gestores genera comportamiento gregario sistémico. (Apesteguia, Oechssler, & Weidenholzer , 2020) [1] demuestran empíricamente que los seguidores de estrategias sociales exhiben una correlación de decisiones significativamente mayor que los inversores autónomos, amplificando la volatilidad del mercado de manera procíclica. Este fenómeno se ve agravado por el efecto disposición, la tendencia asimétrica a realizar ganancias prematuramente y mantener pérdidas latentes, cuya intensidad aumenta en entornos de Social Trading con exposición al éxito ajeno, tal y como documentan (Pelster & Hofmann, 2018) [2].

Secuestro Reputacional (Platform Lock-in)

La reputación acumulada por un gestor queda cautiva en las bases de datos propietarias de la plataforma que la aloja. Aunque la normativa GDPR (Art. 20) garantiza el derecho a la portabilidad de datos, las exportaciones en formato CSV carecen de cualquier firma de inmutabilidad que permita a terceros verificar su autenticidad de manera autónoma (Wohlgemuth, Berger, & Wenzel, 2016) [3]. Un gestor que migra de plataforma debe comenzar su historial reputacional desde cero, lo que genera una barrera de entrada artificial que favorece la concentración del mercado.

Sesgo de Supervivencia

Los sistemas de gestión de bases de datos relacionales con operaciones CRUD (Create, Read, Update, Delete) permiten la eliminación selectiva de cuentas con historial negativo. Esta práctica, análoga al sesgo de supervivencia en fondos de inversión descrito por (Elton, Gruber, & Blake, 2015) [4], distorsiona sistemáticamente los rankings públicos al eliminar del conjunto estadístico las estrategias fallidas, sobrestimando artificialmente los retornos medios del ecosistema.

Ante estas tres fallas, la presente investigación formula la

siguiente pregunta de investigación: **¿Es posible diseñar una arquitectura híbrida de gestión de carteras de inversión que sea inmutable, verificable de forma autónoma e independiente de la plataforma Web2, sin sacrificar rendimiento y preservando la privacidad del gestor?**

Para dar respuesta a esta pregunta, se presenta el Ecosistema Gondor, una arquitectura compuesta por dos módulos complementarios: el Módulo de Gestión Financiera Web2 (denominado Minas Tirith en la implementación), desarrollado en Go para alta concurrencia y eficiencia transaccional, y el Coprocesador Criptográfico ZK-SNARK Web3 (denominado Pelargir), que actúa como puente criptográfico entre el mundo de los datos relacionales y la blockchain pública de Polygon [10]. La integración de ambos módulos configura un paradigma de certificación soberana de reputación financiera cuya arquitectura, implementación y evaluación experimental se describen en las secciones siguientes.

II. Estado del Arte

El espacio de la identidad y la reputación descentralizadas ha experimentado una proliferación de soluciones durante los últimos años. Sin embargo, ninguna de las propuestas existentes aborda de manera integral el problema específico de la certificación privada de reputación financiera derivada de transacciones Web2. A continuación se clasifican las soluciones más relevantes y se delimita el vacío tecnológico que cubre Gondor.

Resistencia Sybil e Identidad Social

Gitcoin Passport [11] (sobre Ceramic Network) y Lens Protocol [12] (grafo social en Polygon, ERC-721) representan el estado del arte en identidad descentralizada. Su limitación cardinal para el caso de uso objeto de este estudio radica en que fueron concebidos para validaciones estáticas o de naturaleza social: no disponen de las primitivas algebraicas necesarias para procesar series temporales financieras con privacidad profunda. Lens Protocol, al operar con un grafo completamente transparente on-chain, introduce filtración directa de estrategias de inversión.

Pruebas de Inclusión y Membresía ZK

Worldcoin (mediante el protocolo Semaphore) y Sismo [14] (circuito Hydra-S1) ofrecen mecanismos de prueba de pertenencia a grupos mediante lógica binaria con alta privacidad. No obstante, su semántica está diseñada para verificar señales de membresía discretas, careciendo de las primitivas algebraicas necesarias para procesar flujos asíncronos de datos financieros cuantitativos, tales como calcular ecuaciones de rentabilidad o verificar firmas sobre series de transacciones en circuitos ZK.

Reputación Crediticia DeFi

Spectral Finance [16] (MACRO Score) y ARCx construyen puntuaciones crediticias a partir de transacciones on-chain públicas. Esta dependencia absoluta del registro público expone la estrategia del inversor a ataques MEV (Maximal Extractable Value) y a ingeniería inversa competitiva. Adicionalmente, ninguna de estas plataformas integra datos procedentes de brokers Web2 regulados (MiFID II), lo que limita su aplicabilidad al ecosistema DeFi nativo.

Puentes Criptográficos Web2-Web3

Reclaim Protocol [15] (zkTLS) representa la aproximación más cercana a la problemática de este trabajo, certificando la procedencia de datos Web2 mediante pruebas ZK sobre sesiones TLS. Sin embargo, su alcance se limita a la atestación de la fuente del dato (data provenance), sin gestionar ciclos de vida de reputación, incentivos de teoría de juegos, ni mitigaciones del sesgo de supervivencia.

La Tabla I sintetiza la comparación multidimensional entre las soluciones existentes y el Ecosistema Gondor en las tres dimensiones críticas: privacidad, verificabilidad y portabilidad.

III. Objetivos y Metodología

Objetivo General

Tabla I

Comparación Multidimensional de Soluciones de Identidad y Reputación Descentralizada

Plataforma / Protocolo	Tipo de Reputación	Privacidad	Verificabilidad	Portabilidad
Gitcoin Passport	Identidad genérica, resistencia Sybil	Baja	Off-chain (Ceramic)	Máxima (DID W3C)
Lens Protocol	Capital social, métricas de influencia	Nula	On-chain (ERC-721)	Alta (interna)
Worldcoin (World ID)	Proof of Personhood biométrico	Alta (Semaphore)	Híbrida (Groth16)	Alta (SDK)
Sismo	Credenciales cruzadas	Altísima (Hydra-S1)	On-chain (snarkjs)	Alta (SBT EVM)
Reclaim (zkTLS)	Procedencia datos Web2	Superior (ZK/TLS)	Off→on-chain	Extrema
Spectral / ARCx	Riesgo crediticio DeFi	Nula (public)	Oráculos on-chain	Excelente
Ecosistema Gondor	**Reputación financiera transaccional**	**Alta (Groth16 ZK)**	**On-chain autónoma**	**Alta (SBT Polygon)**

El objetivo general de este trabajo es diseñar y prototipar un paradigma de validación híbrido que desacople la gestión eficiente del dato financiero (responsabilidad del módulo Web2) de su certificación matemática de la verdad (responsabilidad del módulo Web3), habilitando una reputación financiera soberana, inmutable, portable y verificable de forma autónoma.

Objetivos Específicos

Los objetivos específicos que vertebran la investigación son: (1) implementar un mecanismo de identidad soberana basada en Soulbound Tokens (SBT) sobre EVM que garantice la no transferibilidad de la reputación; (2) diseñar un protocolo de sellado criptográfico de transacciones mediante firmas deterministas EdDSA sobre la curva BabyJubJub en el instante de su creación; (3) construir el circuito aritmético ZK-SNARK BatchTxVerifier en Circom capaz de verificar lotes de transacciones financieras con aritmética de campos finitos sobre BN254; (4) implementar contramedidas programáticas, mecanismo Heartbeat, para mitigar el sesgo de supervivencia en el contexto de privacidad transaccional.

Metodología de Desarrollo

La investigación adopta una metodología de diseño iterativo orientada a artefactos (Design Science Research), en la que cada ciclo de desarrollo integra la especificación de requisitos, la

implementación del prototipo y la evaluación cuantitativa de métricas de rendimiento. La selección de las tecnologías clave responde a criterios técnicos precisos: Go (Golang) por sus primitivas nativas de concurrencia (goroutines, canales) y su rendimiento en operaciones I/O intensivas; Solidity por su capacidad de expresar invariantes de inmutabilidad ejecutados por la EVM; Polygon por su compatibilidad EVM completa con costes de gas uno o dos órdenes de magnitud menores que Ethereum L1; y Circom/SnarkJS [9] por su ecosistema maduro de pruebas ZK-SNARK sobre el esquema Groth16 [8], con soporte para las curvas BN254 y BabyJubJub nativas del sistema de pruebas.

IV. Contribución

Desacoplamiento Híbrido Web2/Web3

La arquitectura fundamental de Gondor se basa en el principio de desacoplamiento por responsabilidad. El Módulo de Gestión Financiera Web2 (Minas Tirith) asume la responsabilidad de la eficiencia operacional: gestiona el ciclo de vida completo de órdenes, posiciones y carteras con latencias subcentesimasegundo sobre una base de datos MySQL. El Coprocesador Criptográfico ZK-SNARK Web3 (Pelargir) asume la responsabilidad de la certificación matemática: procesa los recibos generados por el módulo Web2, construye las estructuras criptográficas necesarias y publica las pruebas en blockchain.

Esta separación permite que la experiencia de usuario en el motor de gestión financiera no se vea degradada por la latencia inherente a los procesos criptográficos y las confirmaciones de bloque on-chain, que operan en ventanas de tiempo de decenas de segundos o minutos.

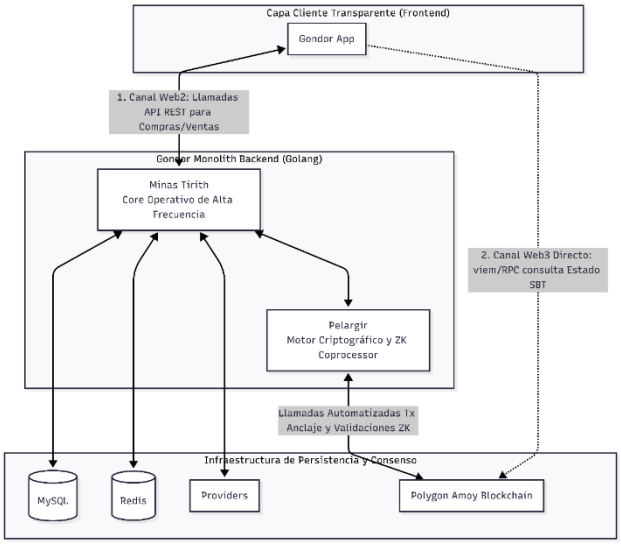


Fig. 1. Arquitectura híbrida del Ecosistema Gondor: desacoplamiento entre el Módulo de Gestión Financiera Web2 y el Coprocesador Criptográfico ZK-SNARK Web3

El Flujo Criptográfico "Verify-me"

El corazón de la contribución técnica reside en el flujo de certificación Verify-Me, articulado en tres etapas secuenciales:

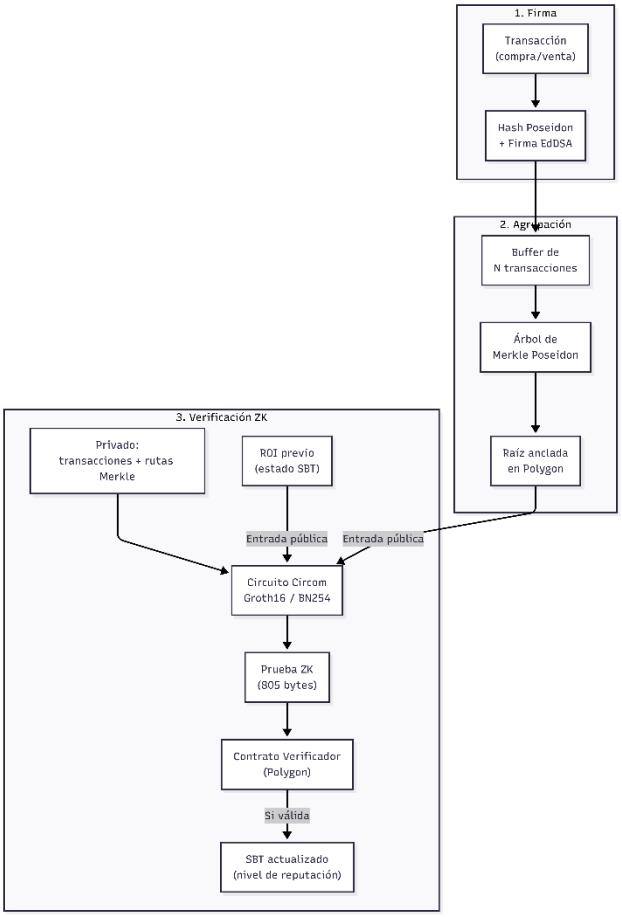


Fig. 2. Flujo de certificación Verify-Me: desde la firma EdDSA de la transacción hasta la actualización del Soulbound Token en Polygon

1. **Firma EdDSA sobre Poseidon**: En el instante de creación de cada transacción financiera, el motor Web2 genera un recibo firmado determinísticamente mediante EdDSA sobre la curva elíptica BabyJubJub [6], empleando la función hash Poseidon [6] como primitiva de hashing. La elección de BabyJubJub y Poseidon no es arbitraria: ambas están optimizadas para operar eficientemente dentro de circuitos aritméticos ZK-SNARK, minimizando el número de restricciones (constraints) necesarias. Cada recibo contiene los campos: identificador de transacción, par de activos, timestamp, precio de entrada, cantidad, dirección pública del gestor y firma EdDSA.
2. **Merkle Batching**: Las transacciones se agrupan en lotes de N=5 elementos. Sobre cada lote se construye un árbol de Merkle con función hash Poseidon de profundidad 10 (capacidad para 1024 hojas). Solo la raíz del árbol (32 bytes) se publica en la blockchain mediante la función anchorRoot del contrato inteligente. Esta estrategia reduce el coste on-chain en un factor proporcional al tamaño del lote, trasladando la mayor parte del coste computacional al servidor off-chain.
3. **Circuito ZK-SNARK BatchTxVerifier**: El circuito aritmético, implementado en Circom 2.0 [9], recibe como entradas privadas los 8 campos de cada transacción del lote y sus correspondientes rutas de Merkle, y como entradas públicas el ROI previo y la raíz de Merkle anclada. El circuito verifica: (a) la firma EdDSA de cada transacción; (b) la pertenencia de cada transacción al árbol de Merkle

anclado; (c) la correcta derivación aritmética del nuevo ROI a partir del estado previo y las operaciones cerradas. La prueba resultante (Groth16 [8]), de 805 bytes, demuestra al verificador que el nuevo ROI publicado en el SBT es correcto respecto a las transacciones comprometidas, sin revelar qué activos se negociaron ni a qué precios.

Mitigación Programática del Sesgo de Supervivencia

La arquitectura incorpora tres mecanismos complementarios para contrarrestar el sesgo de supervivencia en un entorno de privacidad transaccional:

1. Heartbeat Pasivo (On-chain): El contrato inteligente monitoriza el timestamp de la última actualización del SBT. Si transcurren más de 30 días sin que el gestor actualice su reputación, el estado del SBT se modifica automáticamente a inactivo, haciéndolo visible públicamente para cualquier consultor del contrato. Este mecanismo impide que gestores con historial en pérdidas eludan la actualización simplemente abandonando la actividad.
2. Heartbeat Activo (Off-chain): El componente TransactionBatcher del módulo Web2 inyecta transacciones dummy de valor neutro para completar lotes que contienen operaciones cerradas desfavorables pendientes de certificación. Esto impide que el gestor estratégicamente retrase la inclusión de transacciones con pérdidas en los lotes de certificación.
3. Doble Métrica de ROI: El contrato mantiene simultáneamente el ROI Realizado (derivado de posiciones cerradas, objeto de la prueba ZK) y el ROI Invertido (que incluye la exposición latente de posiciones abiertas). La divergencia entre ambas métricas es un indicador robusto de cherry-picking transaccional, es decir, la práctica de certificar únicamente las operaciones exitosas.

V. Resultados

Costes de Gas y Viabilidad Económica

Los experimentos se ejecutaron sobre la red de pruebas Polygon Amoy, proyectando los costes a la red principal de Polygon Mainnet asumiendo un gas price de 30 Gwei y un precio del token POL de $0,45. La Tabla II resume los resultados obtenidos.

El coste del ciclo completo de vida del usuario (~$0,0085) supone una reducción de aproximadamente 26

Tabla II

Consumo de Gas y Proyección de Costes por Operación en Polygon Mainnet

Operación	Gas (medio)	Latencia (ms)	Coste Mainnet*
Mint (Creación SBT)	213.721	17 ms	$0,00288
AnchorRoot (Anclaje Raíz Merkle)	47.680	108 ms	$0,00064
UpdateReputation (Verificación ZK)	363.294	158 ms	~$0,00490
Ciclo Completo	**~624.695**	**~283 ms**	**~$0,00843**

** Gas price = 30 Gwei, precio POL =$0.45*

veces respecto a la proyección equivalente en Ethereum Mainnet (~$0,22 con parámetros similares), validando la elección de Polygon como red de despliegue para el escenario de uso propuesto.

Benchmarks de Generación de Pruebas ZK (Off-chain)

El proceso de generación de pruebas se evaluó sobre hardware de propósito general (Intel Core i7-11ª generación, sin aceleración hardware especializada) para el circuito BatchTxVerifier de N=5 transacciones con árbol de Merkle de profundidad 10. Los resultados, resumidos en la Tabla III, representan la media de 30 ejecuciones independientes.

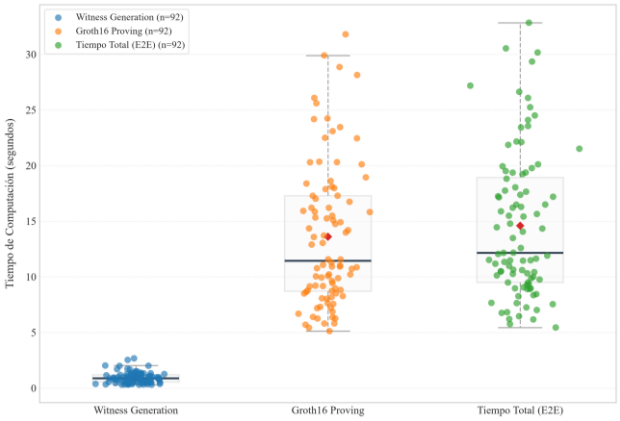


Fig. 3. Distribución de tiempos de generación de prueba Groth16 (N=5 transacciones, 30 ejecuciones, CPU Intel i7-11ª Gen)

Tabla III

Benchmarks de Generación de Prueba ZK-SNARK (Groth16, N=5 transacciones)

Subproceso ZK	Media (ms)	Desv. Típica	Mín. (ms)	Máx. (ms)
Generación de Witness	921	± 489	304	2.670
Generación de Prueba Groth16	13.596	± 6.341	5.118	31.800
Tiempo Total (Prover)	**14.586**	**± 6.531**	**5.445**	**32.840**

Tamaño de los Artefactos Criptográficos

El análisis de los artefactos criptográficos generados revela un perfil de tamaños compatible con los requisitos de despliegue en entornos de producción. El archivo de clave de prueba (.zkey), que encapsula los parámetros del trusted setup específicos del circuito, ocupa 38,5 MB y se mantiene en el servidor Prover. El módulo WebAssembly de generación de witness (.wasm) ocupa 4,2 MB. La prueba serializada que viaja en el calldata de la transacción Ethereum ocupa tan solo 805 bytes, minimizando el coste de gas de la operación de verificación on-chain.

VI. Discusión

Justificación del Sobrecoste de Gas en la Verificación

La operación updateReputationWithProof consume aproximadamente 363.000 unidades de gas, valor superior al observado en protocolos ZK comparables: Worldcoin/Semaphore

(~200.000 gas) y Sismo/Hydra-S1 (~300.000 gas). Esta diferencia no refleja una ineficiencia de implementación, sino una mayor riqueza semántica del circuito. Mientras que Worldcoin verifica una señal binaria de pertenencia a un grupo o Sismo verifica la pertenencia de una credencial a un conjunto, el circuito BatchTxVerifier de Gondor realiza simultáneamente: la verificación de N firmas EdDSA BabyJubJub, la comprobación de N rutas de inclusión en el árbol de Merkle Poseidon de profundidad 10, y la derivación algebraica del nuevo ROI mediante las ecuaciones de rentabilidad ponderadas. Esta complejidad aritmética adicional es el precio necesario para certificar la consistencia del historial financiero completo, no meros bits de membresía.

Análisis del Impacto de la Latencia del Prover

El tiempo medio de generación de prueba de 14,5 segundos en CPU de propósito general podría percibirse como una limitación operacional en un sistema financiero de alta frecuencia. Sin embargo, en el contexto específico de la actualización de reputación, esta latencia es irrelevante para la experiencia de usuario. Las actualizaciones de reputación son procesos asíncronos que se ejecutan en segundo plano a medida que el componente TransactionBatcher acumula lotes completos de operaciones cerradas. La colocación de una orden de inversión no está bloqueada por la generación de su prueba ZK correspondiente. En un despliegue de producción con aceleración hardware (GPU CUDA o FPGA), las latencias observadas podrían reducirse en un orden de magnitud, llevando el tiempo de prueba al rango del segundo.

Limitaciones del Modelo y Contramedidas

La principal limitación del modelo actual es que la reputación on-chain certifica exclusivamente el ROI Realizado, derivado de transacciones ya cerradas. El valor actual de mercado (mark-to-market) de las posiciones abiertas no forma parte del estado certificado en el SBT. Un gestor con posiciones latentes en pérdida que aún no han sido cerradas no verá reflejado este riesgo en su reputación on-chain hasta que cierre dichas posiciones.

Sin embargo, los mecanismos de Heartbeat activo y pasivo limitan de manera robusta el abuso de esta limitación. El Heartbeat pasivo detecta la inactividad total (abandono de cuenta), mientras que el Heartbeat activo fuerza la inclusión periódica de transacciones pendientes. La doble métrica ROI Realizado/ROI Invertido proporciona adicionalmente un indicador cuantitativo de la divergencia entre el historial certificado y la exposición real de la cartera. La integración futura de oráculos de precios Chainlink en el circuito ZK permitiría incorporar el mark-to-market en la certificación, aunque a costa de un incremento significativo del número de constraints del circuito.

VII. Conclusiones

Conclusiones

Este trabajo presenta una demostración empírica de que es posible certificar de forma privada la reputación financiera de un gestor de inversiones de manera inmutable, portable y autogestionada sobre una blockchain pública. La arquitectura híbrida Web2/Web3 propuesta, articulada sobre el desacoplamiento entre el motor de gestión financiera de alta eficiencia y el coprocesador criptográfico ZK-SNARK, resuelve de manera conjunta los tres vectores de fallo identificados en los ecosistemas de Social Trading actuales: la portabilidad soberana de la reputación elimina el secuestro reputacional; la inmutabilidad del SBT y los mecanismos de Heartbeat mitigan el sesgo de supervivencia; y la privacidad matemática del ZK-SNARK preserva la estrategia del gestor frente a competidores.

Los resultados experimentales corroboran la viabilidad comercial del modelo: un coste de ciclo de vida completo de ~$0,0085 en Polygon y una latencia de verificación on-chain inferior a 300 ms sitúan la solución en el orden de magnitud de los costes operacionales de las plataformas de Social Trading Web2 existentes, eliminando así la barrera económica para su adopción.

Trabajo Futuro

Las líneas de investigación futura identificadas son: (1) Circuitos Recursivos mediante IVC (Incrementally Verifiable Computation), paradigmas como Nova o SuperNova que permitirían actualizaciones transaccionales continuas sin la restricción del tamaño de lote fijo, eliminando la latencia acumulada del Heartbeat activo; (2) Ingesta directa mediante zkTLS, integrando Reclaim Protocol para alimentar al coprocesador criptográfico con datos atestados procedentes directamente de brokers regulados bajo MiFID II, cerrando el círculo de confianza desde el origen del dato hasta la blockchain; (3) Trusted Setup Distribuido, implementando una ceremonia MPC (Multi-Party Computation) abierta para la generación de las claves de verificación (.zkey), eliminando el supuesto de confianza en la entidad organizadora del setup; (4) Mark-to-Market on-chain, mediante la integración de oráculos descentralizados de precios (Chainlink) en el circuito ZK para computar el ROI latente de posiciones abiertas y certificar el estado completo de la cartera, no solo el historial realizado.

Referencias

[1] J. Apesteguia, J. Oechssler, and S. Weidenholzer, "Copy trading," Management Science, vol. 66, no. 12, pp. 5608–5622, 2020.

[2] M. Pelster and A. Hofmann, "About the fear of reputational loss: Social trading and the disposition effect," Journal of Banking & Finance, vol. 94, pp. 75–88, 2018.

[3] V. Wohlgemuth, E. S. C. Berger, and M. Wenzel, "More than just financial performance: Trusting investors in social trading," Journal of Business Research, vol. 69, no. 11, pp. 4971–4977, 2016.

[4] E. J. Elton, M. J. Gruber, and C. R. Blake, "Survivorship bias and mutual fund performance," The Review of Financial Studies, vol. 9, no. 4, pp. 1097–1120, 1996.

[5] V. Buterin, "Decentralized society: Finding web3's soul," arXiv preprint arXiv:2205.08373, 2022.

[6] G. Bertoni, J. Daemen, M. Peeters, G. Van Assche, and R. Van Keer, "NIST lightweight cryptography standardization: Permutation-based hash and extendable output functions," NIST, Tech. Rep., 2019.

[7] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, "Bulletproofs: Short proofs for confidential transactions and more," in IEEE Symposium on Security and Privacy (SP), pp. 315–334, 2018.

[8] J. Groth, "On the size of pairing-based non-interactive arguments," in Advances in Cryptology – EUROCRYPT 2016, Lecture Notes in Computer Science, vol. 9666, pp. 305–326, 2016.

[9] Iden3 Team, "Circom: A circuit description language for ZK-SNARK circuits," GitHub Repository, 2021. [Online]. Available: https://github.com/iden3/circom

[10] Polygon Technology, "Polygon PoS: A commit-chain to Ethereum," Technical Whitepaper, 2021. [Online]. Available: https://polygon.technology/papers/pol-whitepaper

[11] Gitcoin Foundation, "Gitcoin Passport: Sybil resistance for web3," Technical Documentation, 2022. [Online]. Available: https://passport.gitcoin.co

[12] Lens Protocol, "Lens Protocol: The decentralized social graph," Technical Whitepaper, 2022. [Online]. Available: https://lens.xyz
[13] Worldcoin Foundation, "World ID Protocol: Proof of personhood," Technical Whitepaper, 2023. [Online]. Available: https://worldcoin.org/blog/worldcoin/worldcoin-white-paper
[14] Sismo, "Sismo Protocol: ZK badges and attestations," Technical Documentation, 2022. [Online]. Available: https://sismo.io
[15] Reclaim Protocol, "zkTLS: Proving web2 data in ZK," Technical Documentation, 2023. [Online]. Available: https://reclaimprotocol.org
[16] Spectral Finance, "MACRO Score: On-chain credit scoring," Technical Whitepaper, 2022. [Online]. Available: https://spectral.finance